



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Learning Human-Robot Co- Manipulation from Human-Human Demonstrations via Vision-Based Diffusion Policy

TESI DI LAUREA MAGISTRALE IN
MECHANICAL ENGINEERING

Author: **Alessandro Gorla**

Student ID: 10727569

Advisor: Prof. Loris Roveda

Co-advisors: Charbel Barsomian

Academic Year: 2025-2026

Abstract

Human–robot collaborative manipulation represents one of the most promising research areas in modern robotics, enabling robots to safely and intuitively interact with humans in shared workspaces. In these scenarios, robots are required not only to execute predefined motions, but also to adapt their behavior dynamically according to human intentions and movements. Achieving smooth and compliant interaction therefore requires the integration of perception, prediction, and advanced control strategies.

This thesis presents a collaborative manipulation framework in which a robotic system generates adaptive motion references from human motion observations. Human body keypoints are extracted through a vision pipeline based on RGB-D sensing and YOLO-based pose estimation, allowing the reconstruction of three-dimensional human motion information. These observations act as conditioning variables for a Transformer-based diffusion model, trained on human–human demonstration data to synthesize future robot trajectories as a sequence of spatial increments.

The predicted trajectories are employed as reference commands for a Cartesian impedance–admittance control layer, which enables compliant and stable physical interaction during collaborative tasks. By combining predictive learning with compliant control, the proposed approach allows the robot to anticipate and adapt to human motion while maintaining safe and flexible interaction behaviour.

The overall system integrates perception, machine learning, and control within a ROS2-based robotic architecture. Real-robot collaborative manipulation experiments showed that the diffusion-assisted pipeline reduced the global RMS interaction force by 37.6% with respect to an admittance-only baseline, highlighting the potential of diffusion-based predictive models to improve anticipation and physical assistance in human–robot collaboration.

Keywords: Human-robot collaborative manipulation; Physical human-robot interaction; Impedance control; Diffusion models; Transformer architecture; Human pose estimation; RGB-D sensing; ROS2.

Abstract in lingua italiana

La manipolazione collaborativa uomo-robot rappresenta una delle aree di ricerca più promettenti nella robotica moderna, poiché consente ai robot di interagire in modo sicuro e intuitivo con gli esseri umani in spazi di lavoro condivisi. In questi scenari, ai robot è richiesto non solo di eseguire movimenti predefiniti, ma anche di adattare dinamicamente il proprio comportamento in funzione delle intenzioni e dei movimenti dell'operatore umano. Un'interazione fluida e cedevole richiede quindi l'integrazione di percezione, predizione e strategie di controllo avanzate.

Questa tesi presenta un framework per la manipolazione collaborativa in cui un sistema robotico genera riferimenti di moto adattivi a partire dall'osservazione del movimento umano. I *keypoint* corporei vengono estratti tramite una pipeline di visione basata su sensori RGB-D e stima della posa mediante YOLO, consentendo la ricostruzione tridimensionale del movimento umano. Queste osservazioni condizionano un modello a diffusione basato su Transformer, addestrato su dimostrazioni uomo-uomo per sintetizzare traiettorie future del robot come sequenze di incrementi spaziali.

Le traiettorie predette sono impiegate come comandi di riferimento per un livello di controllo cartesiano a impedenza-ammettenza, che consente un'interazione fisica stabile e cedevole durante i compiti collaborativi. Combinando apprendimento predittivo e controllo cedevole, l'approccio proposto permette al robot di anticipare e adattarsi al movimento umano, mantenendo un comportamento di interazione sicuro e flessibile.

Il sistema integra percezione, apprendimento automatico e controllo all'interno di un'architettura robotica basata su ROS2. Gli esperimenti su robot reale hanno mostrato che la pipeline assistita dalla diffusione riduce l'RMS della forza globale di interazione del 37.6% rispetto a una baseline basata solo su ammettenza, evidenziando il potenziale dei modelli predittivi a diffusione nel migliorare l'anticipazione e l'assistenza fisica nella collaborazione uomo-robot.

Parole chiave: Manipolazione collaborativa uomo-robot; Interazione fisica uomo-robot; Controllo di impedenza; Modelli di diffusione; Architettura Transformer; Stima della posa umana; Visione RGB-D; ROS 2.

Contents

Abstract	i
Abstract in lingua italiana	iii
Contents	v
1 Introduction	1
1.1 Motivation and Context	1
1.2 Problem Statement	2
1.3 Thesis Objectives and Outline	3
2 State of the Art	5
2.1 Human–Robot Collaboration and Collaborative Manipulation	5
2.1.1 From Industrial Robots to Human–Robot Collaboration	5
2.1.2 Levels of Human–Robot Collaboration	6
2.1.3 Physical Human–Robot Interaction	6
2.1.4 Collaborative Manipulation and Shared-Object Tasks	7
2.1.5 Open Challenges in Collaborative Manipulation	8
2.2 Compliance Control for Physical Human–Robot Interaction	9
2.2.1 From Trajectory Tracking to Interaction Control	9
2.2.2 Impedance Control	9
2.2.3 Admittance Control	10
2.2.4 Variable and Adaptive Compliance Control	11
2.2.5 Stability, Passivity and Redundancy Management	12
2.2.6 Limitations of Reactive Compliance Control	12
2.3 Human Intent Estimation and Prediction	13
2.3.1 Intent Estimation in Human–Robot Collaboration	13
2.3.2 Motion-Based Intent Estimation	14
2.3.3 Force-Based Intent Estimation	15

2.3.4	Posture-Based Intent Estimation and Pose Forecasting	16
2.3.5	Open Challenges in Intent Estimation	16
2.4	Learning-Based Robot Motion Generation	17
2.4.1	Learning from Demonstration	17
2.4.2	Generative Models and Diffusion Policy	17
2.4.3	Diffusion Models for Human–Robot Collaboration	18
2.4.4	Open Challenges in Learning-Based Motion Generation	18
2.5	Human Keypoint Recognition for Collaborative Robotics	19
2.5.1	Human Pose Estimation in Robotics	19
2.5.2	Deep Learning-Based Human Pose Estimation	19
2.5.3	YOLO-Based Pose Estimation	19
2.5.4	Recent 3D Human Reconstruction Approaches	20
2.5.5	Open Challenges in Vision-Based Human Motion Representation . .	20
3	Methodology	21
3.1	Introduction	21
3.1.1	Task Description and Project Objectives	21
3.1.2	Overview of the Proposed Method	22
3.2	RGB-D Data Acquisition and Keypoint Extraction	25
3.2.1	RealSense Recordings	25
3.2.2	YOLO-based Pose Estimation	26
3.2.3	Projection of Keypoints from 2D to 3D	27
3.2.4	Assigning Leader/Follower Roles	28
3.3	Data Preprocessing, Augmentation, and Dataset Construction	28
3.3.1	Signal Organization and Coordinate Representation	28
3.3.2	Geometric Augmentation	29
3.3.3	Conversion from Absolute Coordinates to Delta	30
3.3.4	Handling Missing Values and Validity Masks	31
3.3.5	Episodic Dataset Management	32
3.3.6	Temporal Windowing and Action-Observation Overlap	32
3.3.7	Normalization Strategy	34
3.4	Diffusion Policy for Robot Motion Prediction	35
3.4.1	Action Trajectory Denoising	36
3.4.2	Conditioning on Leader and Follower Motion	37
3.4.3	Transformer Encoder–Decoder Architecture	38
3.4.4	Attention Masks and Validity Masks	42
3.5	Training Procedure	43

3.5.1	Training, Validation and Test Split	43
3.5.2	Batch Construction and Normalization	44
3.5.3	Training Objective	45
3.5.4	Optimization and Model Selection	47
3.6	Low-Level Controller: Impedance–Admittance Control	48
3.6.1	Admittance Layer	49
3.6.2	Cartesian Impedance Tracking	51
3.6.3	Nullspace Control and Torque Safety	52
3.7	Online Execution Architecture	54
3.7.1	Cartesian Projection and Frame Alignment	55
3.7.2	Real-Time Observation Buffer	56
3.7.3	Receding-Horizon Diffusion Prediction	57
3.7.4	Diffusion Increment Transformation and Saturation	58
3.7.5	Multi-Rate Coupling with the Low-Level Controller	58
3.7.6	ROS2 Communication Architecture	59
4	Results	61
4.1	RGB-D Perception Performance	62
4.2	Offline Model Prediction Accuracy	64
4.2.1	Model Evaluation Protocol	64
4.2.2	Prediction Results	65
4.2.3	Qualitative Analysis of Predicted Cartesian Increments	65
4.2.4	Coordinate-Wise Error Analysis	67
4.3	Simulation Results	68
4.3.1	Collaborative Simulation Scenario	68
4.3.2	ROS 2 Execution	69
4.3.3	Robot and Human Follower Trajectory Comparison	70
4.4	Real-Robot Evaluation	73
4.4.1	Experimental Setup	73
4.4.2	Full-Pipeline Execution	73
4.5	Comparative study	78
4.5.1	Experimental Task Definition	78
4.5.2	Force-Based Comparative Metrics	81
5	Conclusion	85
5.1	Conclusions	85
5.2	Main Contributions	87
5.3	Limitations	88

5.4 Future Developments 89

Bibliography 91

A Appendix A 97

A.1 Leader Keypoints Used in the Observations 97

A.2 Training Configuration 98

List of Figures 103

List of Tables 107

Acknowledgements 109

1 | Introduction

1.1. Motivation and Context

Robots are increasingly expected to operate in environments where tasks cannot be fully separated from human activity. In industrial, assistive and service scenarios, many manipulation operations require the physical involvement of both humans and robots, either because the object is too large or heavy to be handled conveniently by a single agent, or because human decision-making and adaptability remain essential during task execution. In this context, physical Human–Robot Collaboration aims to combine the flexibility and perceptual capabilities of humans with the precision, repeatability and load-carrying capabilities of robotic systems.

Collaborative manipulation is a representative example of this problem. When two agents carry or move a shared object, the motion of one participant influences the motion and forces experienced by the other. Humans naturally solve this problem by interpreting body motion, anticipating the partner’s intention and continuously adapting their own behaviour. Reproducing this type of coordination in a robotic follower is challenging, because the robot must not only react to interaction forces, but also generate motion that is coherent with the human partner’s intended movement.

Traditional compliant control strategies, such as impedance and admittance control, are fundamental for safe physical interaction because they allow the robot to respond to external forces and avoid rigid behaviour during contact. However, when used alone, these methods are mainly reactive. The human operator must often physically guide the robot through the task, and this can increase the interaction effort required to complete the manipulation. A more adaptive collaborative robot should instead exploit perceptual information to anticipate the human motion and provide assistance before large interaction forces are generated.

Recent advances in learning-based robot control offer new possibilities for this objective. In particular, diffusion policies have shown strong potential for modelling complex motion distributions and generating short-horizon action sequences from contextual observations.

At the same time, RGB-D cameras and pose-estimation algorithms make it possible to observe human body motion online and extract three-dimensional keypoints that describe the evolution of the human partner’s movement. Combining these elements with compliant low-level control provides a promising direction for human–robot co-manipulation: the robot can use visual information to predict the follower motion that should accompany the observed leader behaviour, while the controller ensures safe and compliant execution during physical contact.

The motivation of this thesis is therefore to investigate whether coordination patterns observed in human–human collaborative carrying demonstrations can be transferred to a robotic follower. Instead of programming a fixed trajectory or relying only on force-based correction, the proposed approach learns from human–human interaction and uses the learned model to support human–robot collaboration. The central idea is that human demonstrations contain useful information about how a follower should move in response to a leader, and that this information can be used to generate predictive motion references for a robot.

1.2. Problem Statement

The specific problem addressed in this thesis is the development of a robotic follower for collaborative manipulation tasks involving a shared object. The human acts as the leader of the movement, while the robot must behave as an adaptive follower, generating Cartesian motion references that are consistent with the observed human behaviour and executable through a compliant controller.

This problem presents several challenges. First, the robot does not have direct access to the human intention. It must infer the future evolution of the task from partial and noisy observations, such as human body keypoints extracted from RGB-D images. These measurements can be affected by occlusions, missing depth values, pose-estimation errors and temporary keypoint losses. Therefore, the learning architecture must be able to exploit visual information while remaining robust to incomplete observations.

Second, the motion to be generated is not an isolated trajectory-tracking problem. In collaborative manipulation, the robot is physically connected to the human through the manipulated object. As a consequence, the generated reference must be compatible with compliant interaction, controller dynamics, workspace limits and safety constraints. A prediction model that performs well offline is not sufficient unless its output can be integrated into a complete robotic control architecture.

Third, there is a domain transition between the data used for learning and the final execution scenario. The available demonstrations are human–human collaborative carrying sequences, where both leader and follower are human participants. During deployment, however, the follower role is assigned to the robot. The thesis therefore investigates whether the follower behaviour learned from human–human demonstrations can be used to generate useful motion references for a robotic manipulator in simulation and on a real robot.

Finally, the goal is not only to reproduce the demonstrated motion geometrically, but also to improve the physical collaboration with the human operator. In a low-level-only admittance controller, the robot motion is mainly driven by interaction forces applied by the human. The proposed approach aims to reduce this dependence by introducing a predictive high-level policy. The effectiveness of the system is therefore evaluated not only through trajectory prediction metrics, but also through force-based comparisons between the diffusion-assisted architecture and an admittance-only baseline.

1.3. Thesis Objectives and Outline

The main objective of this thesis is to design, implement and evaluate a vision-guided human–robot co-manipulation framework that combines RGB-D perception, diffusion-based motion prediction and compliant impedance–admittance control. The proposed system uses human–human demonstrations to learn leader–follower coordination patterns and transfers the learned follower behaviour to a robotic manipulator.

The specific objectives of the thesis are the following. First, a dataset-construction pipeline is developed to convert human–human collaborative carrying demonstrations into temporal sequences suitable for learning. The leader and follower motions are represented through Cartesian displacements, and validity masks are introduced to handle missing or unreliable keypoints. Second, a Transformer-based diffusion policy is trained to predict future follower Cartesian increments from the recent motion of the leader and follower. Third, the learned policy is integrated with a robotic control architecture, where the predicted increments are transformed into reference commands for a compliant impedance–admittance controller. Fourth, the complete system is evaluated at different levels: offline on held-out human–human sequences, in simulation with a robotic follower, and on a real robot during physical human–robot collaboration. Finally, an ablation study compares the diffusion-assisted pipeline with a low-level-only admittance baseline in order to quantify the contribution of the predictive policy to the reduction of interaction forces.

The thesis is organized as follows. Chapter 2 presents the theoretical and methodological

background required for the work, including physical Human–Robot Collaboration, collaborative manipulation, compliant control, human pose estimation and diffusion-based policies. Chapter 3 describes the proposed framework, including dataset construction, preprocessing, model architecture, training procedure, online perception and the integration with the robot controller. Chapter 4 reports the experimental evaluation. The diffusion policy is first assessed offline on unseen sequences, then integrated into a simulated robotic architecture, and finally tested on a real robot. The chapter also includes the ablation study, where the diffusion-assisted pipeline is compared with the admittance-only baseline through trajectory inspection and force-based metrics. Chapter 5 concludes the thesis by summarizing the main findings, discussing the contributions and outlining possible directions for future work.

2 | State of the Art

2.1. Human–Robot Collaboration and Collaborative Manipulation

2.1.1. From Industrial Robots to Human–Robot Collaboration

The role of robots in manufacturing has traditionally been associated with the execution of repetitive and well-defined operations. In structured industrial environments, robotic systems can perform programmed tasks with high accuracy, speed and repeatability. However, this traditional model becomes less effective when production systems require flexibility, frequent task changes and adaptation to uncertain conditions. Villani *et al.* [46] highlight this contrast between the performance of conventional industrial robots in structured scenarios and their limited adaptability in dynamic workspaces.

Human–Robot Collaboration emerged from the need to combine the complementary capabilities of humans and robots. Human operators contribute judgement, perception, dexterity and the ability to react to unexpected events, while robots provide precision, strength and repeatability. The collaborative paradigm therefore shifts the role of the robot from an isolated automation device to a system that can operate in proximity to human workers and contribute to shared tasks [30].

This evolution is also connected to broader changes in industrial production. The Industry 5.0 perspective extends the technological focus of Industry 4.0 by emphasizing human centrality, sustainability and resilience [8]. Within this context, collaborative robots are often discussed not only in terms of productivity, but also as tools for reducing physical workload and improving working conditions in repetitive or ergonomically demanding operations [28].

2.1.2. Levels of Human–Robot Collaboration

Human–robot collaboration can be classified according to the level of proximity, coordination and physical interaction between the two agents. Bauer *et al.* [47] describe a progression from traditional separated robot cells to more integrated forms of interaction, including coexistence, synchronization, cooperation, and collaboration, as depicted in Figure 2.1.

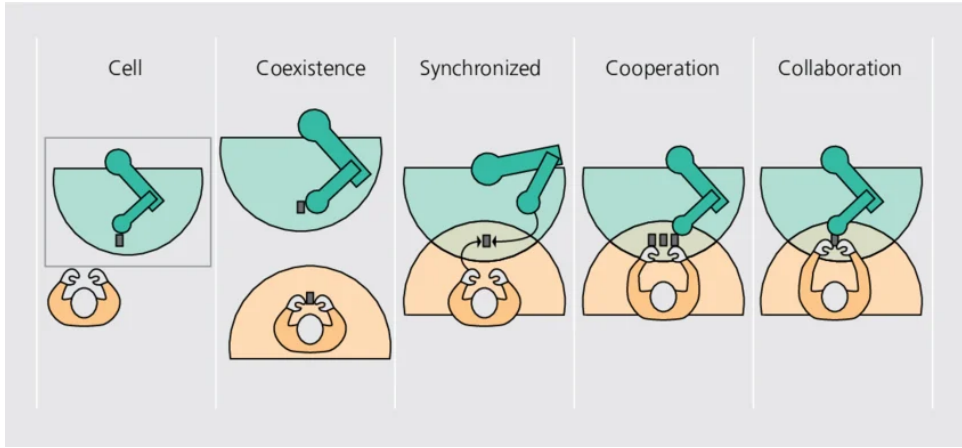


Figure 2.1: Main levels of human–robot collaboration: coexistence, synchronization, cooperation and collaboration. Reproduced from Bauer *et al.* [47].

In coexistence, humans and robots share the same environment but perform independent activities. Synchronization introduces temporal coordination, since the agents operate in the same workspace but at different times. Cooperation involves a shared process with divided tasks, while collaboration represents the most integrated case, where humans and robots work simultaneously toward a common objective. In this last configuration, physical contact may occur directly or through a shared object, making the interaction more demanding from both the control and perception perspectives.

2.1.3. Physical Human–Robot Interaction

Physical Human–Robot Interaction extends collaborative robotics to situations in which contact forces and physical coupling are part of the task. Rather than considering contact only as a safety issue, pHRI studies how robots can exchange forces with humans in a controlled and useful way. De Santis *et al.* [7] provide an early overview of this field, framing pHRI as a research area that includes collision safety, interaction control and physically assisted tasks.

Safety remains a fundamental requirement in collaborative robotics. Standards such as

ISO 10218 and ISO/TS 15066 define collaborative operation modes including safety-rated monitored stop, hand guiding, speed and separation monitoring, and power and force limiting [17, 18]. These requirements are often discussed together with usability and productivity, since overly restrictive safety strategies can limit the practical effectiveness of collaborative applications [26, 46].

Beyond safety, physical collaboration requires the robot to interpret the human partner's behavior and adapt its actions during the task. Ajoudani *et al.* [2] identify perception, control, adaptation, and human intent understanding as central components of advanced Human–Robot Collaboration. This view moves the focus from simple spatial proximity toward systems able to coordinate with human actions in changing interaction conditions.

2.1.4. Collaborative Manipulation and Shared-Object Tasks

Collaborative manipulation is a class of pHRI in which the human and the robot simultaneously act on the same object or tool. In this type of task, the two agents are mechanically coupled through the manipulated object, and the motion of one partner directly affects the interaction experienced by the other. For this reason, collaborative manipulation is usually treated as one of the most demanding forms of HRC, since it requires both physical compliance and coordination between agents [2, 46].

A representative case is collaborative carrying, where two agents jointly transport an extended object such as a table, a beam or a bulky load. In human-human teams, coordination often emerges naturally through motion and force exchange. In Human–Robot Co-Manipulation, instead, the robot must infer how to contribute to the shared movement while avoiding behaviors that oppose the human partner or destabilize the object. Mielke *et al.* [31] address this problem by studying the ambiguity of desired object motion during shared-object transportation.

Several works investigate how the robot can estimate the evolution of the collaborative task from observed human behavior. Motion history has been used to predict short-term human intent and generate robot trajectories in physical co-manipulation [43]. Other approaches combine online intent estimation with constraint-aware control, taking into account human motion, robot kinematics and task constraints [38]. These contributions show how collaborative manipulation requires the integration of prediction and control, while also highlighting the difficulty of modelling human intention in a structured and generalizable way.

Collaborative manipulation is often analysed through leader-follower dynamics. In this interpretation, one partner mainly initiates or guides the movement, while the other adapts

to maintain coordination. Kropivšek Leskovar *et al.* [23] show that these roles can emerge naturally in physical collaboration between human dyads, even without explicit communication. Their results also indicate that the distribution of influence is not necessarily fixed, since the contribution of each partner can vary during the task.

Human-human collaboration has therefore been used as a reference for modelling coordinated behavior in human-robot co-manipulation. Data collected from human dyads can reveal motion and force patterns that are difficult to design analytically. In planar manipulation of extended objects, for example, human-human demonstrations have been used to identify coordination strategies and leader-induced cues for lateral motion [31].

Learning from Demonstration provides a broader framework for using examples to acquire robotic behaviors. Demonstration-based methods reduce the need for manually programmed rules and allow coordination patterns to be learned from observed task executions [3, 33]. However, these methods remain affected by limitations related to data quality, generalization and robustness to variability across users and task conditions.

2.1.5. Open Challenges in Collaborative Manipulation

The literature on collaborative manipulation identifies several recurring challenges. Human intention is often ambiguous, since similar initial motions can correspond to different future goals. Human behavior is also variable across operators, task phases and object configurations, making it difficult to design fixed interaction strategies that remain effective in different conditions [2, 38, 43].

A second challenge concerns the connection between prediction and physical execution. Intent estimation and learning-based motion generation can provide useful references or action sequences, but physical collaboration still requires the robot to regulate contact forces and remain compliant during interaction. Reviews on pHRI and collaborative robotics therefore treat perception, prediction and control as complementary components rather than isolated problems [2, 7, 46].

Overall, the literature shows a progression from classical industrial robotics toward collaborative systems capable of sharing tasks, exchanging forces and adapting to human behavior.

2.2. Compliance Control for Physical Human–Robot Interaction

2.2.1. From Trajectory Tracking to Interaction Control

Classical robot control has traditionally focused on accurate trajectory tracking in joint or Cartesian space. In this formulation, the robot is commanded to follow a desired motion under known kinematic and dynamic assumptions, which is effective in free-space motion and structured environments. Standard robotics textbooks describe this as the conventional basis of robot motion control [40]. However, this approach becomes limited when the robot interacts with a human operator or with uncertain contact conditions, since small deviations from the desired trajectory may generate undesired interaction forces.

One of the early attempts to address interaction with the environment is hybrid position/force control. Raibert and Craig [6] proposed a framework in which the task space is divided into directions where position is controlled and directions where force is regulated. This approach is well suited to tasks with known contact geometry, such as surface following or assembly, but becomes more difficult to apply when contact directions and interaction conditions change during the task.

Physical Human–Robot Interaction introduces contact situations that are less predictable than those typically considered in classical industrial robotics. For this reason, compliant control approaches have become central in pHRI, since they regulate the mechanical response of the robot during interaction rather than enforcing rigid trajectory tracking. Reviews on Human–Robot Collaboration identify compliance as a key requirement for safe and effective physical interaction, especially when the robot shares the workspace or a manipulated object with a human partner [2, 46].

2.2.2. Impedance Control

Impedance control is one of the foundational approaches to robotic interaction. Rather than controlling position and force as independent quantities, the robot is commanded to exhibit a desired dynamic relationship between motion and interaction forces. Hogan [14] introduced this formulation, showing how the robot behavior can be shaped through virtual mechanical parameters such as stiffness, damping and inertia.

The extension of control to the Cartesian operational space further supported the development of interaction control. In many physical interaction tasks, contact occurs at the

end-effector or through a manipulated object, making it more natural to define the desired behavior directly in task space. Khatib [22] provided a framework for controlling robot motion in operational space, which later became an important reference for Cartesian interaction control.

Impedance control and its variants have been widely adopted in robotic manipulation, physical Human–Robot Interaction, rehabilitation, assistive robotics and legged locomotion. A broad review of this field is provided by Abu-Dakka and Saveriano [1], who organize the main developments of impedance-based methods and discuss their applications in tasks requiring controlled physical interaction.

A key limitation of impedance control is that the resulting behavior depends strongly on the selected virtual parameters. High stiffness improves trajectory tracking but reduces compliance, while lower stiffness allows a softer response but may decrease accuracy. In Human–Robot Interaction, a fixed set of stiffness, damping and inertia parameters may be suitable only for specific task phases, while different contact conditions or human behaviors may require different mechanical responses.

2.2.3. Admittance Control

Admittance control follows the opposite causal structure with respect to impedance control. Instead of generating forces or torques to reproduce a desired mechanical behavior, the measured interaction force is used as the input of a virtual dynamical system that produces a motion reference. This reference is then tracked by an inner position or velocity controller. Keemink *et al.* [20] describe admittance control as one of the main approaches for implementing compliant behavior on robotic systems where reliable low-level motion control is available.

This architecture is particularly suitable for platforms where direct torque control is limited or not accessible. In physical Human–Robot Interaction, admittance control allows the robot to modify its commanded motion according to the forces applied by the human user. Its applications include collaborative assembly, hand-guiding, rehabilitation robotics and assistive manipulation [20].

The same review also highlights several practical limitations of admittance control. Since the interaction force is directly used to generate motion references, the behavior of the controller depends on force sensing quality, filtering choices, delays, and parameter tuning. Poorly selected admittance parameters can reduce transparency or compromise stability, especially in the presence of noisy measurements or rapidly changing interaction conditions [20].

2.2.4. Variable and Adaptive Compliance Control

Fixed impedance and admittance controllers rely on constant virtual parameters. This assumption can be restrictive in physical interaction, where the desired robot behavior may vary across task phases, contact conditions, and human actions. Variable compliance approaches address this limitation by adapting stiffness, damping, inertia or admittance parameters during the task.

A broad organization of this field is provided by Abu-Dakka and Saveriano [1]. Their review on Variable Impedance Control and Learning distinguishes between Variable Impedance Control, Variable Impedance Learning, and Variable Impedance Learning Control, as shown in Figure 2.2. This taxonomy separates approaches based on manually designed adaptation laws from methods that learn impedance parameters from demonstrations, interaction data or task performance.

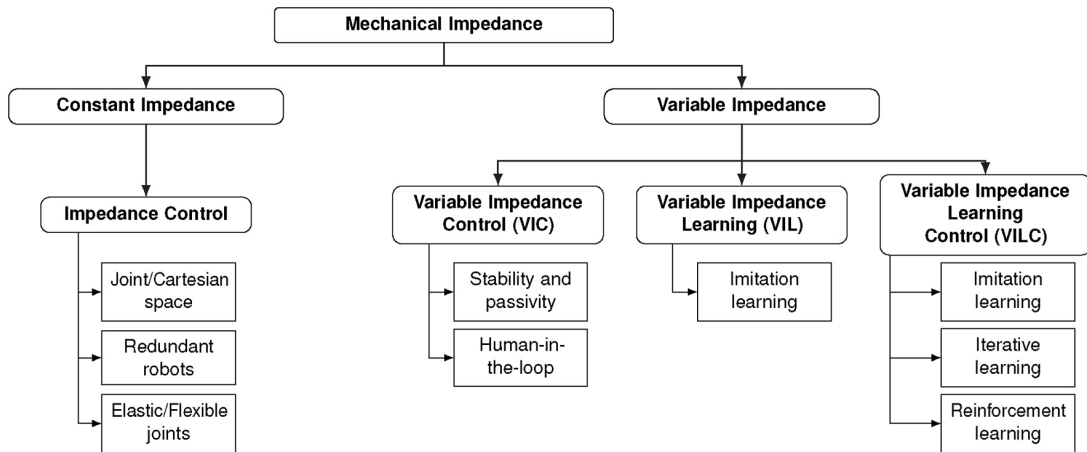


Figure 2.2: Taxonomy of variable impedance control and learning approaches. Adapted from Abu-Dakka and Saveriano [1].

Learning-based adaptation has been explored as a way to regulate compliance under uncertain interaction dynamics. Roveda *et al.* [34] propose a model-based reinforcement learning variable impedance controller, where the interaction dynamics are learned online and used within an optimization framework to adapt the robot compliance during collaborative tasks. In a later work, a Q-learning-based model predictive variable impedance controller is introduced for physical Human–Robot Collaboration, combining intention recognition with adaptive impedance regulation [35].

Variable admittance control follows a similar principle, but acts on the parameters of the admittance model rather than on impedance parameters. Landi *et al.* [25] propose an admittance parameter adaptation strategy for pHRI in which deviations from the nominal

behavior are detected and the controller parameters are modified while preserving passivity. This contribution addresses the trade-off between improving interaction transparency and maintaining stable behavior during online parameter variation.

2.2.5. Stability, Passivity and Redundancy Management

Stability is a recurring issue in compliant control because the robot interacts with humans and environments whose dynamics are uncertain and time-varying. Passivity-based methods address this problem by analysing the energetic behavior of the interactive system and limiting the energy that the controller can inject during contact.

The concepts of Passivity Observer and Passivity Controller were introduced by Hanaford and Ryu [11]. Their framework monitors the energy exchanged during interaction and modifies the control action when the system risks becoming non-passive. This idea has influenced several later approaches for stable interaction control, especially in scenarios involving haptic devices, teleoperation and physical robot interaction.

Energy-based formulations have also been applied to variable stiffness control. Ferraguti *et al.* [9] use energy-tank methods to preserve passivity when impedance parameters are changed online. This addresses a specific limitation of variable compliance: adapting stiffness or damping can improve interaction behavior, but parameter variation may also introduce energy into the system if not properly regulated.

More recent work has continued to investigate passivity-preserving strategies for compliant control. Stability and passivity issues in variable admittance control are discussed in relation to the trade-off between transparency, responsiveness, and safe interaction [49]. These studies show that online adaptation of compliance parameters requires additional mechanisms to avoid unstable behavior.

Redundancy management represents another relevant aspect in collaborative manipulation, especially for robots with more degrees of freedom than those strictly required by the Cartesian task. Ficuciello *et al.* [10] combine variable impedance control with redundancy resolution, showing how the robot null space can be exploited to satisfy secondary objectives while preserving the primary task. Such objectives may include joint-limit avoidance, singularity avoidance, posture optimization and manipulability improvement.

2.2.6. Limitations of Reactive Compliance Control

Impedance and admittance control provide fundamental tools for regulating physical interaction, but they mainly define how the robot reacts to forces or deviations from a

desired motion. In broader reviews of Human–Robot Collaboration, compliant control is usually discussed together with perception, adaptation, and human intent understanding, rather than as a complete solution on its own [2].

This limitation becomes more evident in collaborative manipulation, where the robot is expected not only to react to the human partner, but also to anticipate the future evolution of the task. Hoffman *et al.* [13] identify human intent inference and action prediction as key elements for collaborative robots that must provide effective assistance. Without such predictive capabilities, the robot behavior remains mainly reactive and may respond only after the human has already changed the interaction.

The reviewed literature therefore shows that compliant control provides the physical basis for safe interaction, while several open issues remain. Fixed compliance may be insufficient across different task phases, variable compliance requires stable adaptation mechanisms, and reactive control does not directly address the prediction of human motion. These limitations have led to research directions that combine compliance control with perception, intent estimation and learning-based motion generation.

2.3. Human Intent Estimation and Prediction

2.3.1. Intent Estimation in Human–Robot Collaboration

Intent estimation is a central topic in Human–Robot Collaboration, especially in scenarios where the robot must adapt its behavior to the actions of a human partner. Effective assistance requires the ability to anticipate human behavior rather than only reacting after an action has already occurred. Hoffman *et al.* [13] identify human intent inference and action prediction as key requirements for collaborative robots operating in shared tasks.

In broader reviews of Human–Robot Collaboration, intent understanding is usually discussed together with perception, control, safety, and adaptation. Ajoudani *et al.* [2] position intent estimation as one of the components required for advanced collaborative behavior, since the robot must interpret human actions and modify its own response according to the expected evolution of the task.

Human intention can be inferred from different categories of information. Kekana *et al.* [21] provide a taxonomy of human intention recognition frameworks for industrial collaborative robotics, classifying intention cues into physical, physiological and contextual information. As shown in Figure 2.3, physical cues include motion, posture, gestures, gaze,

and interaction forces; physiological cues refer to biosignals related to the human state; and contextual cues describe task information, environment structure, and collaboration conditions.

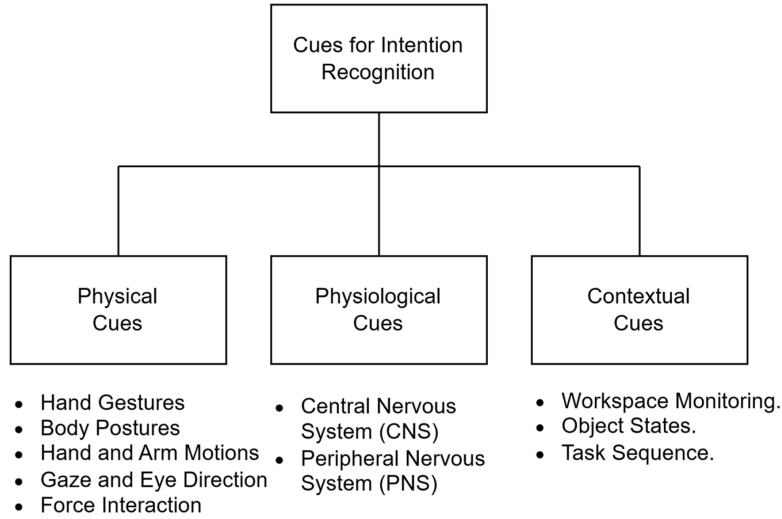


Figure 2.3: Cues for human intention recognition in industrial collaborative robotics. Adapted from Kekana *et al.* [21].

Among these categories, the most commonly used data sources in physical Human–Robot Collaboration are motion-based, force-based and posture-based cues. Motion-based methods exploit the recent evolution of human, robot or object trajectories; force-based methods use the interaction forces exchanged during physical contact; posture-based methods rely on body configuration or skeleton representations to describe the human state. These three modalities are particularly relevant in collaborative manipulation because they are directly related to the physical evolution of the shared task.

2.3.2. Motion-Based Intent Estimation

Human motion is one of the most commonly used sources of information for intent estimation. In collaborative manipulation, hand, arm, or object motion can provide indications about the future evolution of the task. Since the human partner often communicates intention through movement rather than explicit commands, motion history becomes a useful predictive cue.

In physical co-manipulation, past motion data have been used to estimate short-term human intent and generate robot trajectories [43]. Townsend *et al.* [43] formulate intent prediction in terms of future team velocity, showing how recent movement history can support short-horizon prediction during collaborative manipulation.

Other approaches combine motion-based intent estimation with control constraints. Shao *et al.* [38] propose an online framework in which human intent is represented through a low-dimensional dynamic-system model and estimated using a particle filter. Their method considers human manipulability and robot kinematic constraints, linking intent estimation with physically feasible robot motion generation.

Motion-based methods are especially relevant when the human does not explicitly command the robot, but expresses the desired evolution of the task through movement. Their main limitation is the ambiguity of partial observations: different future actions may be compatible with the same initial motion, especially in dynamic collaborative scenarios.

2.3.3. Force-Based Intent Estimation

Interaction forces are particularly informative in physical Human–Robot Interaction because they directly reflect the coupling between human, robot and manipulated object. In collaborative manipulation, forces and torques can reveal whether the human intends to translate, rotate, accelerate or stop the shared object.

Human-human co-manipulation studies show that force and motion data can reveal coordination strategies between partners. Mielke *et al.* [31] analyse extended-object manipulation in human dyads and identify force-related patterns associated with leader-induced lateral motion. Their work highlights how physical interaction data can be used to understand coordination mechanisms in shared-object tasks.

Force signals have also been used directly as inputs for intent recognition and robot response generation. Lai *et al.* [24] introduce physical Human–Robot Interaction Primitives based on interaction forces, using force measurements to estimate the user’s intent during tasks such as reaching and obstacle avoidance. In this formulation, interaction forces are not only control inputs, but also informative signals for recognizing the human objective.

Other works combine force and kinematic information to recognize higher-level collaborative intentions. Force-kinematic features extracted from physical human-human object manipulation have been used for symbolic intent recognition [36], while interaction force decomposition has been investigated to distinguish the contribution of different agents in multi-agent co-manipulation [39].

Force-based approaches are highly relevant in contact-rich tasks, but they depend on reliable force/torque sensing and on the interpretation of signals that may vary with object dynamics, human strategy and task constraints.

2.3.4. Posture-Based Intent Estimation and Pose Forecasting

Body posture provides a broader representation of human behavior than hand motion alone. Skeleton-based representations capture the configuration of the torso, arms and legs, allowing intention to be inferred from whole-body motion, preparatory movements and changes in direction.

In collaborative pushing and pulling tasks, spatio-temporal skeleton data have been used to classify whether the human intends to push, pull or remain idle [41]. Solak *et al.* [41] use this prediction to select an appropriate robot assistance strategy, showing how body posture can support intent recognition in physically demanding collaborative tasks.

Pose forecasting represents a related direction, where the objective is not only to classify the current intention but to predict the future evolution of the human body. Sampieri *et al.* [37] address pose forecasting in industrial Human–Robot Collaboration using past skeleton observations to predict future human poses. This contribution is relevant because it treats human pose as a temporal signal and uses it to anticipate future motion in collaborative environments.

Posture-based methods are useful when intention is expressed through full-body movement or when force information is not available. Their main limitations are related to occlusions, sensing noise, missing keypoints and variability in human motion patterns.

2.3.5. Open Challenges in Intent Estimation

The reviewed literature shows that motion, force and posture cues provide complementary information for human intention estimation. Motion-based approaches describe the temporal evolution of human or object trajectories, force-based methods capture the physical coupling between agents, and posture-based methods provide a structured representation of the human body configuration.

At the same time, intent estimation remains affected by several limitations. Human behavior is variable and only partially observable, while different intentions may generate similar initial motions. Sensing modalities can also be noisy or incomplete, especially in physical collaboration where the human, robot and object dynamics are coupled. For this reason, several works combine multiple cues or integrate intent estimation with control and prediction methods [2, 38, 43].

2.4. Learning-Based Robot Motion Generation

2.4.1. Learning from Demonstration

Learning from Demonstration has been widely studied as a framework for acquiring robot behaviors from examples. Argall *et al.* [3] provide one of the main surveys in this area, organizing the field according to demonstration acquisition, policy learning and behavior reproduction. Their work establishes Learning from Demonstration as an alternative to manually programmed robot behaviors, especially in tasks where explicit analytical rules are difficult to design.

A broader and more recent overview is provided by Osa *et al.* [33], who review imitation learning approaches in robotics, including trajectory-based methods, probabilistic models and reinforcement-learning-based formulations. Their review also highlights recurring limitations of demonstration-based methods, such as dependence on data quality, limited generalization and sensitivity to distribution shifts between training demonstrations and execution.

2.4.2. Generative Models and Diffusion Policy

Learning-based robot motion generation has progressively moved from deterministic trajectory reproduction toward probabilistic and generative formulations. This direction is particularly relevant when the same observation can correspond to different admissible future actions. Generative models allow robot policies to represent variability in the demonstrations and to generate action sequences rather than isolated commands.

Denoising Diffusion Probabilistic Models were introduced by Ho *et al.* [12] as generative models based on the iterative reversal of a noising process. Although originally developed for data generation, diffusion models have later been adapted to robotics as conditional models for action generation.

Chi *et al.* [5] introduce Diffusion Policy for visuomotor robotic control. Their method formulates policy learning as a conditional denoising problem, where the model generates action sequences conditioned on sensory observations. The policy predicts a sequence of future actions and executes them in a receding-horizon manner, allowing temporally coherent robot behavior. The main contribution of Diffusion Policy lies in modelling robot actions as conditional trajectories with potentially multimodal distributions. Compared with conventional behavior cloning, this formulation improves the representation of complex manipulation behaviors and supports closed-loop execution. However, the method

remains dependent on demonstration quality and coverage, and the generated actions still require a low-level control layer for physical execution.

2.4.3. Diffusion Models for Human–Robot Collaboration

Recent works have extended diffusion-based policies to collaborative Human–Robot Interaction. Ng *et al.* [32] propose Diffusion Co-Policy, a Transformer-based diffusion model trained on human-human demonstrations for collaborative tasks. The model generates coordinated human–robot action sequences, allowing the robot to act as a cooperative partner rather than as an isolated manipulator.

Diffusion Co-Policy formulates collaborative action generation as a joint generative problem. Instead of estimating human intent as an explicit symbolic variable, the model learns coordination patterns from demonstrations and generates robot actions conditioned on the observed interaction context. This approach addresses temporal consistency and multimodality in cooperative behavior, which are central aspects in collaborative manipulation. At the same time, the method remains dependent on task-specific demonstrations and does not remove the need for a physically safe execution layer. As in other learning-based approaches, generalization to new users, objects and task configurations remains an open issue.

2.4.4. Open Challenges in Learning-Based Motion Generation

The literature on learning-based motion generation shows a progression from imitation learning toward generative policy learning. Demonstration-based methods provide a foundation for acquiring behaviors from examples, while diffusion-based policies extend this framework by modelling action generation as a conditional trajectory denoising problem.

Several challenges remain open. Demonstration-based policies depend on the diversity and quality of the collected data, and their performance can degrade outside the training distribution [3, 33]. Generative policies can represent multimodal behaviors, but their outputs must still be physically feasible, stable and compatible with robot and environment constraints. In collaborative manipulation, these issues are amplified by human variability and by the physical coupling between agents.

2.5. Human Keypoint Recognition for Collaborative Robotics

2.5.1. Human Pose Estimation in Robotics

Human pose estimation provides a structured representation of the human body from visual data. In Human–Robot Collaboration, this representation is commonly used to describe the operator through anatomical landmarks such as shoulders, elbows, wrists, hips, knees and ankles.

2.5.2. Deep Learning-Based Human Pose Estimation

Deep learning methods have significantly advanced human pose estimation by replacing handcrafted visual pipelines with learned visual representations. OpenPose, proposed by Cao *et al.* [4], introduced a real-time multi-person pose estimation framework based on Part Affinity Fields. The method detects body keypoints and associates them across people in the image, making multi-person skeleton extraction practical in unstructured scenes.

High-Resolution Networks were later introduced by Sun *et al.* [42] to improve the spatial accuracy of human pose estimation. Unlike architectures that progressively reduce and then recover image resolution, HRNet maintains high-resolution representations throughout the network. This contribution is relevant for keypoint localization, where preserving spatial detail directly affects the accuracy of the estimated body joints.

2.5.3. YOLO-Based Pose Estimation

YOLO-based pose estimation integrates human detection and keypoint prediction within a single efficient architecture. Maji *et al.* [29] introduced YOLO-Pose as a real-time multi-person pose estimation method based on the YOLO framework. The method predicts human keypoints together with person detections, commonly following the COCO keypoint format [44]. The main contribution of YOLO-based pose estimation lies in its computational efficiency. Since detection and keypoint estimation are performed jointly, these methods are suitable for applications requiring continuous perception under latency constraints. The predicted confidence scores also allow unreliable keypoints to be filtered or weighted during downstream processing.

However, YOLO-based methods share common limitations with other image-based pose

estimators. Their accuracy can decrease in the presence of occlusions, partial body visibility or unusual body configurations.

2.5.4. Recent 3D Human Reconstruction Approaches

Recent research has also investigated full-body 3D reconstruction methods, which provide richer representations than sparse keypoints. SMPL introduced a parametric model of human body shape and pose [27], while Human Mesh Recovery methods estimate a full 3D body mesh directly from images [19]. More recent approaches, such as SAM 3D Body [48], further explore single-image 3D human reconstruction using modern segmentation and foundation-model-based pipelines.

These methods extend human perception from sparse landmark estimation toward dense body reconstruction. Their main advantage is the richer geometric representation of the human body, but this comes at the cost of higher complexity and greater dependency on visual quality.

2.5.5. Open Challenges in Vision-Based Human Motion Representation

The literature on vision-based human motion representation shows a progression from 2D pose estimation to RGB-D-based 3D keypoint extraction and full-body reconstruction. Several limitations remain open. Keypoint detectors can fail under occlusions, partial visibility or uncommon postures, while RGB-D reconstruction is affected by depth noise and calibration errors. Full-body reconstruction methods provide more complete information, but may be less suitable when low latency and compact representations are required. The choice between sparse keypoints and dense body reconstruction therefore depends on the required trade-off between accuracy, robustness, computational cost, and real-time applicability.

3 | Methodology

3.1. Introduction

3.1.1. Task Description and Project Objectives

This thesis centers on the paradigm of collaborative carrying, a fundamental human–robot interaction challenge where two agents must seamlessly coordinate their physical actions to jointly manipulate and transport a shared object. Within the framework of this study, the interaction dynamics are structured around an asymmetric leader–follower configuration, wherein a human operator naturally guides the movement and a robotic manipulator dynamically adapts as the follower. Consequently, rather than executing rigid or predefined trajectories, the robot’s primary objective is to continuously synthesize fluid motion references that mirror and anticipate the observed behavior of its human partner, thereby offering intuitive task support while preserving the organic flexibility of the cooperative execution.

The goal of the developed method is to predict a plausible follower response to the recent evolution of the interaction. The problem is formulated as a conditional trajectory prediction task: given a short time window of human motion observations, the system estimates the robot’s future response. The predicted motion is represented as Cartesian increments, which can then be translated into kinematic references for the robot control layer.

The design of the proposed methodology is guided by five main objectives:

- **Compact state representation:** The system must employ a lean and semantically relevant representation of human motion, based on the extraction of body keypoints rather than on the processing of raw images.
- **Learning from demonstrations:** The model must learn cooperative behavior from real human demonstrations, eliminating the need to manually program fixed and predefined interaction rules.

- **Real-time online prediction:** The system must generate short, temporally consistent motion references fast enough for real-time execution, allowing the robot response to be continuously updated as new human motion observations become available.
- **Architectural modularity:** The prediction module, based on machine learning, must remain separate from the compliant control layer. This decoupling ensures that high-level strategic motion generation and low-level physical interaction management can be addressed independently.
- **Safety and physical compliance:** The framework must guarantee physical safety during the interaction. By leveraging the modular separation, the system ensures that even in the presence of prediction uncertainties or sudden human movements, the compliant control layer can absorb forces and prevent hazardous behaviors.

3.1.2. Overview of the Proposed Method

The overall architecture can be divided into four main blocks: initially, RGB-D recordings are processed to extract three-dimensional keypoints of the human body; the episodic datasets of the extracted keypoints are then filtered, augmented and transformed into structured time windows; these windows are then used to train a diffusion model capable of predicting future increments of the follower’s motion. Finally, the trained model is evaluated both in simulation and in the real world.

Figure 3.1 provides a high-level overview of the proposed methodology and shows how the main components are connected, from RGB-D data acquisition to learning-based prediction and robot control.

The first stage of the system involves the perception of human motion. Data are acquired using an RGB-D camera and collected as video streams. From each frame, the human pose is extracted using a pose estimation model, which provides the two-dimensional position of the main body keypoints. These points are then combined with depth information to obtain three-dimensional coordinates in the camera reference frame. In this way, the raw RGB-D frames are replaced by a more compact and semantically meaningful representation of the human body since the movement of the shoulders, elbows, wrists, hips and lower limbs contains relevant information about the leader’s posture, direction of movement and intention.

The extracted keypoint trajectories are then transformed into an episodic temporal dataset. Each recording is interpreted as an ordered sequence and divided into windows containing

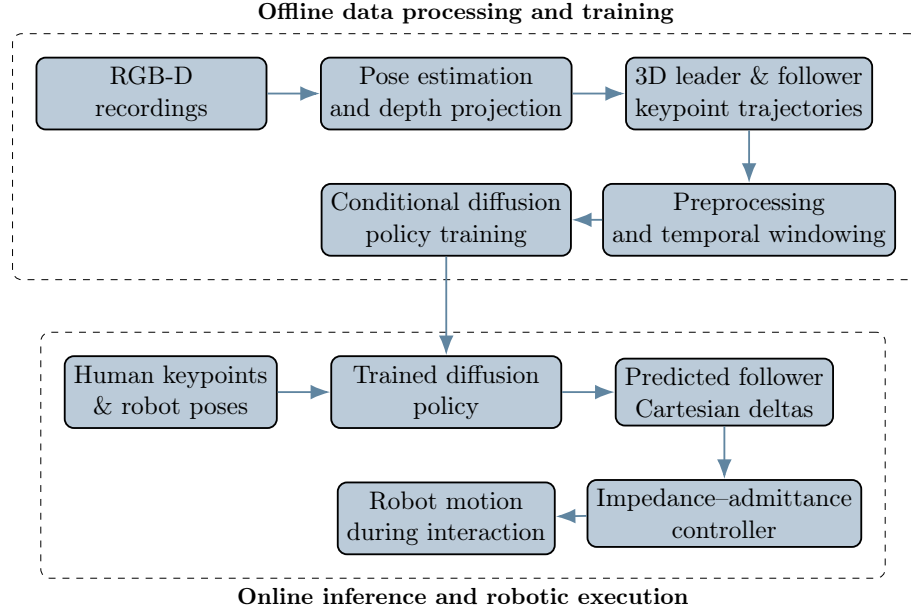


Figure 3.1: High-level architecture of the proposed methodology. The offline pipeline (top) processes human demonstrations to train the diffusion model, while the online pipeline (bottom) uses the trained policy to predict Cartesian increments for the impedance controller.

a portion of past observations and a portion of future actions. This allows the model to learn from short and coherent temporal segments while preserving the sequential structure of the movement. Since temporary keypoint loss is common in real RGB-D data, missing values are not discarded. Instead, they are handled through validity masks associated with the numerical data. These masks are used during normalization, training and evaluation, preventing missing values from influencing the loss or the computed metrics.

The trajectories are organized according to a leader–follower structure, as paired temporal sequences: the leader and follower past motion represents the contextual information from which the collaborative intention can be inferred, while the follower future motion represents the response that the model has to learn. During training, this relationship is learned from demonstrated human–human motion; while in inference, the follower role is assigned to the robot, which should generate a motion compatible with the observed behavior of the leader and with the recent evolution of the interaction.

As previously stated, the system frames the learning process as a conditional trajectory prediction task, shifting the focus from absolute spatial mapping to relative motion dynamics. To decouple the prediction from the absolute position of the subjects in the scene, both the observed history and the target motion are represented as Cartesian in-

crements (deltas) rather than absolute coordinates. In this way, the dependence on the initial position of the subjects in the scene is canceled and the learning process focuses on the continuous dynamics of movement. The objective is shifted from reproducing absolute spatial positions to predicting a plausible cooperative response consistent with the observed evolution of the interaction.

The core of the method is a conditional diffusion policy trained to generate short future motion sequences of the follower. The model receives the recent history of the interaction as input and produces a sequence of Cartesian deltas that represents the expected follower response. This allows the system to move beyond a purely deterministic prediction and to generate temporally coherent motion references for the collaborative task.

Once trained, the model is used to provide motion references to the robot: during the interaction, at each step, the most recent observations are processed and a future sequence of follower deltas is generated. Only the next action is applied, while the prediction is continuously updated as new observations become available. In this way, the generated motion can adapt to the evolution of the human movement over time.

In the complete robotic system, the diffusion policy does not act directly at the torque level. At each high-level update, it predicts a short trajectory of follower Cartesian increments expressed in the model frame. The first non-overlapping future increment is selected, transformed into the robot frame, bounded and transmitted to the low-level controller. The low-level impedance–admittance controller maintains the desired Cartesian position as an internal state. At every low-level iteration, this state is updated by the admittance increment and, whenever a new diffusion prediction is available, by the corresponding diffusion increment. The resulting Cartesian reference is tracked through impedance control and converted into joint torques. This separation allows the learning model to focus on cooperative motion prediction, while the low-level controller manages physical interaction, stability, compliance and actuation safety.

The remainder of the methodology is organized as follows. Section 3.2 describes the acquisition of the human–human demonstrations using an RGB-D camera, including YOLO-based pose estimation, depth projection and the assignment of leader and follower roles. Section 3.3 presents the preprocessing and dataset construction pipeline, including geometric augmentation, conversion from absolute coordinates to Cartesian increments, missing-value masks, episodic management, temporal windowing, action–observation overlap and masked feature-wise normalization. Section 3.4 introduces the conditional diffusion policy, its trajectory-denoising process, the leader–follower conditioning representation and the Transformer encoder–decoder architecture. The training

procedure, including dataset splitting, batch construction, the masked denoising objective, cumulative trajectory regularization and model selection, is detailed in Section 3.5. The impedance–admittance controller and its torque-level execution mechanisms are described in Section 3.6, while Section 3.7 explains the complete online pipeline, including frame alignment, real-time observation construction, receding-horizon prediction, command saturation and multi-rate coupling.

3.2. RGB-D Data Acquisition and Keypoint Extraction

The first phase of the methodology involves converting the raw RGB-D recordings into structured three-dimensional trajectories based on body keypoints estimated by the YOLO pose model. The process concludes with the creation of a numeric dataset file, in which each row captures the state of the system at a single instant (frame) of the demonstration. The overall workflow adopted to extract the 3D keypoint trajectories from the RGB-D recordings is summarized in Figure 3.2.

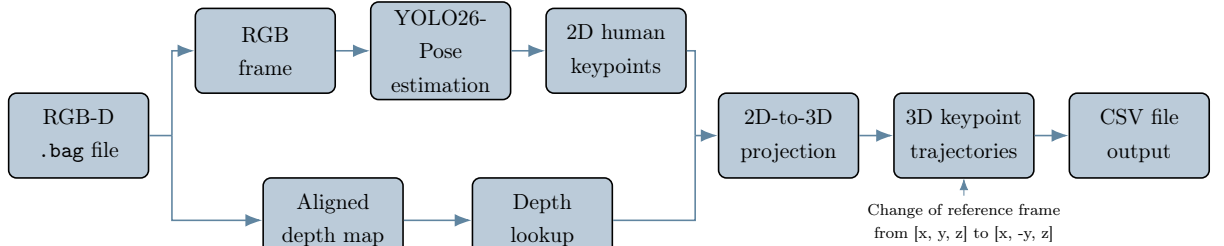


Figure 3.2: 3D keypoint extraction pipeline from RGB-D recordings. The flow is split into two branches: the RGB frame is used to estimate 2D keypoints via YOLO, while the aligned depth map provides the depth information needed to project the points into the camera’s reference frame. The resulting 3D trajectories are finally saved in CSV format.

3.2.1. RealSense Recordings

The data were collected using an Intel RealSense D415 camera, a device based on active stereoscopic depth technology [15]. Using the RealSense Viewer software [16], the acquisition sessions were saved in `.bag` format. This choice proved useful, as it encapsulates the RGB and depth streams in a single container and enables the recordings to be reprocessed offline multiple times without altering their physical fidelity.

After recording, the sequences were trimmed to isolate only the segments of useful inter-

action. Preparation phases and irrelevant video portions were discarded, providing the extraction pipeline with intrinsically cleaner temporal sequences that are more focused on the collaborative task.

During offline processing, the RealSense pipeline aligns the depth stream with the RGB stream, ensuring that each identified pixel coordinate is associated with the corresponding depth value. Furthermore, leveraging information provided by the RealSense SDK—such as the depth scale and intrinsic optics parameters—the system prepares the data for rigorous 3D deprojection.

3.2.2. YOLO-based Pose Estimation

Human posture analysis is performed on each RGB frame using an Ultralytics YOLO26 pose model. YOLO, acronym for *You Only Look Once*, is a family of real-time computer vision models designed to process an image in a single forward pass. In the pose estimation configuration, the model performs human detection and keypoint regression within the same inference pipeline, returning for each detected subject both the body keypoint coordinates and the corresponding confidence values [45].

The YOLO pose model follows the COCO keypoint convention, which defines a standard set of 17 anatomical landmarks for human pose estimation [44]. In this work, however, only a targeted subset of these points is retained: nose, shoulders, elbows, wrists, hips, knees and ankles listed in Appendix A.1. Facial details such as eyes and ears are discarded in favor of a more compact anatomical representation, while preserving the body landmarks that are most relevant for interpreting posture, arm motion and movement direction during collaborative carrying.

To preserve data quality, each detected keypoint is filtered according to its confidence score. If the confidence value is below the selected threshold, the corresponding keypoint is considered unreliable and is left as missing. This avoids introducing low-confidence measurements into the dataset. Missing observations are not reconstructed at this stage, but are instead handled later through validity masks during preprocessing, normalization and training.

For offline dataset extraction, the YOLO26x-Pose model was used, an extra-large variant with 57.6 million parameters. This prioritizes keypoint localization accuracy, which is crucial for the subsequent 2D-to-3D projection and the construction of reliable training trajectories. Because this processing occurs offline on recorded `.bag` files, the higher computational cost is acceptable. Conversely, for online inference, a lighter YOLO26s-Pose variant (10.4 million parameters) was chosen to minimize latency and enable real-

time execution without sacrificing robust pose estimation.

3.2.3. Projection of Keypoints from 2D to 3D

Since the extracted keypoints initially reside in the two-dimensional domain of the image, they must be projected into three-dimensional space using the aligned depth map.

The process associates each pixel coordinate with its corresponding depth measurement. If the exact value is corrupted or null, the algorithm inspects a narrow spatial neighborhood, extracting the median of adjacent valid depth values. This method provides significant stability to the system, mitigating the effect of local measurements affected by visual noise.

Once a reliable depth is acquired, the camera's intrinsic parameters guide the pixel's de-projection, generating a 3D point in the sensor's reference frame. The resulting Cartesian triplets are shifted from the RealSense predefined reference frame shown in Figure 3.3: the x-axis describes the horizontal, the y-axis is inverted to point upward (according to an intuitive visual representation), and the z-axis maps the actual depth of the scene.

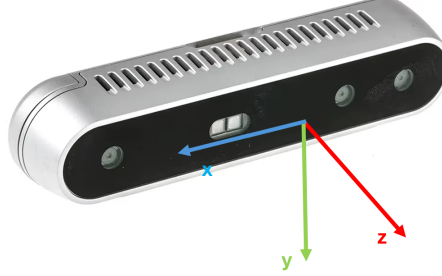


Figure 3.3: Original camera reference frame

Finally, to smooth out the physiological jitter between adjacent frames, the spatial coordinates p_t are filtered using an exponential moving average, defined as:

$$\hat{p}_t = \alpha p_t + (1 - \alpha) \hat{p}_{t-1} \quad (3.1)$$

The smoothing factors are calibrated separately for the three axes, with $\alpha_x = 0.20$, $\alpha_y = 0.20$, and $\alpha_z = 0.15$, assigning stronger smoothing to the z-direction, which is the most vulnerable to noise fluctuations.

3.2.4. Assigning Leader/Follower Roles

Once posture estimation and spatial reconstruction of the human keypoints are complete, the system assigns the cooperative roles to the detected subjects and stably labels them as leader and follower.

Role initialization occurs by analyzing the subjects' spatial arrangement within the first frame, based on the known starting position of the leader. In subsequent frames, a persistent tracking algorithm—based on the spatial proximity of body centroids—ensures identities consistency, eliminating the risk of role swapping during the more dynamic phases of the demonstration.

The culmination of this processing flow is the dataset preparation. For each processed frame, the follower is represented solely by the 3D coordinates of its right wrist, which constitutes the motion target to be predicted. The leader, on the other hand, is profiled through the entire set of its body keypoints, recorded in their respective 3D coordinates. If an anatomical point escapes detection, its measurement remains empty: no artificial early interpolation is performed.

3.3. Data Preprocessing, Augmentation, and Dataset Construction

After the RGB-D acquisition and keypoint extraction phase, the resulting CSV files contain time series of three-dimensional absolute coordinates for the leader and follower. Before they can be used for diffusion policy training, these data must be transformed into a structured dataset suitable for sequential learning. This phase includes signal organization, geometric augmentation, conversion from absolute coordinates to movement increments, handling missing values, episodic management, temporal windowing and masked feature-wise normalization. The complete preprocessing workflow adopted to transform the extracted RGB-D keypoint trajectories into model-ready training samples is summarized in Figure 3.4.

3.3.1. Signal Organization and Coordinate Representation

Each processed recording is saved as a CSV file, where each row corresponds to a frame of the demonstration. The follower is represented by the three-dimensional absolute coordinates of the right wrist, while the leader is represented by a selected set of body keypoints. Each keypoint is described using Cartesian coordinates $[x, y, z]$ according to

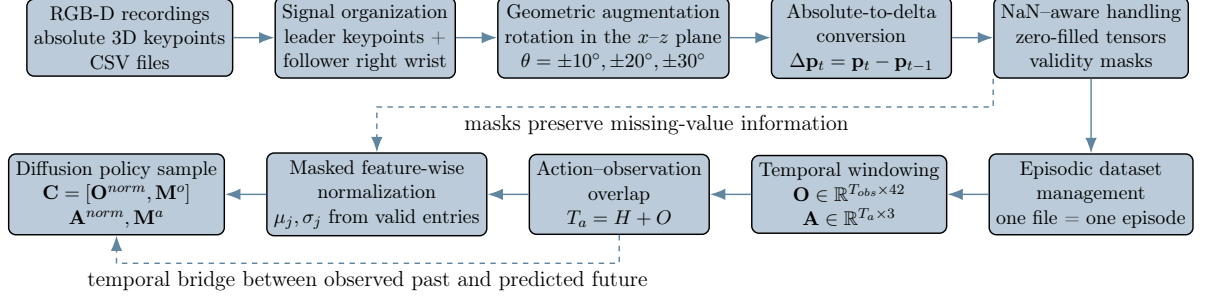


Figure 3.4: Preprocessing pipeline used to transform raw RGB-D keypoint recordings into training samples for the diffusion policy. Absolute trajectories are organized, augmented, converted into Cartesian deltas, represented through NaN-aware tensors and validity masks, split into episodic temporal windows and normalized with masked feature-wise statistics.

the convention introduced in Subsection 3.2.3. This convention is maintained throughout preprocessing and dataset construction, ensuring that all demonstrations have the same spatial structure before training.

3.3.2. Geometric Augmentation

To increase the spatial variability of the training data, a geometric data augmentation procedure is applied to the extracted trajectories. Augmentation consists of rotating the Cartesian coordinates in the x - z plane, equivalent to a rotation around the vertical y -axis. In this work, six additional rotated versions are generated for each trajectory, using rotation angles of $\theta = \pm 10^\circ$, $\pm 20^\circ$, and $\pm 30^\circ$. This transformation simulates variations in the relative orientation between the subjects and the camera, while preserving the temporal structure of the demonstrated movement.

For each coordinate triplet, the rotation is defined as:

$$\begin{aligned}
 x_{rot} &= x \cos(\theta) - z \sin(\theta), \\
 z_{rot} &= x \sin(\theta) + z \cos(\theta), \\
 y_{rot} &= y.
 \end{aligned} \tag{3.2}$$

The same transformation is applied consistently to both the leader and follower coordinates. This is important because the relative motion between the two agents must remain physically consistent even after augmentation. The goal of this phase is not to generate new motor patterns, but to expose the model to equivalent demonstrations observed from slightly different spatial configurations.

3.3.3. Conversion from Absolute Coordinates to Delta

After augmentation, the trajectories are converted from absolute Cartesian coordinates to temporal differences. Instead of learning the absolute position of the follower in the scene, the model is trained to predict Cartesian increments of motion from the recent delta-based evolution of the interaction. Let $\mathbf{p}_t \in \mathbb{R}^{1 \times 3}$ denote a generic Cartesian position at timestep t :

$$\mathbf{p}_t = \begin{bmatrix} x_t & y_t & z_t \end{bmatrix}. \quad (3.3)$$

The corresponding delta representation is defined as

$$\Delta \mathbf{p}_t = \mathbf{p}_t - \mathbf{p}_{t-1} = \begin{bmatrix} \Delta x_t & \Delta y_t & \Delta z_t \end{bmatrix}. \quad (3.4)$$

In this work, two different delta-based variables are used: the observation vector and the action vector. The observation vector describes the recent state of the leader–follower interaction and contains the follower right-wrist delta together with the deltas of the selected leader keypoints:

$$\mathbf{o}_t = \begin{bmatrix} \Delta \mathbf{p}_t^{F, rw} & \Delta \mathbf{p}_t^{L, 1} & \Delta \mathbf{p}_t^{L, 2} & \dots & \Delta \mathbf{p}_t^{L, 13} \end{bmatrix} \in \mathbb{R}^{1 \times 42}, \quad (3.5)$$

where $\Delta \mathbf{p}_t^{F, rw} \in \mathbb{R}^{1 \times 3}$ is the delta motion of the follower right wrist and $\Delta \mathbf{p}_t^{L, j} \in \mathbb{R}^{1 \times 3}$ is the delta motion of the j -th selected leader keypoint. Therefore, the observation vector contains three components associated with the follower and thirty-nine components associated with the leader. The action vector corresponds only to the future motion of the follower right wrist, which is the target to be predicted by the diffusion policy:

$$\mathbf{a}_t = \Delta \mathbf{p}_t^{F, rw} = \begin{bmatrix} \Delta x_t^{F, rw} & \Delta y_t^{F, rw} & \Delta z_t^{F, rw} \end{bmatrix} \in \mathbb{R}^{1 \times 3}. \quad (3.6)$$

This representation offers two main advantages for the considered task. First, it reduces the dependence on the absolute initial position of the subjects in the scene. Second, it shifts the learning problem from the reproduction of absolute spatial positions to the prediction of local motion dynamics, which are more directly related to the collaborative interaction. The conversion is performed while preserving missing values. If either $\mathbf{p}_t$ or $\mathbf{p}_{t-1}$ is unavailable for a given coordinate, the corresponding delta value is also marked as missing. This avoids introducing unreliable motion increments when the original keypoint trajectory contains interruptions due to occlusions, low-confidence detections or invalid depth measurements.

3.3.4. Handling Missing Values and Validity Masks

Unavailable coordinates are stored as NaN values in the CSV files. Since these values cannot be directly propagated through the neural network, a NaN-aware representation is adopted during dataset construction. The purpose of this strategy is to preserve the information about missing measurements while ensuring that all numerical tensors passed to the model contain finite values. For each observation window and action window, two binary validity masks are defined. Let $\mathbf{O} \in \mathbb{R}^{T_{obs} \times 42}$ denote an observation window and $\mathbf{A} \in \mathbb{R}^{T_a \times 3}$ denote the corresponding action trajectory. The observation validity mask is defined as

$$M_{t,j}^o = \begin{cases} 1, & \text{if } O_{t,j} \text{ is available,} \\ 0, & \text{otherwise,} \end{cases} \quad (3.7)$$

with $\mathbf{M}^o \in \{0, 1\}^{T_{obs} \times 42}$. Similarly, the action validity mask is defined as

$$M_{i,j}^a = \begin{cases} 1, & \text{if } A_{i,j} \text{ is available,} \\ 0, & \text{otherwise,} \end{cases} \quad (3.8)$$

with $\mathbf{M}^a \in \{0, 1\}^{T_a \times 3}$. After the masks are created, all NaN values in the numerical tensors are replaced with zeros. This produces four tensors for each training sample:

$$\mathbf{O}^{fill}, \quad \mathbf{M}^o, \quad \mathbf{A}^{fill}, \quad \mathbf{M}^a. \quad (3.9)$$

The filled tensors contain finite numerical values, while the masks retain the information about which entries were originally valid. As a result, a zero-filled value is not automatically interpreted as a real measurement, because its corresponding mask entry specifies whether it represents a valid zero delta or a missing coordinate. The masks are used at different stages of the pipeline. During normalization, statistics are computed only from valid entries, preventing missing values from influencing the estimated mean and standard deviation. During training, the action mask is used to exclude invalid target coordinates from the denoising loss. During evaluation, the same principle is applied so that missing ground-truth values do not affect the reported metrics. The observation mask is also used as part of the model input. After normalization, the observation values are concatenated with their corresponding validity mask:

$$\mathbf{c}_t = \begin{bmatrix} \mathbf{o}_t^{norm} & \mathbf{m}_t^o \end{bmatrix} \in \mathbb{R}^{1 \times 84}. \quad (3.10)$$

In this way, the conditioning input explicitly informs the model about the reliability of each observed coordinate. The physical observation vector contains 42 features, while the effective conditioning token provided to the Transformer contains 84 features. An alternative solution would have been to discard all windows containing missing values. However, this would have significantly reduced the amount of available data and would have trained the model only on ideal sequences where all keypoints are always detected. This assumption is not realistic during real-time inference, where keypoints may be temporarily unavailable due to occlusions, low confidence detections, depth errors or partial body visibility. Therefore, the masking strategy is adopted both to handle missing values during training and to improve the robustness of the diffusion policy in practical deployment conditions.

3.3.5. Episodic Dataset Management

The recorded datasets are treated as episodes, preserving the temporal structure of the original demonstration. Each file corresponds to a specific execution of the task, characterized by its own initial conditions and motion dynamics.

The episodic structure is also exploited during training: instead of forming mini-batches by randomly mixing samples from all demonstrations, the batch sampler creates batches using windows extracted from the same episode. This choice makes each optimization step more coherent, since the samples within a batch come from trajectories with similar characteristics.

At the same time, diversity is maintained across the training process by sampling from different episodes throughout the epoch. As a result, the model benefits from both locally consistent batches and exposure to the full variability of the recorded demonstrations.

3.3.6. Temporal Windowing and Action-Observation Overlap

The diffusion policy is trained on fixed-length temporal windows extracted from the recorded demonstrations, where each training sample is composed of two temporally related segments: a conditioning observation window and a target action window. The observation window provides the recent history of the interaction (past movements of the leader and follower), while the action window contains the future increments of the follower that the model must predict.

Let T_{obs} denote the number of past observations used for conditioning, H the nominal prediction horizon, and O the number of overlapping timesteps between the end of the

observation window and the beginning of the action window. The model does not process only the strictly future part of the trajectory. Instead, the denoising decoder receives an action sequence of length $T_a = H + O$. For a window starting at timestep t , the observation segment contains the samples from t to $t + T_{obs} - 1$, while the action segment begins O steps before the end of the observation window. Consequently, the first O action steps overlap with the final portion of the observed trajectory, whereas the remaining H steps correspond to future actions beyond the observation horizon. As illustrated in Figure 3.5, the action window is not placed entirely after the observation window. Instead, its first part overlaps with the final portion of the observation history. This overlap creates a temporal bridge between the observed past and the predicted future. In the adopted configuration, $T_{obs} = 30$, $H = 5$, and $O = 2$. Therefore, the target sequence given to the denoising decoder includes both the overlapping transition region and the future predictive region.

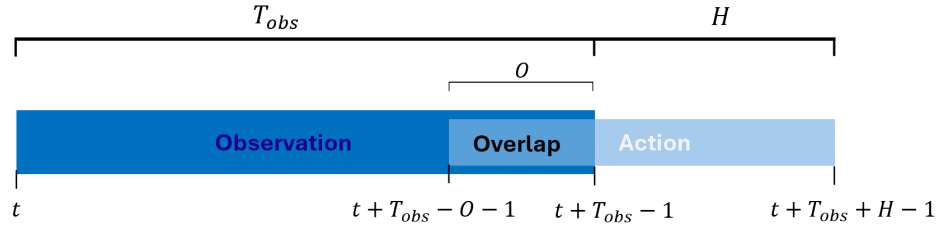


Figure 3.5: Training window with observation-action overlap. The conditioning window in dark blue contains T_{obs} past observations, whereas the target action sequence in light blue includes an overlap region O followed by a future prediction horizon H .

Incorporating this overlap provides two primary methodological advantages. First, it ensures movement fluidity and mitigates discontinuities. Because the model predicts positional increments (deltas), predicting actions strictly after the conditioning window could cause abrupt transitions. The overlap provides a smoother blend between the observed past and the generated future, acting as a natural smoothing mechanism.

Second, it enforces consistency in multimodal predictions since diffusion policy excels at capturing diverse valid behaviors, the overlapping region anchors the newly generated sequence to the ongoing motion, preventing the robot from oscillating between different modes across consecutive chunks. These benefits align the training objective seamlessly with the online execution policy. At inference time, the model samples an entire action trajectory through the reverse diffusion process, but the controller applies only the first action after the overlapping region. This represents the first genuinely future delta increment with respect to the observation-action overlap.

3.3.7. Normalization Strategy

Before being fed to the diffusion model, both conditioning observations and target actions are normalized. Since the input data are represented as Cartesian deltas, the magnitude of the different features may vary depending on the keypoint, the axis and the role of the subject. For this reason, normalization is performed independently for each feature, rather than using a single global mean and standard deviation for the whole tensor.

Let $\mathbf{O}^{fill} \in \mathbb{R}^{N \times T_{obs} \times 42}$ denote the zero-filled observation windows in the training set, and let $\mathbf{M}^o \in \{0, 1\}^{N \times T_{obs} \times 42}$ be the corresponding observation validity mask, where N is the number of training windows. For each observation feature j , the mean is computed only over valid entries:

$$\mu_j^o = \frac{\sum_{n=1}^N \sum_{t=1}^{T_{obs}} M_{n,t,j}^o O_{n,t,j}^{fill}}{\sum_{n=1}^N \sum_{t=1}^{T_{obs}} M_{n,t,j}^o}. \quad (3.11)$$

The corresponding standard deviation is computed as

$$\sigma_j^o = \sqrt{\frac{\sum_{n=1}^N \sum_{t=1}^{T_{obs}} M_{n,t,j}^o \left(O_{n,t,j}^{fill} - \mu_j^o \right)^2}{\sum_{n=1}^N \sum_{t=1}^{T_{obs}} M_{n,t,j}^o}}. \quad (3.12)$$

The normalized observation value is then obtained as

$$O_{n,t,j}^{norm} = M_{n,t,j}^o \frac{O_{n,t,j}^{fill} - \mu_j^o}{\sigma_j^o}. \quad (3.13)$$

The multiplication by the validity mask ensures that invalid entries remain neutral after normalization. Therefore, missing coordinates do not produce artificial normalized values and remain equal to zero in the numerical tensor.

The same strategy is applied independently to the target action trajectories. Let $\mathbf{A}^{fill} \in \mathbb{R}^{N \times T_a \times 3}$ be the zero-filled action tensor and $\mathbf{M}^a \in \{0, 1\}^{N \times T_a \times 3}$ the corresponding action validity mask. For each action feature j , the normalization statistics are computed as

$$\mu_j^a = \frac{\sum_{n=1}^N \sum_{t=1}^{T_a} M_{n,t,j}^a A_{n,t,j}^{fill}}{\sum_{n=1}^N \sum_{t=1}^{T_a} M_{n,t,j}^a}, \quad (3.14)$$

$$\sigma_j^a = \sqrt{\frac{\sum_{n=1}^N \sum_{t=1}^{T_a} M_{n,t,j}^a \left(A_{n,t,j}^{fill} - \mu_j^a \right)^2}{\sum_{n=1}^N \sum_{t=1}^{T_a} M_{n,t,j}^a}}. \quad (3.15)$$

The normalized action tensor is then defined as

$$A_{n,t,j}^{norm} = M_{n,t,j}^a \frac{A_{n,t,j}^{fill} - \mu_j^a}{\sigma_j^a}. \quad (3.16)$$

This formulation makes the normalization explicitly feature-wise. Each coordinate component of each keypoint has its own mean and standard deviation. As a consequence, the normalization does not mix different axes, different keypoints or leader and follower variables. This is important because the statistical distribution of a wrist displacement, for example, can differ from that of a shoulder, hip or ankle displacement, and the depth direction can exhibit different noise characteristics from the horizontal and vertical directions.

All normalization statistics are computed only from the training set. The same parameters are then reused for validation, testing and online inference. This prevents information from the validation or test set from influencing the training procedure and ensures that the model always receives data scaled according to the same convention used during training.

3.4. Diffusion Policy for Robot Motion Prediction

The high-level policy is formulated as a Denoising Diffusion Probabilistic Model (DDPM). The architecture builds on the diffusion-policy framework proposed by Chi et al. [5] and follows the idea of learning collaborative robot behavior from human–human demonstrations explored by Ng et al. [32].

The core predictive module of the proposed framework is a conditional diffusion policy that generates short follower motion trajectories from the recent evolution of the collaborative interaction. In contrast to direct deterministic regression, the model is trained to denoise an entire action trajectory. This formulation is well suited to collaborative carrying because the follower response is not necessarily limited to a single deterministic displacement: different smooth reactions may be plausible for the same short observation history. The diffusion model therefore learns a conditional distribution over future Cartesian increments rather than a fixed one-to-one mapping.

The model uses the observation and action variables introduced in Section 3.3.3. The conditioning sequence contains the recent leader–follower motion history, while the action trajectory represents the future Cartesian increments of the follower right wrist. During training, this signal is extracted from human–human demonstrations; during robotic execution, the generated follower deltas are interpreted as Cartesian motion references for the robot end-effector.

To maintain consistency with the notation introduced in the preprocessing section, the symbol t denotes a physical timestep of the demonstration, and τ denotes the first physical timestep following the observation window. The symbol ℓ denotes a diffusion timestep. Action tokens are indexed by $i = 1, \dots, T_a$, while $j = 1, \dots, 3$ identifies a Cartesian action component. When two action-token positions appear in the same attention equation, i denotes the query position and i' denotes the key position. The index $s = 0, \dots, T_{obs}$ identifies a position in the encoder memory, with $s = 0$ corresponding to the diffusion-timestep token. Finally, $b = 1, \dots, B$ indexes the elements of a training batch.

3.4.1. Action Trajectory Denoising

The normalized action matrix $\mathbf{A}^{norm}$ introduced in the previous section constitutes the clean sample of the diffusion process. Therefore, the diffusion notation is defined through

$$\mathbf{X}_0 \equiv \mathbf{A}^{norm} = \begin{bmatrix} \mathbf{a}_{\tau-O}^{norm} \\ \mathbf{a}_{\tau-O+1}^{norm} \\ \vdots \\ \mathbf{a}_{\tau+H-1}^{norm} \end{bmatrix} \in \mathbb{R}^{T_a \times 3}, \quad T_a = H + O. \quad (3.17)$$

where τ denotes the first timestep after the observation window, O is the number of overlapping timesteps, H is the future prediction horizon, and T_a is the total action sequence length provided to the denoising decoder. The diffusion process corrupts this clean trajectory through a sequence of Gaussian perturbations. For a diffusion step ℓ , the forward noising process is defined as

$$q(\mathbf{X}_\ell | \mathbf{X}_{\ell-1}) = \mathcal{N}\left(\sqrt{1 - \beta_\ell} \mathbf{X}_{\ell-1}, \beta_\ell \mathbf{I}\right). \quad (3.18)$$

Here, $\ell = 1, \dots, N_{train}$ denotes the theoretical diffusion timestep, while $\ell = 0$ identifies the clean sample $\mathbf{X}_0$. β_ℓ is the variance schedule. Defining $\alpha_\ell = 1 - \beta_\ell$ and $\bar{\alpha}_\ell = \prod_{u=1}^{\ell} \alpha_u$, where $u = 1, \dots, \ell$ is a dummy index over the diffusion schedule. The noisy trajectory at

an arbitrary step can be sampled directly as

$$\mathbf{X}_\ell = \sqrt{\bar{\alpha}_\ell} \mathbf{X}_0 + \sqrt{1 - \bar{\alpha}_\ell} \boldsymbol{\epsilon}, \quad (3.19)$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ with $\boldsymbol{\epsilon} \in \mathbb{R}^{T_a \times 3}$ is Gaussian noise with the same shape as the action trajectory. The implemented model follows the ϵ -prediction formulation. Given the noisy action trajectory $\mathbf{X}_\ell$, the diffusion timestep ℓ , and the conditioning sequence $\mathbf{C}_\tau$, the network predicts the noise component that was added to the clean trajectory:

$$\hat{\boldsymbol{\epsilon}}_\theta = f_\theta(\mathbf{X}_\ell, \ell, \mathbf{C}_\tau), \quad (3.20)$$

where θ denotes the trainable parameters of the denoising network. The clean trajectory implied by the prediction can be recovered as

$$\hat{\mathbf{X}}_0 = \frac{\mathbf{X}_\ell - \sqrt{1 - \bar{\alpha}_\ell} \hat{\boldsymbol{\epsilon}}_\theta}{\sqrt{\bar{\alpha}_\ell}}. \quad (3.21)$$

During inference, the reverse process starts from Gaussian noise and iteratively applies the DDPM scheduler update using the predicted noise.

3.4.2. Conditioning on Leader and Follower Motion

Using the observation vector $\mathbf{o}_t$ defined in Section 3.3.3, the conditioning matrix is obtained by stacking the last T_{obs} observations:

$$\mathbf{O}_\tau = \begin{bmatrix} \mathbf{o}_{\tau-T_{obs}} \\ \mathbf{o}_{\tau-T_{obs}+1} \\ \vdots \\ \mathbf{o}_{\tau-1} \end{bmatrix} \in \mathbb{R}^{T_{obs} \times 42}, \quad (3.22)$$

where τ denotes the first timestep after the observation window.

Using the masked representation introduced in Section 3.3.4, the conditioning input is constructed by concatenating the normalized observation values with the corresponding observation validity mask:

$$\mathbf{c}_t = \begin{bmatrix} \mathbf{o}_t^{norm} & \mathbf{m}_t^o \end{bmatrix} \in \mathbb{R}^{1 \times 84}. \quad (3.23)$$

The complete conditioning sequence provided to the model is therefore

$$\mathbf{C}_\tau = \begin{bmatrix} \mathbf{c}_{\tau-T_{obs}} \\ \mathbf{c}_{\tau-T_{obs}+1} \\ \vdots \\ \mathbf{c}_{\tau-1} \end{bmatrix} \in \mathbb{R}^{T_{obs} \times 84}. \quad (3.24)$$

3.4.3. Transformer Encoder–Decoder Architecture

The denoising network is implemented as a masked Transformer encoder–decoder. The encoder processes the conditioning sequence extracted from the recent leader–follower motion history, while the decoder processes the noisy action trajectory at the current diffusion step and predicts the Gaussian noise associated with each action token. This structure allows the model to combine temporal information from the observation window with the partially denoised action trajectory. The internal structure of the denoising network is summarized in Figure 3.6. The diagram highlights how the conditioning sequence and the noisy action trajectory are embedded separately, combined through encoder–decoder attention, and finally projected into the predicted noise trajectory.

Both branches first project their inputs into a common embedding space of dimension $d_{emb} = 64$.

For the conditioning branch, each token $\mathbf{c}_t \in \mathbb{R}^{1 \times 84}$ is embedded through a linear layer:

$$\mathbf{z}_t^c = \mathbf{c}_t \mathbf{W}_c^\top + \mathbf{b}_c, \quad t = \tau - T_{obs}, \dots, \tau - 1. \quad (3.25)$$

where $\mathbf{W}_c \in \mathbb{R}^{d_{emb} \times 84}$, $\mathbf{b}_c, \mathbf{z}_t^c \in \mathbb{R}^{1 \times d_{emb}}$.

$\mathbf{W}_c$ and $\mathbf{b}_c$ are learnable parameters of the conditioning embedding layer. The input token $\mathbf{c}_t$ already contains both the normalized observation values and their corresponding validity mask. This linear projection maps the 84-dimensional conditioning token into the internal Transformer embedding space. In this way, the model can learn how to combine the normalized Cartesian deltas with the corresponding mask values, assigning different relevance to reliable measurements and to zero-filled missing entries. The embedded conditioning sequence is denoted by

$$\mathbf{Z}_\tau^c = [\mathbf{z}_{\tau-T_{obs}}^c, \dots, \mathbf{z}_{\tau-1}^c]^\top \in \mathbb{R}^{T_{obs} \times d_{emb}}. \quad (3.26)$$

The diffusion timestep ℓ is embedded using sinusoidal features. For embedding index

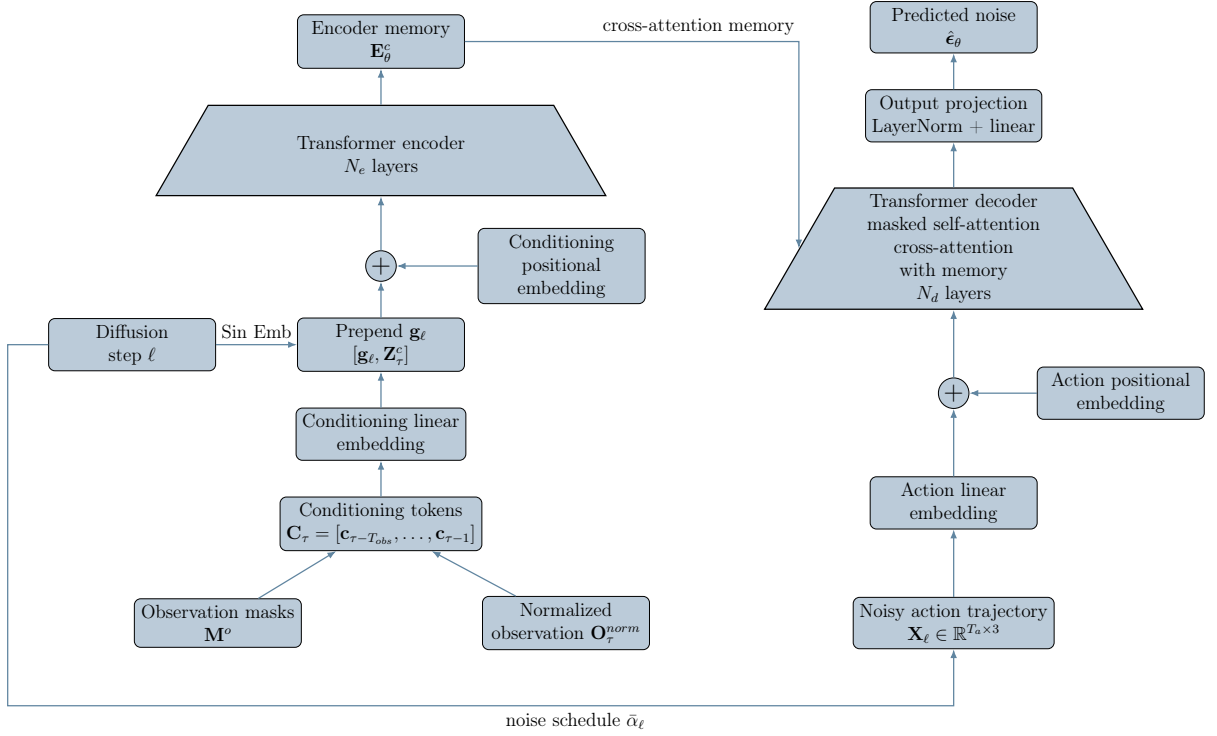


Figure 3.6: Transformer denoiser architecture used inside the conditional diffusion policy. The conditioning branch embeds the normalized observation tokens and their validity masks, prepends the sinusoidal diffusion-step embedding, and adds learnable positional embeddings before the Transformer encoder. The action branch embeds the noisy action trajectory and adds action positional embeddings before the Transformer decoder. The decoder performs masked self-attention over the action tokens and cross-attention over the encoded conditioning memory. The output projection maps the decoder features to the predicted Gaussian noise $\hat{\epsilon}_\theta$

$m = 0, \dots, d_{emb}/2 - 1$, the frequency is logarithmically spaced as

$$\omega_m = \exp\left(-\frac{m \log(10000)}{d_{emb}/2 - 1}\right), \quad (3.27)$$

and the timestep embedding is defined as

$$\mathbf{g}_\ell = \begin{bmatrix} \sin(\ell\omega_0) & \cdots & \sin(\ell\omega_{d_{emb}/2-1}) & \cos(\ell\omega_0) & \cdots & \cos(\ell\omega_{d_{emb}/2-1}) \end{bmatrix}. \quad (3.28)$$

The embedding $\mathbf{g}_\ell \in \mathbb{R}^{1 \times d_{emb}}$ does not represent a physical timestep of the demonstrated motion. Instead, it represents the current noise level of the reverse diffusion process. This is necessary because the same noisy trajectory $\mathbf{X}_\ell$ must be interpreted differently depending on whether the model is operating at an early highly noisy denoising step or

at a later step closer to the clean action trajectory.

The timestep embedding is prepended to the embedded observation tokens, producing a conditioning sequence of length $T_{obs} + 1$. Learnable positional embeddings $\mathbf{P}^c$ are then added before passing the sequence through the Transformer encoder:

$$\mathbf{E}_\theta^c = \text{Encoder}_\theta \left(\begin{bmatrix} \mathbf{g}_\ell \\ \mathbf{z}_{\tau-T_{obs}}^c \\ \mathbf{z}_{\tau-T_{obs}+1}^c \\ \vdots \\ \mathbf{z}_{\tau-1}^c \end{bmatrix} + \mathbf{P}^c \right), \quad (3.29)$$

The conditioning positional-embedding matrix and the resulting encoder memory satisfy $\mathbf{P}^c, \mathbf{E}_\theta^c \in \mathbb{R}^{(T_{obs}+1) \times d_{emb}}$. $\mathbf{E}_\theta^c$ represents the encoded conditioning memory used by the decoder that contains a contextual representation of the observation history conditioned on the current diffusion step. The first token carries information about the noise level, while the remaining tokens encode the recent leader–follower motion and the reliability of each observed coordinate. This memory is later accessed by the decoder through cross-attention.

The action branch receives the noisy action trajectory $\mathbf{X}_\ell \in \mathbb{R}^{T_a \times 3}$. Each noisy action token is projected into the same embedding space:

$$\mathbf{z}_i^a = \mathbf{X}_\ell[i, :] \mathbf{W}_a^\top + \mathbf{b}_a, \quad i = 1, \dots, T_a. \quad (3.30)$$

where $\mathbf{W}_a \in \mathbb{R}^{d_{emb} \times 3}$, $\mathbf{b}_a, \mathbf{z}_i^a \in \mathbb{R}^{1 \times d_{emb}}$.

$\mathbf{W}_a$ and $\mathbf{b}_a$ are learnable parameters of the action embedding layer. The action embedding is separate from the conditioning embedding because the two inputs have different meanings and dimensions: conditioning tokens describe the observed interaction history, whereas action tokens represent the noisy follower trajectory that must be denoised.

After adding learnable action positional embeddings,

$$\mathbf{Z}^a = \begin{bmatrix} \mathbf{z}_1^a \\ \mathbf{z}_2^a \\ \vdots \\ \mathbf{z}_{T_a}^a \end{bmatrix} + \mathbf{P}^a. \quad (3.31)$$

the resulting sequence is processed by the Transformer decoder:

$$\mathbf{H}_\theta^a = \text{Decoder}_\theta(\mathbf{Z}^a, \mathbf{E}_\theta^c). \quad (3.32)$$

The action positional embeddings, the embedded action sequence and the decoder output have dimensions $\mathbf{P}^a, \mathbf{Z}^a, \mathbf{H}_\theta^a \in \mathbb{R}^{T_a \times d_{emb}}$.

The decoder output $\mathbf{H}_\theta^a$ contains one latent representation for each action token. Each representation is obtained by combining information from the noisy action sequence itself and from the conditioning memory produced by the encoder.

Each decoder layer contains self-attention over the noisy action tokens, cross-attention over the encoded conditioning memory, and a feed-forward block. Temporal attention masks are applied inside the decoder to regulate the information flow among action tokens and between action tokens and conditioning tokens. These masks are described separately in Section 3.4.4, together with the validity masks used to handle missing keypoint data.

The attention operation follows the standard scaled dot-product form:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}} + \mathbf{S}\right) \mathbf{V}. \quad (3.33)$$

Here, $\mathbf{Q}$ is the query matrix, while $\mathbf{K}$ and $\mathbf{V}$ are the key and value matrices. For each attention head, $\mathbf{Q} \in \mathbb{R}^{n_q \times d_h}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{n_k \times d_h}$, $\mathbf{S} \in \mathbb{R}^{n_q \times n_k}$, where n_q and n_k denote the numbers of query and key/value tokens, while d_h is the per-head dimension. In the decoder, $\mathbf{S}$ corresponds either to the target temporal mask or to the memory temporal mask, depending on whether self-attention or cross-attention is being computed.

The decoder does not directly output Cartesian increments. Instead, it produces latent action features in the embedding space. A final LayerNorm followed by a linear projection maps the decoder features back to the action dimension:

$$\hat{\mathbf{e}}_\theta = \text{LayerNorm}(\mathbf{H}_\theta^a) \mathbf{W}_{out}^\top + \mathbf{1}_{T_a} \mathbf{b}_{out}. \quad (3.34)$$

with $\mathbf{W}_{out} \in \mathbb{R}^{3 \times d_{emb}}$, $\mathbf{b}_{out} \in \mathbb{R}^{1 \times 3}$, $\mathbf{1}_{T_a} \in \mathbb{R}^{T_a \times 1}$ and $\hat{\mathbf{e}}_\theta \in \mathbb{R}^{T_a \times 3}$.

The output has the same shape as the noisy action trajectory and represents the predicted Gaussian noise for each follower Cartesian increment.

In the adopted configuration, the model uses four encoder layers, four decoder layers, four attention heads, GELU activations, pre-normalization, and a feed-forward hidden dimension equal to $4d_{emb}$.

3.4.4. Attention Masks and Validity Masks

The Transformer decoder uses temporal attention masks to regulate the flow of information during the denoising process. These masks are applied inside the attention mechanism as additive terms before the softmax operation. A mask value equal to 0 leaves the corresponding attention score unchanged, while a value equal to $-\infty$ prevents attention to that token. In this way, the architecture can explicitly constrain which temporal positions are visible to each decoder token.

Two temporal attention masks are used in the decoder. The first one is the self-attention mask, which acts on the noisy action trajectory. Given action token indices $i, i' = 1, \dots, T_a$, the causal self-attention mask is defined as $\mathbf{S}^{self} \in \mathbb{R}^{T_a \times T_a}$, where

$$S_{i,i'}^{self} = \begin{cases} 0, & i' \leq i, \\ -\infty, & i' > i, \end{cases} \quad (3.35)$$

As a result, each action token can attend only to itself and to previous action tokens. This imposes a causal temporal structure inside the decoder self-attention and prevents the representation of an action token from depending on later action tokens in the sequence.

The second mask is the cross-attention mask, which regulates the access from decoder action tokens to the encoded conditioning memory. The encoder memory is composed of one diffusion timestep token followed by the conditioning observation tokens. Let $s = 0$ denote the diffusion timestep token in the encoder memory, while $s = 1, \dots, T_{obs}$ denote the encoded observation tokens. The implemented memory mask is defined as $\mathbf{S}^{mem} \in \mathbb{R}^{T_a \times (T_{obs}+1)}$, where

$$S_{i,s}^{mem} = \begin{cases} 0, & i \geq s, \\ -\infty, & i < s. \end{cases} \quad (3.36)$$

This formulation always allows every decoder token to attend to the diffusion timestep token, since for $s = 0$ the condition $i \geq 0$ is always satisfied. For the observation tokens, instead, the mask introduces a temporal prefix structure: an action token can attend only to the portion of the conditioning memory that is temporally compatible with its position in the action sequence. This avoids allowing early action tokens to access later conditioning tokens that would break the intended temporal alignment between observations and generated actions.

These temporal attention masks should not be confused with the validity masks introduced in Section 3.3.4. The attention masks operate inside the Transformer and determine which tokens can exchange information through self-attention or cross-attention. Validity masks,

instead, are data masks associated with missing numerical measurements. They indicate whether a coordinate was actually observed or whether it was introduced as a zero-filled placeholder.

In the proposed implementation, the observation validity mask is concatenated to the normalized observation values and provided to the encoder as part of the conditioning input. Therefore, it does not block attention directly, but informs the model about the reliability of each observed coordinate. The action validity mask is used during training to exclude unavailable target coordinates from the denoising objective. Consequently, validity masks handle the reliability of the data, while attention masks regulate the temporal structure of information flow inside the Transformer.

3.5. Training Procedure

After the construction of the episodic temporal windows and the definition of the conditional diffusion policy, the model is trained on demonstrated leader–follower interactions. The complete training pipeline is summarized in Figure 3.7. The diagram connects the main steps described in this section: episodic demonstrations are first converted into temporal windows, then normalized and organized into conditioning sequences and clean action trajectories. During training, the clean action trajectory is corrupted through the DDPM forward process, and the Transformer denoiser is optimized to predict the added noise from the noisy action trajectory, the sampled diffusion timestep, and the conditioning sequence.

The complete numerical configuration is reported in Appendix A.2; only the parameters needed to explain the methodology are retained here.

3.5.1. Training, Validation and Test Split

The temporal windows extracted from the demonstrations are divided into training and validation sets using a 90%–10% split. A fixed random seed is used to make this division reproducible. The test set is not sampled from the same pool of windows. Instead, a separate held-out CSV trajectory is reserved for final evaluation, preventing test samples from entering the training or validation sets.

Mini-batches preserve the episodic organization of the demonstrations. A same-episode batch sampler groups windows according to their episode index, so every mini-batch contains samples extracted from one demonstration only. During training, windows are shuffled within each episode and the resulting batches are shuffled across episodes. Shuf-

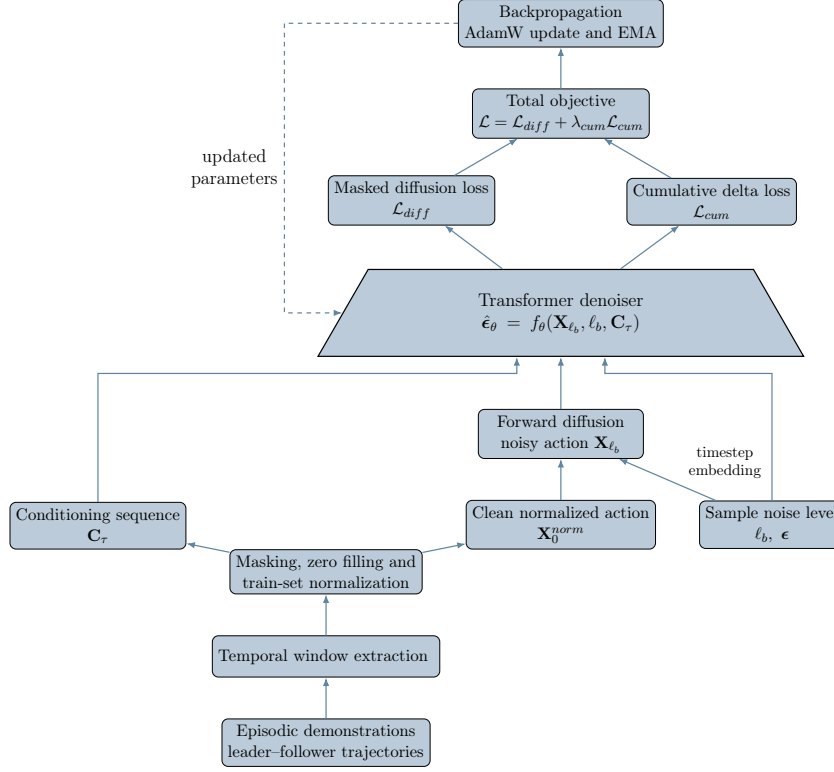


Figure 3.7: Training pipeline of the conditional diffusion policy. Episodic demonstrations are converted into temporal windows and organized into conditioning sequences and clean normalized action trajectories. At each iteration, a sampled diffusion timestep and Gaussian noise corrupt the clean action trajectory through the DDPM forward process. The Transformer denoiser predicts the added noise from the noisy trajectory, timestep, and conditioning sequence. The masked diffusion loss and auxiliary cumulative delta loss are combined and optimized with AdamW, while an exponential moving average of the model parameters is maintained.

fling is disabled during validation and testing to obtain deterministic evaluation. This organization avoids mixing demonstrations with different local motion characteristics inside the same optimization step while exposing the model to the complete set of episodes over an epoch.

3.5.2. Batch Construction and Normalization

Mini-batches use the NaN-aware representation introduced in Section 3.3.4. Missing observation and action values are replaced with zeros, while the corresponding validity matrices $\mathbf{M}^o$ and $\mathbf{M}^a$ preserve which entries were originally available. The observation validity matrix is concatenated with the normalized observation matrix and is therefore included in

the conditioning input $\mathbf{C}_\tau$. The action validity matrix is not provided to the Transformer; it is used only by the training objective to exclude unavailable target components.

Feature-wise Gaussian normalization statistics are computed from valid training-set entries only. The resulting means and standard deviations are reused without modification during validation, testing, and online inference. These statistics are saved in each checkpoint because both the conditioning observations and the generated actions must use the same scaling convention adopted during training.

3.5.3. Training Objective

Training optimizes a masked denoising objective. Let B denote the batch size and T_a the number of action tokens. The index $b = 1, \dots, B$ identifies a batch element, $i = 1, \dots, T_a$ identifies an action token, $j = 1, 2, 3$ identifies a Cartesian action component, and $r = 1, \dots, i$ is a dummy index used only to traverse the action prefix ending at token i .

For each batch element b , a diffusion timestep is sampled independently:

$$\ell_b \sim \mathcal{U}\{1, \dots, N_{train}\}, \quad N_{train} = 100. \quad (3.37)$$

The mathematical formulation uses one-based diffusion indices, with $\ell = 0$ reserved for the clean sample. The implementation uses the equivalent zero-based scheduler indices $0, \dots, N_{train} - 1$.

For batch element b , let $\mathbf{X}_0^{(b)} \in \mathbb{R}^{T_a \times 3}$ denote the clean normalized action matrix and let $\boldsymbol{\epsilon}^{(b)} \in \mathbb{R}^{T_a \times 3}$ be an independent Gaussian noise sample:

$$\boldsymbol{\epsilon}^{(b)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (3.38)$$

The clean matrix $\mathbf{X}_0^{(b)}$ corresponds to the normalized action matrix $\mathbf{A}^{norm}$ introduced during preprocessing. The DDPM forward process produces the noisy action matrix $\mathbf{X}_{\ell_b}^{(b)}$, whose element at action token i and Cartesian component j is

$$x_{\ell_b, b, i, j} = \sqrt{\bar{\alpha}_{\ell_b}} x_{0, b, i, j} + \sqrt{1 - \bar{\alpha}_{\ell_b}} \epsilon_{b, i, j}. \quad (3.39)$$

The Transformer predicts the sampled noise from the noisy action matrix, the diffusion timestep, and the corresponding conditioning matrix:

$$\hat{\boldsymbol{\epsilon}}_\theta^{(b)} = f_\theta\left(\mathbf{X}_{\ell_b}^{(b)}, \ell_b, \mathbf{C}_\tau^{(b)}\right). \quad (3.40)$$

The scalar $\hat{\epsilon}_{\theta,b,i,j}$ denotes the (i, j) -th element of $\hat{\epsilon}_{\theta}^{(b)}$.

Let $M_{b,i,j}^a$ denote the corresponding element of the action validity matrix $\mathbf{M}^{a,(b)} \in \{0, 1\}^{T_a \times 3}$. The primary diffusion loss is the mean squared noise-prediction error computed only over valid action components:

$$\mathcal{L}_{diff} = \frac{\sum_{b=1}^B \sum_{i=1}^{T_a} \sum_{j=1}^3 M_{b,i,j}^a (\hat{\epsilon}_{\theta,b,i,j} - \epsilon_{b,i,j})^2}{\max \left\{ 1, \sum_{b=1}^B \sum_{i=1}^{T_a} \sum_{j=1}^3 M_{b,i,j}^a \right\}}. \quad (3.41)$$

The denominator is the number of valid scalar action components in the batch, while the maximum with 1 reproduces the numerical safeguard used by the implementation.

Because the actions represent Cartesian increments, errors affecting consecutive tokens may accumulate after trajectory integration. The clean normalized action implied by the predicted noise is therefore reconstructed element-wise as

$$\hat{x}_{0,b,i,j} = \frac{x_{\ell_b,b,i,j} - \sqrt{1 - \bar{\alpha}_{\ell_b}} \hat{\epsilon}_{\theta,b,i,j}}{\sqrt{\bar{\alpha}_{\ell_b}}}. \quad (3.42)$$

A cumulative validity mask determines whether the complete prefix ending at token i is available:

$$\bar{M}_{b,i,j}^a = \prod_{r=1}^i M_{b,r,j}^a. \quad (3.43)$$

It is equal to 1 only if component j is valid at every action token from $r = 1$ to $r = i$. For each valid prefix, the accumulated normalized-delta error is

$$e_{b,i,j}^{cum} = \sum_{r=1}^i M_{b,r,j}^a (\hat{x}_{0,b,r,j} - x_{0,b,r,j}). \quad (3.44)$$

The auxiliary cumulative loss is consequently

$$\mathcal{L}_{cum} = \frac{\sum_{b=1}^B \sum_{i=1}^{T_a} \sum_{j=1}^3 \bar{M}_{b,i,j}^a (e_{b,i,j}^{cum})^2}{\max \left\{ 1, \sum_{b=1}^B \sum_{i=1}^{T_a} \sum_{j=1}^3 \bar{M}_{b,i,j}^a \right\}}. \quad (3.45)$$

This term is evaluated over the complete $T_a = H + O = 7$ token sequence, including the

$O = 2$ overlapping actions and the subsequent $H = 5$ future actions.

The complete objective is

$$\mathcal{L} = \mathcal{L}_{diff} + \lambda_{cum}\mathcal{L}_{cum}, \quad \lambda_{cum} = 10^{-4}. \quad (3.46)$$

The primary term trains the network to predict the Gaussian noise associated with each valid action component, while the auxiliary term discourages trajectory drift after integration.

3.5.4. Optimization and Model Selection

The network is optimized using AdamW with a batch size of 128, an initial learning rate $\eta_0 = 10^{-4}$, momentum coefficients $(\beta_1, \beta_2) = (0.95, 0.999)$, and weight decay 10^{-4} . The learning rate follows a cosine annealing schedule over the maximum $E = 3000$ training epochs:

$$\eta_e = \eta_{min} + \frac{1}{2}(\eta_0 - \eta_{min}) \left[1 + \cos\left(\frac{\pi e}{E}\right) \right], \quad (3.47)$$

where e denotes the current epoch and $\eta_{min} = 10^{-6}$ is the minimum learning rate. Before each optimizer update, the gradient norm is clipped to 1.0 to reduce unstable parameter updates.

An exponential moving average of the model parameters is maintained with decay $\rho = 0.995$:

$$\theta_e^{EMA} = \rho\theta_{e-1}^{EMA} + (1 - \rho)\theta_e, \quad (3.48)$$

where θ_e denotes the parameters after the optimizer update at epoch e . The EMA model is used during validation and subsequent inference.

Checkpoint selection is based on the masked denoising objective. Each checkpoint stores the model weights, EMA weights when enabled, loss histories, conditioning and action-column metadata, and the train-set normalization parameters required for inference.

3.6. Low-Level Controller: Impedance–Admittance Control

The low-level controller is responsible for the physical execution of the incremental Cartesian commands generated by the high-level diffusion policy. Its structure follows the Cartesian interaction-control formulation presented by Siciliano et al. [40]: an admittance layer converts the measured interaction wrench into a Cartesian motion, while an impedance layer generates the joint torques required to track the resulting reference. The specific contribution introduced in this work concerns the construction of the reference. Instead of treating the admittance displacement as a temporary offset with respect to an independently defined nominal trajectory, every admittance increment is integrated into the same persistent Cartesian target updated by the diffusion policy. Consequently, a displacement produced through physical guidance remains part of the commanded motion after contact is removed. A schematic of the control loop is shown in Figure 3.8

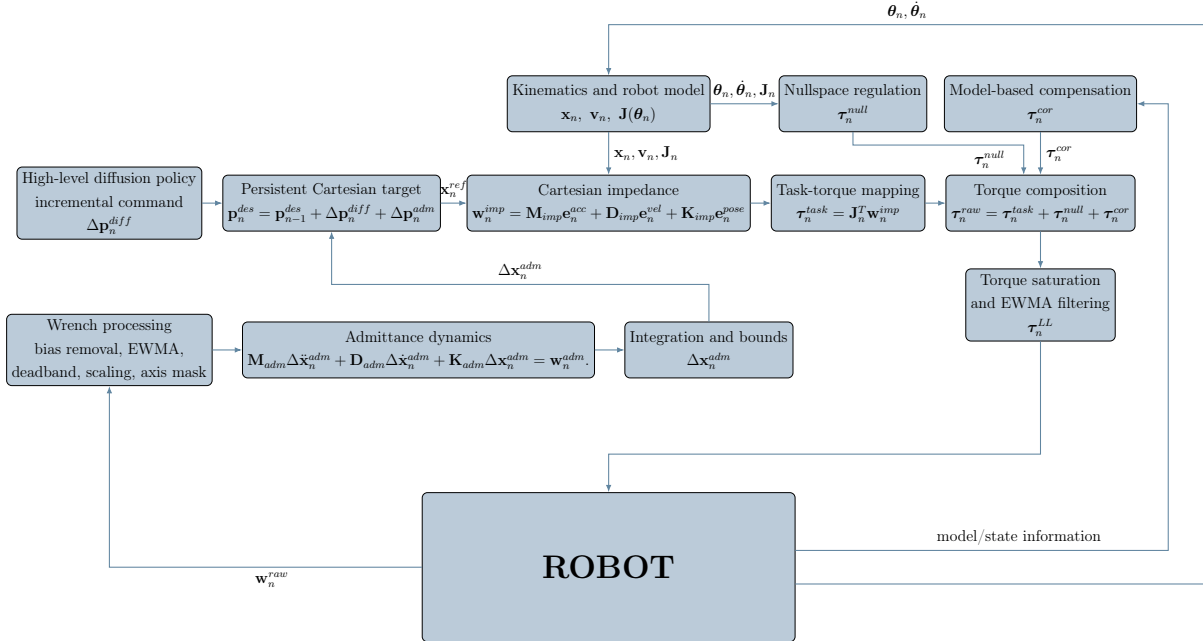


Figure 3.8: Structure of the low-level impedance–admittance controller. The diffusion policy and the admittance dynamics generate incremental Cartesian corrections that update a shared persistent target. Cartesian impedance maps the reference-tracking errors into a task wrench, which is converted into joint torques and combined with nullspace and Coriolis terms before torque-safety filtering.

Let n denote the low-level control index. The low-level loop runs at

$$f_{LL} = 1000 \text{ Hz}, \quad T_{LL} = 1 \text{ ms}. \quad (3.49)$$

The controller receives, at each step, a desired Cartesian pose

$$\mathbf{x}_n^{des} = (\mathbf{p}_n^{des}, \boldsymbol{\eta}_n^{des}), \quad (3.50)$$

where $\mathbf{p}_n^{des} \in \mathbb{R}^3$ is the desired end-effector position and $\boldsymbol{\eta}_n^{des}$ is the desired end-effector orientation quaternion. In the experiments, the Cartesian position is updated by the high-level policy, while the end-effector orientation is kept locked to the initial orientation in order to maintain a stable grasping configuration.

3.6.1. Admittance Layer

The admittance layer introduces compliant behavior by modifying the desired Cartesian reference according to the measured interaction wrench. Let

$$\mathbf{w}_n^{raw} = \begin{bmatrix} \mathbf{f}_n^{raw} \\ \boldsymbol{\tau}_n^{raw} \end{bmatrix} \in \mathbb{R}^6 \quad (3.51)$$

be the raw external wrench associated with the end-effector. In simulation, this wrench is obtained from the simulated contact interaction, whereas on the real robot it is estimated from the measured external joint torques through the robot model and Jacobian.

Before being used in the admittance dynamics, the wrench is processed to reduce noise and compensate for constant offsets. A bias term $\mathbf{b}_w$ is estimated during an initial calibration phase and subtracted from the measured signal. The resulting wrench is filtered using an exponential moving average:

$$\bar{\mathbf{w}}_n = \alpha_w \bar{\mathbf{w}}_{n-1} + (1 - \alpha_w) (\mathbf{w}_n^{raw} - \mathbf{b}_w), \quad (3.52)$$

where $\alpha_w \in [0, 1]$ is the wrench filtering coefficient.

The filtered wrench is then processed through a component-wise deadband, a scaling factor and an axis-selection mask:

$$\mathbf{w}_n^{adm} = \mathbf{S}_{adm} \gamma_w \mathcal{D}(\bar{\mathbf{w}}_n), \quad (3.53)$$

where $\mathcal{D}(\cdot)$ is the deadband operator, γ_w is a wrench scaling factor and $\mathbf{S}_{adm}$ is a diagonal matrix selecting the admittance axes. Admittance is enabled only along the translational directions:

$$\mathbf{S}_{adm} = \text{diag}(1, 1, 1, 0, 0, 0). \quad (3.54)$$

Therefore, the admittance layer modifies the position reference, while the orientation reference remains locked.

The admittance behavior is defined by a virtual mass–damper–spring system:

$$\mathbf{M}_{adm} \Delta \ddot{\mathbf{x}}_n^{adm} + \mathbf{D}_{adm} \Delta \dot{\mathbf{x}}_n^{adm} + \mathbf{K}_{adm} \Delta \mathbf{x}_n^{adm} = \mathbf{w}_n^{adm}. \quad (3.55)$$

The admittance matrices are diagonal. For each translational axis, the damping coefficient is computed as

$$D_{adm,i} = 2\zeta_{adm} \sqrt{M_{adm,i} K_{adm,i}}, \quad i \in \{x, y, z\}, \quad (3.56)$$

where ζ_{adm} is the damping factor.

The dynamics are integrated at the low-level control rate:

$$\Delta \ddot{\mathbf{x}}_n^{adm} = \mathbf{M}_{adm}^{-1} (\mathbf{w}_n^{adm} - \mathbf{D}_{adm} \Delta \dot{\mathbf{x}}_{n-1}^{adm} - \mathbf{K}_{adm} \Delta \mathbf{x}_{n-1}^{adm}), \quad (3.57)$$

$$\Delta \dot{\mathbf{x}}_n^{adm} = \Delta \dot{\mathbf{x}}_{n-1}^{adm} + \Delta \ddot{\mathbf{x}}_n^{adm} \Delta t_n, \quad (3.58)$$

$$\Delta \mathbf{x}_n^{adm} = \Delta \mathbf{x}_{n-1}^{adm} + \Delta \dot{\mathbf{x}}_n^{adm} \Delta t_n. \quad (3.59)$$

Both the admittance velocity and displacement are bounded:

$$\|\Delta \mathbf{p}_n^{adm}\|_2 \leq \Delta p_{max}, \quad \|\Delta \dot{\mathbf{p}}_n^{adm}\|_2 \leq \Delta \dot{p}_{max}. \quad (3.60)$$

This prevents contact forces or wrench estimation errors from producing excessive deviations from the nominal trajectory.

The Cartesian reference tracked by the impedance controller is then

$$\mathbf{p}_n^{ref} = \mathbf{p}_n^{des} + \Delta \mathbf{p}_n^{adm}. \quad (3.61)$$

With orientation locking enabled, the orientation reference is

$$\boldsymbol{\eta}_n^{ref} = \boldsymbol{\eta}_0, \quad (3.62)$$

where $\boldsymbol{\eta}_0$ is the end-effector orientation measured when the controller is initialized.

3.6.2. Cartesian Impedance Tracking

The corrected Cartesian reference is tracked by a Cartesian impedance controller. Let $\mathbf{p}_n$ and $\boldsymbol{\eta}_n$ be the measured end-effector position and orientation quaternion. The Cartesian pose error is

$$\mathbf{e}_n^{pose} = \begin{bmatrix} \mathbf{p}_n^{ref} - \mathbf{p}_n \\ \mathbf{e}_n^{rot} \end{bmatrix} \in \mathbb{R}^6. \quad (3.63)$$

The rotational component is computed from the relative quaternion

$$\boldsymbol{\eta}_n^{err} = \boldsymbol{\eta}_n^{ref} \otimes \boldsymbol{\eta}_n^{-1}, \quad (3.64)$$

which is converted into the rotation-vector error $\mathbf{e}_n^{rot} \in \mathbb{R}^3$.

The measured end-effector velocity is obtained from the geometric Jacobian:

$$\mathbf{v}_n = \mathbf{J}(\boldsymbol{\theta}_n) \dot{\boldsymbol{\theta}}_n. \quad (3.65)$$

The reference velocity is computed from the variation of the Cartesian reference:

$$\mathbf{v}_n^{ref} = \begin{bmatrix} \frac{\mathbf{p}_n^{ref} - \mathbf{p}_{n-1}^{ref}}{\Delta t_n} \\ \boldsymbol{\omega}_n^{ref} \end{bmatrix}, \quad (3.66)$$

where $\boldsymbol{\omega}_n^{ref}$ is obtained from the relative rotation between consecutive reference orientations. The translational and rotational components of this velocity are bounded before being used by the controller. The velocity error is

$$\mathbf{e}_n^{vel} = \mathbf{v}_n^{ref} - \mathbf{v}_n. \quad (3.67)$$

Reference and measured accelerations are estimated from consecutive twists. The resulting signals are norm-bounded and filtered through an exponential moving average before computing

$$\mathbf{e}_n^{acc} = \mathbf{a}_n^{ref} - \mathbf{a}_n. \quad (3.68)$$

The desired task-space wrench is then generated by

$$\mathbf{w}_n^{imp} = \mathbf{M}_{imp} \mathbf{e}_n^{acc} + \mathbf{D}_{imp} \mathbf{e}_n^{vel} + \mathbf{K}_{imp} \mathbf{e}_n^{pose}, \quad (3.69)$$

where $\mathbf{M}_{imp}$, $\mathbf{D}_{imp}$ and $\mathbf{K}_{imp}$ are diagonal Cartesian inertia, damping and stiffness matrices. The corresponding task torque is obtained through the transpose of the geometric Jacobian:

$$\boldsymbol{\tau}_n^{task} = \mathbf{J}^T(\boldsymbol{\theta}_n) \mathbf{w}_n^{imp}. \quad (3.70)$$

3.6.3. Nullspace Control and Torque Safety

A nullspace torque is added to exploit the redundancy of the 7-DoF manipulator without directly interfering with the Cartesian task. The nullspace action biases the joints toward the centre of their admissible range:

$$\boldsymbol{\tau}_n^{null,raw} = \mathbf{K}_{null} (\boldsymbol{\theta}_{mid} - \boldsymbol{\theta}_n) - \mathbf{D}_{null} \dot{\boldsymbol{\theta}}_n. \quad (3.71)$$

The torque is projected into the approximate nullspace of the Cartesian task using a damped Jacobian-based projector:

$$\mathbf{N}(\boldsymbol{\theta}_n) = \mathbf{I} - \mathbf{J}^T(\boldsymbol{\theta}_n) (\mathbf{J}(\boldsymbol{\theta}_n) \mathbf{J}^T(\boldsymbol{\theta}_n) + \lambda^2 \mathbf{I})^{-1} \mathbf{J}(\boldsymbol{\theta}_n), \quad (3.72)$$

$$\boldsymbol{\tau}_n^{null} = \mathbf{N}(\boldsymbol{\theta}_n) \boldsymbol{\tau}_n^{null,raw}. \quad (3.73)$$

The commanded torque before safety filtering is

$$\boldsymbol{\tau}_n^{raw} = \boldsymbol{\tau}_n^{task} + \boldsymbol{\tau}_n^{null} + \boldsymbol{\tau}_n^{cor}, \quad (3.74)$$

where $\boldsymbol{\tau}_n^{cor}$ is the Coriolis compensation supplied by the robot model. To avoid discontinuous actuation, the variation of each commanded joint torque is bounded with respect to the latest desired torque accepted by the robot interface:

$$\boldsymbol{\tau}_n^{sat} = \boldsymbol{\tau}_n^{prev} + \text{clip}(\boldsymbol{\tau}_n^{raw} - \boldsymbol{\tau}_n^{prev}, -\Delta\boldsymbol{\tau}_{max}, \Delta\boldsymbol{\tau}_{max}). \quad (3.75)$$

The torque command is finally filtered using an exponential moving average:

$$\boldsymbol{\tau}_n^{LL} = \alpha_\tau \boldsymbol{\tau}_{n-1}^{LL} + (1 - \alpha_\tau) \boldsymbol{\tau}_n^{sat}. \quad (3.76)$$

The resulting torque command τ_n^{LL} is sent either to the simulated robot interface or directly to the real Franka torque-control loop.

The controller combines Cartesian impedance tracking, force-based admittance correction and nullspace posture regulation. The values of the parameters of the implemented low-level controller are summarized in Table 3.1.

Table 3.1: Default parameters of the real-robot low-level controller. EWMA denotes an exponentially weighted moving average filter.

Parameter	Value	Description
α_w^F	0.75	EWMA coefficient used to filter the reconstructed external wrench.
α_w^τ	0.35	EWMA coefficient used to filter the commanded joint torques.
t_{bias}, T_{bias}	1.0, 3.0	Start delay and duration of the wrench-bias calibration phase.
m_t^{adm}	0.5	Virtual translational mass of the admittance model.
d_t^{adm}	180.0	Virtual translational damping terms of the admittance model.
k_t^{adm}	20.0	Virtual translational stiffness terms of the admittance model.
F_{db}	2.5	Force deadband used to reduce the effect of small wrench fluctuations.
γ_w	5.0	Scaling factor applied to the force-driven admittance correction.
$\Delta p_{max}^{adm}, \Delta \dot{p}_{max}^{adm}$	0.1, 0.5	Bounds on admittance displacement and velocity.
m_t^{imp}, m_r^{imp}	3.0, 0.02	Cartesian translational and rotational virtual mass of the impedance law.
d_t^{imp}, d_r^{imp}	10, 5	Cartesian translational and rotational virtual damping of the impedance law.
k_t^{imp}, k_r^{imp}	300, 30	Cartesian translational and rotational virtual stiffness of the impedance law.
k_{null}, d_{null}	15.0, 4.0	Joint-space stiffness and damping used for nullspace posture regulation.
λ	0.08	Nullspace damping coefficient used in the nullspace projection.

3.7. Online Execution Architecture

After training, the diffusion policy is deployed as the high-level motion generator of the robotic system. Its role is not to generate joint torques directly, but to predict short-horizon Cartesian position increments for the robot end-effector. These increments are produced in the same model frame used during training and are then transformed into Cartesian increments expressed in the robot frame for the low-level impedance–admittance controller.

The online inference pipeline follows a sequence of steps. First, each RGB-D frame is processed to extract the leader keypoints and compute their Cartesian increments. At the same time, the most recent robot feedback is used to compute the follower increment associated with the robot end-effector. Since the diffusion model and the robot controller operate in different coordinate frames, the robot feedback increment is mapped from the robot frame to the model frame before being inserted into the observation vector. The resulting observation is appended to a temporal buffer, which provides the conditioning window for the diffusion policy.

At each high-level update, the diffusion policy predicts a short sequence of future follower increments in the model frame. Following a receding-horizon strategy, only the first non-overlapping future increment is selected. This increment is mapped into the robot frame, bounded to limit the commanded Cartesian displacement, and published to the low-level controller. The low-level controller then integrates the diffusion increment together with the increment generated by the admittance dynamics into its Cartesian position state, as illustrated in Figure 3.9.

The online architecture is multi-rate. The perception and diffusion loop is indexed by k and runs at the camera rate:

$$f_D \simeq 30 \text{ Hz}, \quad T_D \simeq 33.3 \text{ ms}. \quad (3.77)$$

The low-level controller is indexed by n and runs at the nominal torque-control frequency:

$$f_{LL} = 1000 \text{ Hz}, \quad T_{LL} = 1 \text{ ms}. \quad (3.78)$$

At the nominal camera rate, the number of low-level iterations between two consecutive diffusion updates is approximately $N_{LL/D} \simeq \frac{f_{LL}}{f_D} \simeq 33$ control iterations.

In this section, only Cartesian positions and position increments are described explicitly. End-effector orientation locking, admittance correction, Cartesian impedance tracking

and torque generation are handled by the low-level controller described in Section 3.6.

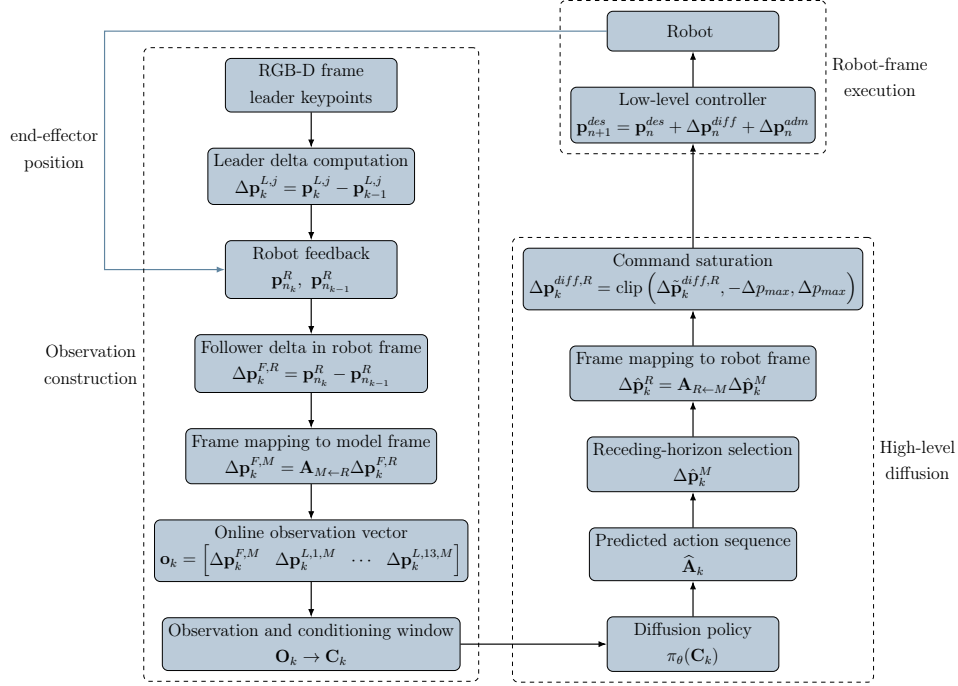


Figure 3.9: Online inference and execution pipeline. The RGB-D perception module extracts the leader keypoints and constructs the conditioning sequence. The diffusion policy predicts a sequence of Cartesian increments in the model frame, from which the first non-overlapping future increment is selected. The selected increment is mapped into the robot frame, bounded and published to the low-level controller. When it becomes available, the controller applies the diffusion increment once and combines it with the admittance contribution in the recursive update of the desired Cartesian position.

3.7.1. Cartesian Projection and Frame Alignment

A central requirement of the online diffusion node is frame consistency. The diffusion policy was trained using Cartesian increments expressed in the same camera-based convention adopted during dataset construction. During online execution, however, the low-level controller receives Cartesian increments expressed in the robot base frame, therefore, the vision node must align the robot feedback, the human observations and the predicted diffusion command before they are used online.

To keep the notation compact, two frames are considered in this section. The model frame, denoted by M , is the frame convention used by the diffusion policy and by the RGB-D keypoint dataset. The robot frame, denoted by R , is the robot base frame used by the low-level controller. The linear mapping from the robot frame to the model frame

is denoted by $\mathbf{A}_{M \leftarrow R}$, while the inverse mapping is denoted by $\mathbf{A}_{R \leftarrow M}$.

The frame alignment is obtained through an AprilTag-based calibration procedure. Since the physical size of the AprilTag is known, the RGB-D camera can estimate the tag pose in 3D space. By placing the tag with axes aligned to the robot reference directions, the estimated tag pose provides the geometric relation between the camera-based model frame and the robot base frame. The mapping $\mathbf{A}_{M \leftarrow R}$ also includes the axis convention used during dataset construction, including the sign change applied to the camera vertical axis.

The transformation is applied to Cartesian increments. Therefore, only the linear part of the frame mapping is required, and no translation term appears in the following equations. First, the robot feedback increment is computed in the robot frame:

$$\Delta \mathbf{p}_k^{F,R} = \mathbf{p}_{n_k}^R - \mathbf{p}_{n_{k-1}}^R. \quad (3.79)$$

It is then mapped into the model frame before being inserted into the online observation vector:

$$\Delta \mathbf{p}_k^{F,M} = \mathbf{A}_{M \leftarrow R} \Delta \mathbf{p}_k^{F,R}. \quad (3.80)$$

This ensures that the follower feedback and the leader keypoint increments are expressed in the same frame convention used during training.

The diffusion policy then predicts a Cartesian increment in the model frame, denoted by $\Delta \hat{\mathbf{p}}_k^M$. Before this increment can be used as a robot command, it is mapped back to the robot frame:

$$\Delta \mathbf{p}_k^{diff,R} = \mathbf{A}_{R \leftarrow M} \Delta \hat{\mathbf{p}}_k^M. \quad (3.81)$$

In this way, the diffusion model operates in the same representation used during training, while the low-level controller receives Cartesian increments expressed in the robot base frame.

3.7.2. Real-Time Observation Buffer

At runtime, the observation buffer is updated at the perception rate. For each RGB-D frame, the vision node extracts the leader keypoints and computes their Cartesian increments. For the j -th leader keypoint, the increment is

$$\Delta \mathbf{p}_k^{L,j} = \mathbf{p}_k^{L,j} - \mathbf{p}_{k-1}^{L,j}. \quad (3.82)$$

The leader keypoint increments are already expressed in the model frame M , because they

are obtained from the RGB-D perception pipeline using the same coordinate convention adopted during training.

The follower signal used online is obtained from robot feedback and then mapped to the model frame using the alignment introduced in Section 3.7.1. The follower increment used in the observation vector is therefore

$$\Delta \mathbf{p}_k^{F,M} = \mathbf{A}_{M \leftarrow R} \left(\mathbf{p}_{n_k}^R - \mathbf{p}_{n_{k-1}}^R \right). \quad (3.83)$$

This preserves the same delta-based representation used during training.

The online observation vector is

$$\mathbf{o}_k = \left[\Delta \mathbf{p}_k^{F,M} \quad \Delta \mathbf{p}_k^{L,1,M} \quad \Delta \mathbf{p}_k^{L,2,M} \quad \dots \quad \Delta \mathbf{p}_k^{L,13,M} \right] \in \mathbb{R}^{42}. \quad (3.84)$$

Missing keypoints are handled through the same mask-based mechanism used during training. The temporal observation window is

$$\mathbf{O}_k = \begin{bmatrix} \mathbf{o}_{k-T_{obs}} \\ \mathbf{o}_{k-T_{obs}+1} \\ \vdots \\ \mathbf{o}_{k-1} \end{bmatrix} \in \mathbb{R}^{T_{obs} \times 42}. \quad (3.85)$$

The diffusion policy receives the normalized conditioning

$$\mathbf{C}_k = \begin{bmatrix} \mathbf{c}_{k-T_{obs}} \\ \vdots \\ \mathbf{c}_{k-1} \end{bmatrix}, \quad \mathbf{c}_q = \begin{bmatrix} \mathbf{o}_q^{norm} & \mathbf{m}_q^o \end{bmatrix}. \quad (3.86)$$

3.7.3. Receding-Horizon Diffusion Prediction

At each high-level step k , the diffusion policy predicts a short sequence of future follower Cartesian increments. The action sequence has length

$$T_a = O + H, \quad (3.87)$$

where O is the overlap region and H is the future prediction horizon. The predicted action sequence is written as

$$\hat{\mathbf{A}}_k = \begin{bmatrix} \Delta \hat{\mathbf{p}}_{k-O}^M \\ \dots \\ \Delta \hat{\mathbf{p}}_{k-1}^M \\ \Delta \hat{\mathbf{p}}_k^M \\ \dots \\ \Delta \hat{\mathbf{p}}_{k+H-1}^M \end{bmatrix}. \quad (3.88)$$

Only the first non-overlapping future increment, $\Delta \hat{\mathbf{p}}_k^M$, is selected.

At the next camera frame, the observation buffer is shifted forward and a new denoising process is performed. This gives the high-level controller a receding-horizon behavior: the robot does not follow a long open-loop trajectory, but continuously updates its local motion reference according to the most recent human motion observations.

3.7.4. Diffusion Increment Transformation and Saturation

The selected increment is mapped from the model frame to the robot frame:

$$\Delta \tilde{\mathbf{p}}_k^{diff,R} = \mathbf{A}_{R \leftarrow M} \Delta \hat{\mathbf{p}}_k^M. \quad (3.89)$$

Before publication, each Cartesian component is bounded:

$$\Delta \mathbf{p}_k^{diff,R} = \text{clip} \left(\Delta \tilde{\mathbf{p}}_k^{diff,R}, -\Delta p_{max}, \Delta p_{max} \right). \quad (3.90)$$

The resulting increment is published to the low-level controller. An integrated Cartesian pose is also maintained by the high-level node for visualization and monitoring purposes, but it does not constitute the command used by the torque controller.

3.7.5. Multi-Rate Coupling with the Low-Level Controller

The diffusion policy and the low-level controller operate at different frequencies. The admittance contribution is computed at every low-level iteration, whereas a new diffusion contribution becomes available only when the high-level inference process is completed.

Let ν_k denote the low-level iteration at which the diffusion increment generated at high-level step k becomes available to the controller. The diffusion contribution applied at low-level iteration n is defined as

$$\Delta \mathbf{p}_n^{diff} = \begin{cases} \Delta \mathbf{p}_k^{diff,R}, & n = \nu_k, \\ \mathbf{0}, & \text{otherwise.} \end{cases} \quad (3.91)$$

At the nominal diffusion frequency of 30 Hz and low-level frequency of 1000 Hz, two consecutive diffusion updates are separated by approximately

$$\nu_{k+1} - \nu_k \simeq \frac{f_{LL}}{f_D} \simeq 33 \quad (3.92)$$

low-level iterations.

At every low-level iteration, the desired Cartesian position is updated recursively according to

$$\mathbf{p}_{n+1}^{des} = \mathbf{p}_n^{des} + \Delta \mathbf{p}_n^{adm} + \Delta \mathbf{p}_n^{diff}. \quad (3.93)$$

Therefore, $\Delta \mathbf{p}_n^{adm}$ contributes to the position update at each low-level iteration, while $\Delta \mathbf{p}_n^{diff}$ is non-zero only when a new diffusion prediction is available. Once the update has been performed, $\mathbf{p}_{n+1}^{des}$ is stored and becomes the initial desired position for the following iteration.

The Cartesian reference tracked by the impedance controller is

$$\mathbf{p}_{n+1}^{ref} = \mathbf{p}_{n+1}^{des}. \quad (3.94)$$

In this way, the diffusion increment is applied only once when it becomes available; it is not repeatedly integrated during the subsequent low-level iterations. Its effect is nevertheless retained because it has already been incorporated into the updated desired position.

3.7.6. ROS2 Communication Architecture

The high-level diffusion policy and the low-level impedance–admittance controller are implemented as separate ROS2 nodes. Their main communication loop is based on two topics. The vision–diffusion node publishes the Cartesian increment selected by the policy on `/delta_diffusion` using a `geometry_msgs/Vector3Stamped` message. The low-level controller receives this increment and incorporates it into the recursive update of the desired Cartesian position.

In the opposite direction, the controller publishes the measured end-effector pose on `/actual_ee_pose` using a `geometry_msgs/PoseStamped` message. This feedback is received by the high-level node and is used for online state reconstruction and diagnostic pose integration. The direct communication with the RGB-D camera and the Franka robot is handled through their respective hardware interfaces and therefore does not introduce additional ROS2 topics into the main control loop. Figure 3.10 summarizes this communication structure between camera, ROS2 nodes and robot.

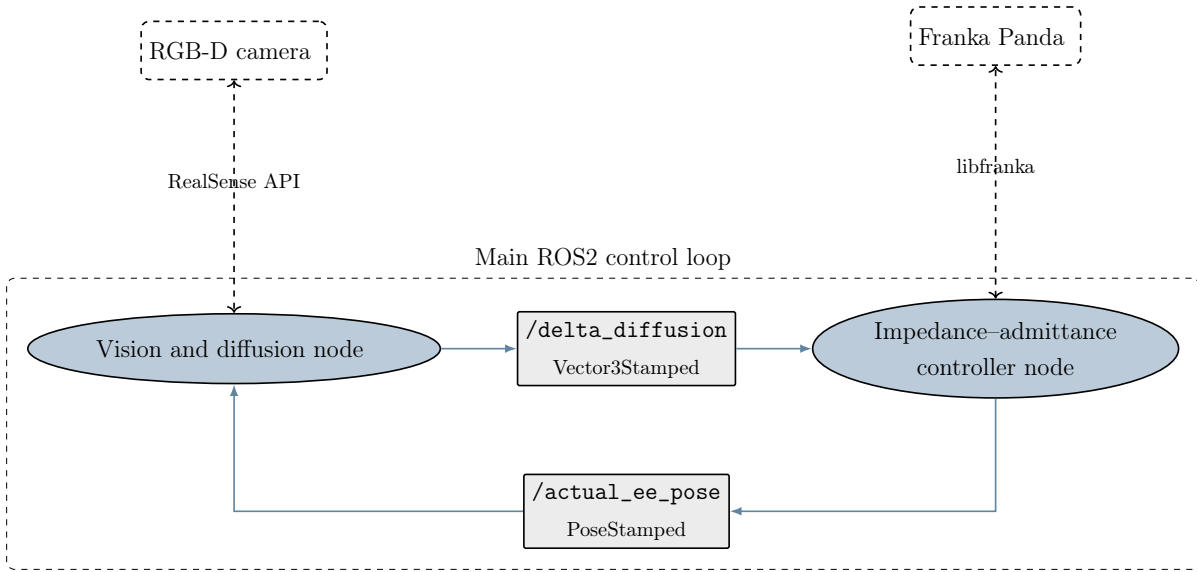


Figure 3.10: ROS2 communication graph of the online robotic system. The diffusion increment is transmitted from the high-level node to the low-level controller through `/delta_diffusion`, while `/actual_ee_pose` provides end-effector feedback in the opposite direction. Dashed connections represent direct hardware interfaces rather than ROS2 topics.

4 | Results

The proposed system was designed for collaborative object transport, where the human and the robot jointly manipulate a shared load. In this context, the objective is not only to reproduce a desired trajectory, but also to support the human operator by anticipating the intended cooperative motion and reducing the interaction effort required to guide the object. The RGB-D perception pipeline observes the motion of the human leader, the diffusion policy predicts a compatible follower motion, and the impedance–admittance controller executes the generated reference while preserving compliant physical interaction.

The evaluation of the system is therefore carried out at different levels; however, all the metrics presented in this chapter are computed on denormalized data. Section 4.1 analyses the individual perception and prediction modules. The RGB-D perception pipeline is evaluated in terms of reference keypoint-detection performance, qualitative behavior and practical limitations related to depth-based three-dimensional reconstruction.

The offline diffusion policy is then assessed on held-out test sequences by comparing the predicted follower wrist increments with the corresponding ground-truth motion in Section 4.2.

The second part of the chapter moves from component-level evaluation to closed-loop execution. In Section 4.3, a MuJoCo simulation is used to test the complete pipeline under controlled conditions and to compare the trajectory executed by the simulated robot with the human follower trajectory recorded in the demonstrations.

The real-robot evaluation, in Section 4.4, analyses one representative full-pipeline execution to verify the online behavior of the perception, prediction and control modules during physical collaboration.

Finally, a comparative study, presented in Section 4.5, compares the complete diffusion-assisted architecture with a low-level impedance–admittance baseline in which the diffusion policy is disabled. This comparison is used to quantify the effect of the predictive high-level policy on the physical interaction forces required from the human operator. Overall, the chapter evaluates the system in terms of perception reliability, offline prediction accuracy, simulated trajectory reproduction, real-time execution behavior and

interaction-force reduction.

4.1. RGB-D Perception Performance

The RGB-D perception pipeline combines YOLO26-based human pose estimation with depth measurements provided by the Intel RealSense camera. Since the pose models were used with their publicly released pretrained weights, their reference accuracy was assessed from the official COCO-Keypoints benchmark rather than through model retraining on the collaborative-carrying dataset.

COCO evaluates pose estimation using Object Keypoint Similarity (OKS), which accounts for the distance between predicted and annotated keypoints, the scale of the detected person and the localization uncertainty associated with each body joint. Table 4.1 reports the official validation results for the two model variants used in this work. The larger model was employed during offline dataset extraction, where localization accuracy was prioritized, whereas the smaller model was selected for online execution to reduce latency.

Table 4.1: Published COCO-Keypoints validation performance of the YOLO26 pose models used in the perception pipeline [45].

Model	Application	$\text{mAP}_{50:95}^{\text{pose}}$	$\text{mAP}_{50}^{\text{pose}}$	Parameters
YOLO26x-Pose	Offline extraction	71.6	91.6	57.6 M
YOLO26s-Pose	Online inference	63.0	86.6	10.4 M

The results indicate that the smaller model preserves a high keypoint-detection accuracy while substantially reducing model complexity. On an NVIDIA 5080 with TensorRT, Ultralytics reports reference inference times of 20.2 ms for YOLO26x-Pose and 8.3 ms for YOLO26s-Pose [45].

In addition to the published COCO-Keypoints metrics, qualitative examples of the pose-estimation output are shown in Fig. 4.1 and Fig. 4.2. The first example refers to the offline dataset extraction phase and shows the subset of human keypoints extracted from the human–human demonstrations and used as input features for the diffusion model training. The second example refers to the online inference scenario, where the YOLO26 pose-estimation output is visualized together with the detected bounding box. In the second example, the AprilTag marker used for camera-to-robot frame calibration is also visible.

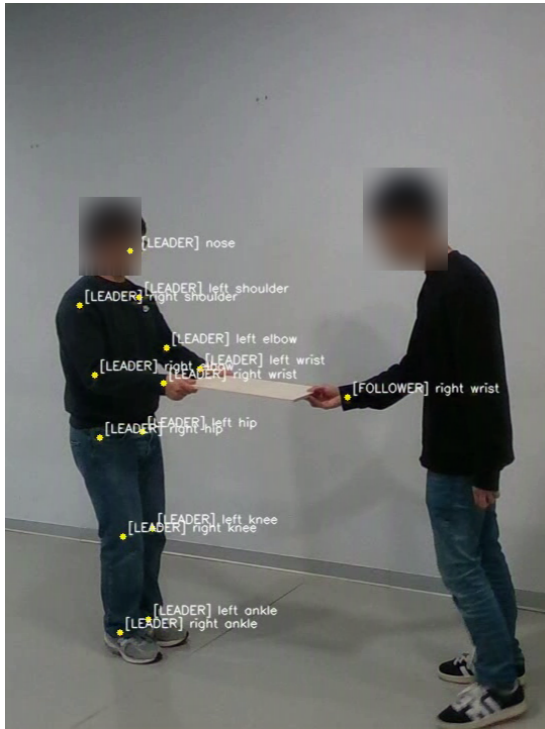


Figure 4.1: Offline human-human keypoint extraction used to build the training datasets.

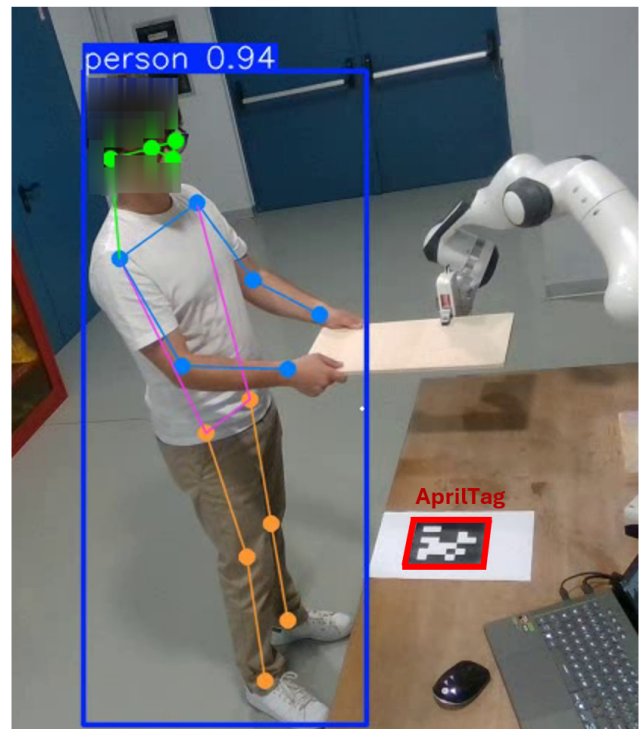


Figure 4.2: Online human-robot perception output during the co-manipulation with the robot.

The conversion from two-dimensional keypoints to three-dimensional coordinates introduces an additional source of uncertainty associated with depth sensing. For the D415 camera family, the manufacturer reports a depth Z -accuracy of approximately $\pm 2\%$ under the specified calibration and operating conditions [15]. Consequently, the published YOLO metrics describe only image-plane keypoint localization and do not represent the complete accuracy of the reconstructed 3D trajectories.

4.2. Offline Model Prediction Accuracy

This section evaluates the ability of the diffusion policy to predict the Cartesian motion increments of the follower right wrist. Since the output is a continuous trajectory increment rather than a discrete class, performance is quantified using complementary regression metrics: mean squared error (MSE), mean absolute error (MAE), root mean squared error (RMSE), and the coefficient of determination R^2 .

4.2.1. Model Evaluation Protocol

The evaluation was conducted using the EMA model trained with the hyperparameters listed in Table A.2. The normalization statistics stored from the training set were applied unchanged to all test data; consequently, no information from the test sequences was used to rescale the model inputs or targets.

The offline evaluation was performed on three independent zero-shot episodic datasets, which contain different motion sequences and therefore provide a more informative assessment of generalization than a single test trajectory. Overall, the three datasets contained 5034 timesteps and resulted in 198 independent prediction windows, as summarized in Table 4.2. Each prediction window covered a temporal sequence of $S_{\text{length}} = 35$ timesteps. The first $T_{\text{obs}} = 30$ timesteps were used as conditioning information, while the remaining $H = 5$ timesteps represented the future trajectory predicted by the diffusion policy. Since the data were acquired at 30 Hz, corresponding to the frame rate of the RGB-D camera, the full window corresponded to approximately 1.17 s, with 1.00 s used for conditioning and 0.17 s used for future prediction. An action–observation overlap of $O = 2$ timesteps was also included, so that the action window had length $T_a = 7$ timesteps.

Test episode	Timesteps in the episode	Number of sequences	Valid target timesteps
Manipulation1	2851	113	2322
Manipulation2	953	37	759
Manipulation3	1230	48	1002
Total	5034	198	4083

Table 4.2: Composition of the three held-out test episodes used to evaluate the model. A valid target value denotes one non-missing scalar coordinate included in the metric computation.

Each prediction window was evaluated independently; therefore, the reported quantities measure single-window predictive accuracy and are not affected by error accumulation across successive predictions.

Let $\mathcal{V}$ be the set of valid scalar target coordinates, with $N_v = |\mathcal{V}|$, and let Δy_i and $\Delta \hat{y}_i$ denote the measured and predicted Cartesian increments. The error metrics are defined as

$$\text{MSE} = \frac{1}{N_v} \sum_{i \in \mathcal{V}} (\Delta \hat{y}_i - \Delta y_i)^2, \quad (4.1)$$

$$\text{MAE} = \frac{1}{N_v} \sum_{i \in \mathcal{V}} |\Delta \hat{y}_i - \Delta y_i|, \quad \text{RMSE} = \sqrt{\text{MSE}}. \quad (4.2)$$

Finally, the coefficient of determination is

$$R^2 = 1 - \frac{\sum_{i \in \mathcal{V}} (\Delta y_i - \Delta \hat{y}_i)^2}{\sum_{i \in \mathcal{V}} (\Delta y_i - \Delta \bar{y})^2}, \quad (4.3)$$

where $\Delta \bar{y}$ is the mean of the valid ground-truth targets.

4.2.2. Prediction Results

Table 4.3 reports the results for each test sequence and for the pooled test set. The pooled metrics were computed on denormalized data, directly over all 4083 valid target values and are therefore weighted by the amount of valid data in each sequence.

Test sequence	MSE [mm ²]	MAE [mm]	RMSE [mm]	R^2
Manipulation1	5.249	1.449	2.291	0.8849
Manipulation2	3.359	1.198	1.833	0.9301
Manipulation3	5.986	1.505	2.447	0.8171
Pooled test set	5.078	1.416	2.253	0.8817

Table 4.3: Offline prediction metrics. Errors are computed on Cartesian wrist-motion increments.

On the complete test set, the model achieved an MAE of 1.416 mm, an RMSE of 2.253 mm, and a pooled coefficient of determination of $R^2 = 0.8817$. These metrics indicate that the model captured a substantial proportion of the variation in the follower wrist increments across previously unseen sequences.

4.2.3. Qualitative Analysis of Predicted Cartesian Increments

While the scalar metrics quantify the average prediction error, they do not show whether the model reproduces the direction, amplitude, and temporal evolution of the motion. Therefore, the predicted Cartesian increments were also compared qualitatively with the corresponding ground-truth values for each test sequence. Each figure shows window predictions where the model output is fed back into subsequent predictions.

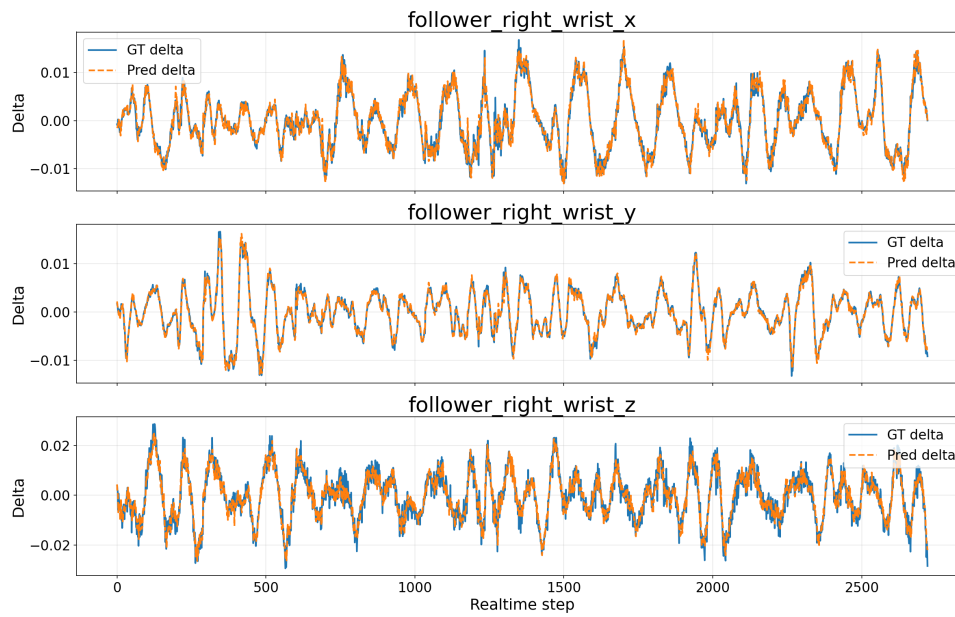


Figure 4.3: Ground-truth and predicted follower wrist increments along the x , y , and z directions for Manipulation1

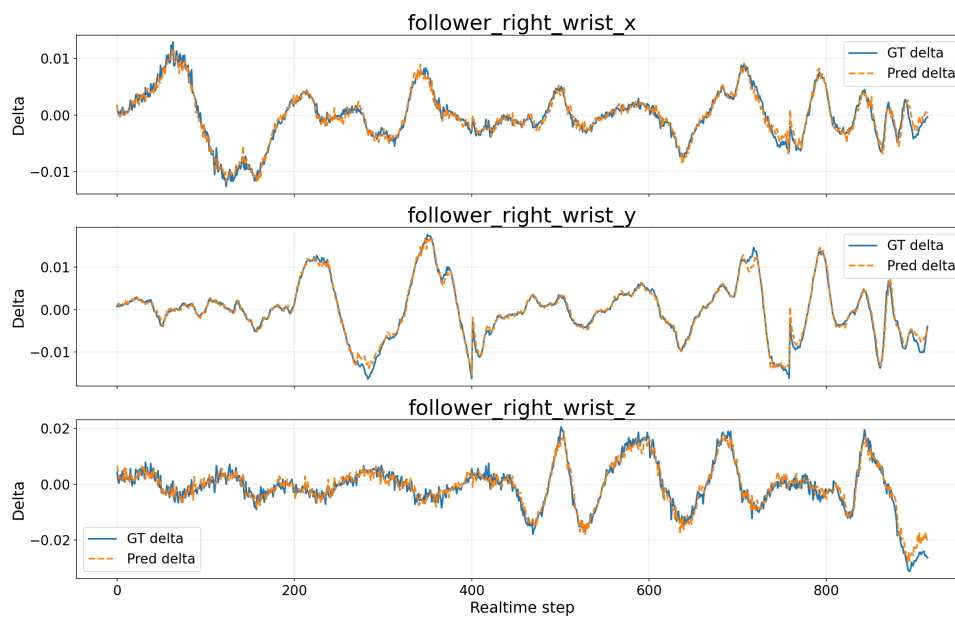


Figure 4.4: Ground-truth and predicted follower wrist increments along the x , y , and z directions for Manipulation2

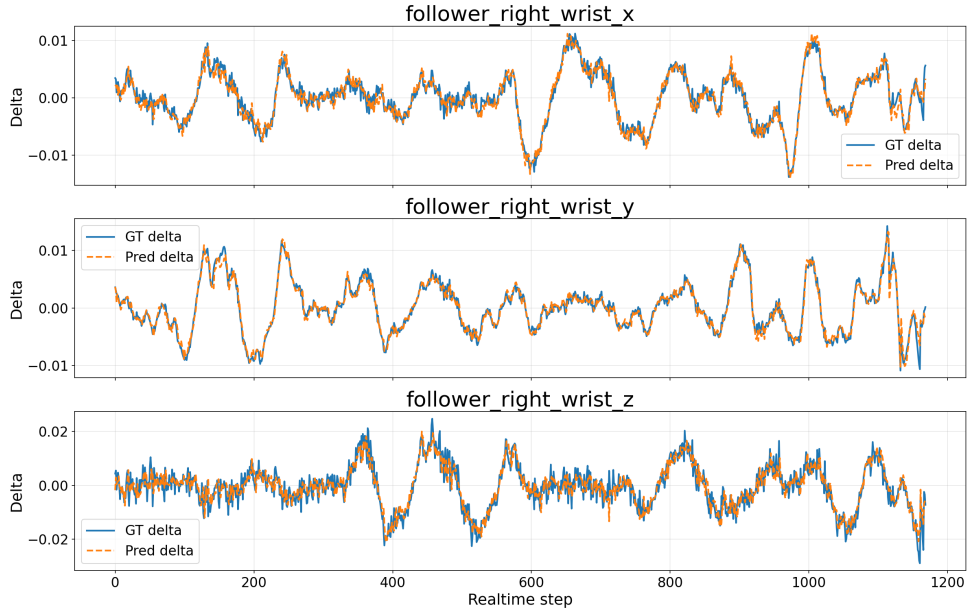


Figure 4.5: Ground-truth and predicted follower wrist increments along the x , y , and z directions for Manipulation3

The qualitative comparison supports the numerical results reported in Table 4.3. Overall, the model reproduces the main direction and temporal structure of the demonstrated motion.

4.2.4. Coordinate-Wise Error Analysis

To identify whether the prediction error was concentrated along a particular Cartesian direction, the pooled MAE and RMSE were also computed separately for the x , y , and z wrist increments.

Coordinate	MAE [mm]	RMSE [mm]
x	1.138	1.688
y	0.831	1.317
z	2.280	3.264

Table 4.4: Coordinate-wise prediction error of the model, pooled over the three held-out test sequences.

The y component was predicted most accurately, with an RMSE of 1.317 mm. The z component, instead, exhibited the largest error, reaching an RMSE of 3.264 mm. This behavior is consistent with the additional uncertainty introduced by depth-based 3D reconstruction.

Overall, the results show that the diffusion policy can reproduce local follower wrist-motion increments with millimeter-scale error on three unseen sequences. The consistency of the results across the datasets supports the model’s ability to generalize beyond the training recordings.

4.3. Simulation Results

4.3.1. Collaborative Simulation Scenario

The collaborative task was reproduced in MuJoCo using a simulated Franka Panda robot and a rigid plate representing the jointly manipulated object. The plate has dimensions $0.48 \times 0.32 \times 0.03$ m and a mass of 1.2 kg. The robot grasps one side of the plate while the human operator is holding the other side.

As shown in Figure 4.6, the human hands are represented by two cyan spheres positioned at the corresponding grasping points. The spheres are visual markers without collision properties. They belong to a common human grasp frame connected to the plate through a MuJoCo weld constraint.

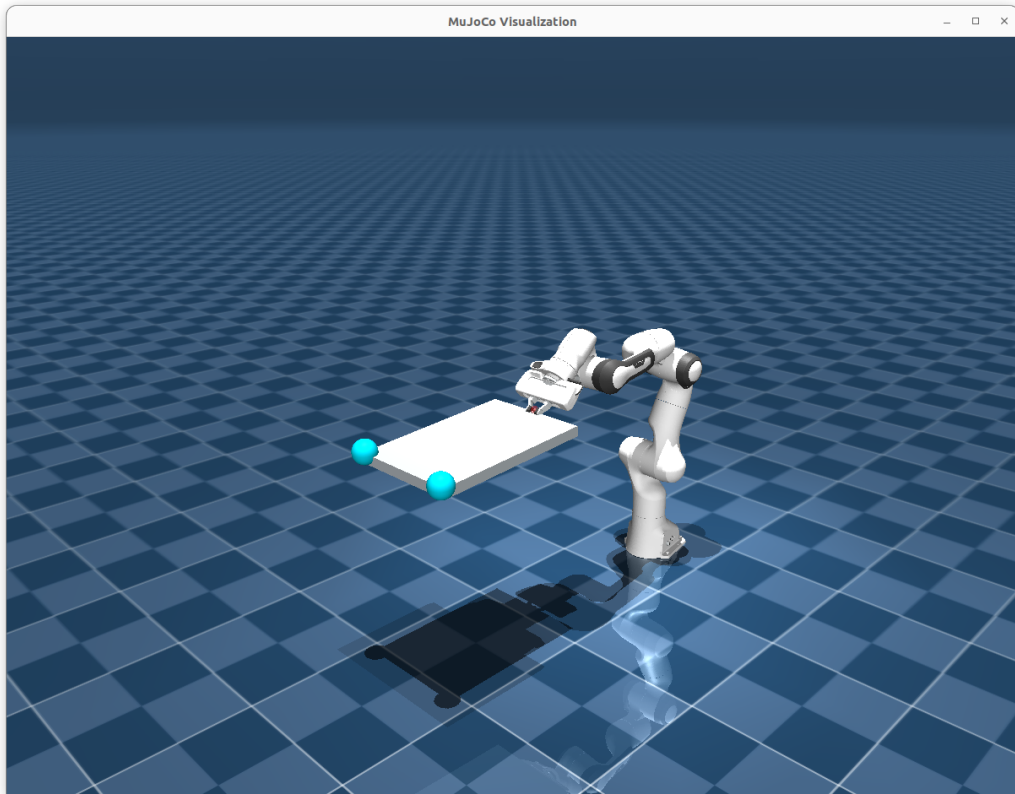


Figure 4.6: MuJoCo collaborative manipulation scenario.

The setup allows the recorded human wrist motion to drive one side of the plate, while the Franka Panda reacts through the same impedance–admittance control structure used in the complete pipeline. In this way, the simulation provides a controlled intermediate validation between offline prediction accuracy and real-robot execution.

The translational motion of the human hands frame is obtained from the average displacement of the left and right human wrists:

$$\Delta \tilde{\mathbf{p}}_k^L = \frac{1}{2} \left(\Delta \mathbf{p}_k^{L,\text{left}} + \Delta \mathbf{p}_k^{L,\text{right}} \right). \quad (4.4)$$

The offline dataset used for the simulation contains both the recorded leader trajectory and the recorded human-follower trajectory. However, in the simulation experiment, the recorded human-follower motion was not provided to the policy as part of the conditioning input. Instead, at each simulation timestep k , the conditioning window was constructed using the recorded leader displacements from the held-out sequence and the recent displacements actually executed by the simulated robot, which represented the follower:

$$\mathbf{c}_k = \{ \Delta \mathbf{p}_i^L, \Delta \mathbf{p}_i^{F,\text{robot}} \}_{i=k-T_{\text{obs}}}^{k-1}, \quad (4.5)$$

where k denotes the current simulation timestep at which the policy is queried, T_{obs} is the number of past observation steps used in the conditioning window, $\Delta \mathbf{p}_i^L$ denotes the recorded leader displacement at timestep i , and $\Delta \mathbf{p}_i^{F,\text{robot}}$ denotes the displacement actually executed by the robotic follower at timestep i .

In this way, the policy was evaluated in closed-loop simulation without access to the ground-truth follower motion, while still allowing the simulated robotic trajectory to be compared directly with the original human-follower demonstration.

4.3.2. ROS 2 Execution

The simulation experiment was executed through ROS 2 coordinating three main nodes. The `mujoco-Ros2` loads the robot and collaborative scene and receives the joint torque commands and the human wrist displacements, while publishing the robot joint state and the wrench measured at the end-effector.

The `impadm_controller` implements the Cartesian impedance–admittance controller that runs at 1000 Hz. It receives the desired end-effector Cartesian pose, the simulated joint state and the measured wrench, and computes the joint torques to track the desired

Cartesian positions. Before the collaborative motion begins, a 5 s calibration procedure estimates the wrench bias.

Finally, the `diffusion_offline_node` executes the diffusion policy at 30 Hz using a previously recorded collaborative sequence. The node publishes the leader wrist displacements that are used to move the simulated human hands at each step, to mimic a real experiment with a human behaving unpredictably. This node also publishes the Cartesian target position generated by the model for the robot follower to track.

4.3.3. Robot and Human Follower Trajectory Comparison

The objective of the simulation experiment was to determine whether the robot arm can safely reproduce the motion of the human follower in the recorded dataset. The same held-out episode used during the offline evaluation of the diffusion policy, `Manipulation3`, was employed.

The comparison between the trajectory executed by the simulated robot and the ground-truth human follower is shown in Figure 4.7. In this figure, the simulated robot follows

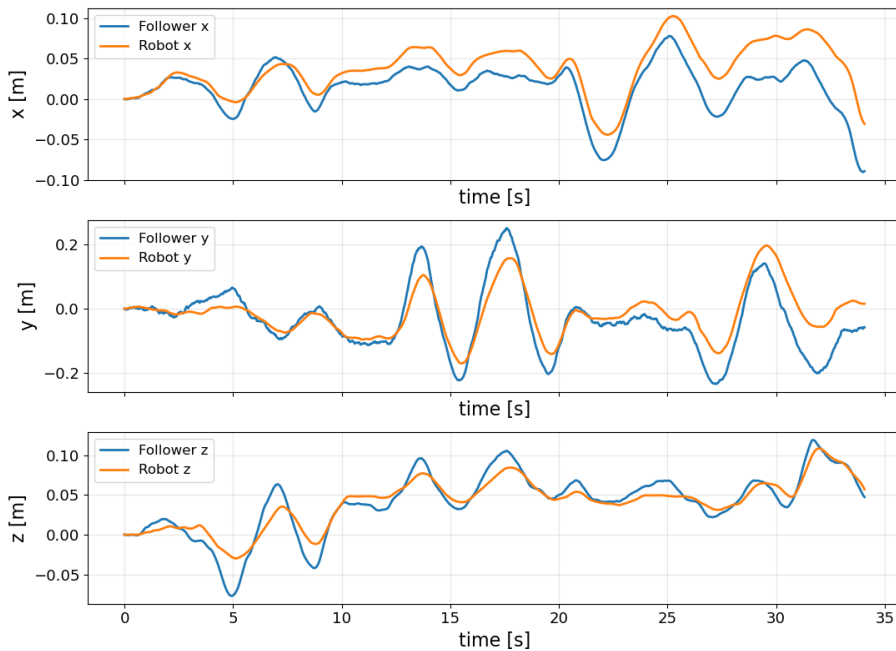


Figure 4.7: Comparison between the trajectory executed by the simulated robot and the human follower trajectory recorded in the dataset.

the same overall displacement trend as the human follower trajectory, although the two paths do not perfectly overlap.

The coordinate-wise plots show that the closed-loop robot motion is smoother than the

recorded human trajectory and exhibits a small temporal shift. This is expected because the simulated trajectory is not obtained by directly replaying the ground-truth follower motion, but by integrating the diffusion-policy predictions through the impedance-admittance controller and the simulated object dynamics.

The comparative analysis between the simulated robot and the human follower was based on three complementary indicators: trajectory accuracy, response delay and directional intent agreement.

The mean absolute trajectory error (MAE) represents the average Euclidean distance between the trajectory executed by the robot and the human follower trajectory. A small value indicates that the robot remains spatially close to the human demonstration throughout the sequence.

The response delay was estimated from the normalized cross-correlation between the ground-truth and robot displacement sequences. The estimated delay was then obtained as

$$\hat{\ell} = \arg \max_{\ell} \rho_{\Delta p}(\ell), \quad \hat{t}_{\text{delay}} = \frac{\hat{\ell}}{f_s}, \quad (4.6)$$

where $\rho_{\Delta p}(\ell)$ denotes the normalized cross-correlation at lag ℓ and $f_s = 30$ Hz is the trajectory sampling frequency. This indicator measures the temporal delay between the demonstrated follower motion and the corresponding robot response.

Finally, the correctness of the inferred motion intent, C_{intent} , was evaluated through the directional intent agreement:

$$C_{\text{intent}} = \frac{(\mathbf{p}_F^{\text{robot}})^\top \mathbf{p}_F^{\text{GT}}}{\|\mathbf{p}_F^{\text{robot}}\|_2 \|\mathbf{p}_F^{\text{GT}}\|_2 + \varepsilon}, \quad (4.7)$$

where $\mathbf{p}_F^{\text{robot}}$ is the Cartesian displacement vector executed by the robot over the prediction horizon, and $\mathbf{p}_F^{\text{GT}}$ is the corresponding ground-truth displacement of the human follower obtained from the offline demonstration. The metric is computed as the normalized dot product between these two vectors and therefore measures their directional agreement. The term ε is a small positive constant introduced for numerical stability.

The index lies in the interval $[-1, 1]$. A value close to 1 indicates that the robot moves in the same overall direction as the human follower and therefore correctly reproduces the demonstrated motion intent. Values close to 0 indicate an orthogonal response, while

negative values indicate motion in the opposite direction.

The trajectory MAE quantifies the spatial similarity between the robot and human follower motions, while the estimated delay characterizes the temporal responsiveness of the complete system. The directional intent agreement evaluates whether the robot interprets and reproduces the overall direction of the demonstrated follower motion. Taken together, these indicators describe whether the robot follows a trajectory that is accurate, timely and consistent with the intended collaborative movement. The numerical results of the three comparative metrics are reported in Table 4.5.

Table 4.5: Trajectory reproduction and intent-estimation results in simulation.

Indicator	Result
Trajectory MAE [mm]	54.8
Estimated response delay [s]	0.13
Directional intent agreement C_{intent}	0.75

The resulting trajectory MAE of 54.8 mm indicates that the robot does not exactly reproduce the demonstrated follower path, but remains within a moderate spatial deviation over the complete closed-loop execution. This error is considerably larger than the offline single-window prediction error because the simulated trajectory includes cumulative integration, controller dynamics, interaction with the virtual object and the effect of using the robot’s own executed motion as part of the conditioning input.

The estimated response delay was 0.13 s, corresponding to approximately four samples at the 30 Hz high-level update rate. This value indicates a limited delay between the demonstrated follower motion and the robot response. This delay is expected in a causal vision-based pipeline, since the command is generated only after image acquisition, key-point extraction, filtering, diffusion inference and communication with the low-level controller. It is acceptable in this case because the task involves smooth and relatively slow co-manipulation motions. Moreover, the impedance–admittance controller provides local compliance, allowing small timing mismatches to be absorbed without abrupt robot responses. Finally, the directional intent agreement $C_{\text{intent}} = 0.75$ shows that, despite the spatial tracking error, the robot moves in a direction largely consistent with the demonstrated follower motion.

Overall, the simulation results confirm that the diffusion-based pipeline can be integrated with the impedance–admittance controller to generate a coherent closed-loop follower behavior. The robot does not exactly reproduce the ground-truth human follower trajectory,

as expected from the accumulation of prediction, control and interaction effects, but it follows the same general motion direction with a limited response delay. The simulation therefore provides an intermediate validation step: it shows that the predictions obtained offline can be transformed into feasible robot motion before the system is evaluated in real human–robot collaboration experiments.

4.4. Real-Robot Evaluation

4.4.1. Experimental Setup

The proposed system was evaluated online using a real Franka robot and an Intel RealSense D415 RGB-D camera. The high-level perception and diffusion pipeline was executed on a laptop equipped with an NVIDIA GeForce RTX 5080 GPU, which was used to accelerate both YOLO-based pose estimation and diffusion-model inference. The human motion was processed at 30 Hz, consistently with the camera frame rate, while the low-level Cartesian impedance–admittance controller operated at 1000 Hz to ensure stable and compliant robot execution.

Across the real-robot experiments, multiple objects with different geometry, mass and grasping characteristics were used to test the pipeline under different manipulation conditions.

4.4.2. Full-Pipeline Execution

After the simulation validation, the complete pipeline was tested during a real collaborative manipulation task performed mainly along the y direction, moving from the right to the left side of the robot. This representative execution was selected to inspect the behavior of the online system under physical human–robot interaction. The objective of this analysis is not to compare all experimental tasks, but to verify that the diffusion policy provides a coherent predictive motion contribution and that the generated reference can be tracked by the low-level impedance–admittance controller.

The selected manipulation is useful for this purpose because the motion is simple enough to interpret clearly, while still requiring the human and the robot to coordinate through the shared object. Therefore, it provides a compact real-robot validation of the complete perception–prediction–control pipeline.

The commanded motion was first decomposed into the contribution generated by the diffusion policy and the correction introduced by the admittance loop. Let $\Delta \mathbf{x}_k^{\text{diff}}$ be the

incremental Cartesian command published by the diffusion policy at high-level update k . Its cumulative contribution over the analysed interval is

$$\mathbf{c}_k^{\text{diff}} = \sum_{i=1}^k \Delta \mathbf{x}_i^{\text{diff}}. \quad (4.8)$$

The admittance contribution is obtained from the logged Cartesian admittance offset, expressed relative to the beginning of the analysed segment:

$$\mathbf{c}_k^{\text{adm}} = \sum_{i=1}^k \Delta \mathbf{x}_i^{\text{adm}} \quad (4.9)$$

This formulation is used because the admittance signal is inspected at the reference-pose level, where it represents the correction added to the nominal desired pose generated by the high-level pipeline. Plotting $\|\mathbf{c}_k^{\text{diff}}\|_2$ and $\|\mathbf{c}_k^{\text{adm}}\|_2$ over the normalized task progress shows whether the robot motion is mainly anticipated by the diffusion policy or generated reactively through physical interaction. The resulting decomposition is reported in Figure 4.8.

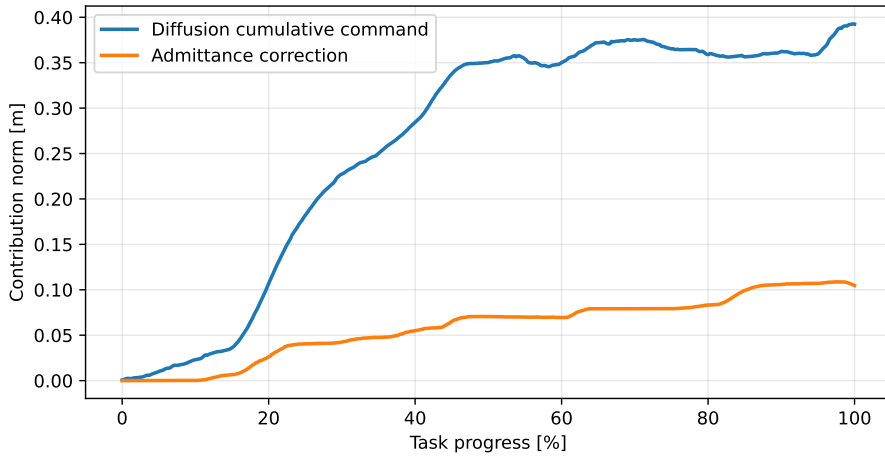


Figure 4.8: Cumulative contribution of the diffusion policy and admittance correction during the representative real-robot manipulation.

Figure 4.8 shows how the two motion components evolve during the execution. The diffusion contribution represents the main part of the commanded motion, while the admittance loop provides a smaller corrective contribution. This indicates that the robot motion is not generated only as a reactive response to interaction forces, but is largely anticipated by the vision-based diffusion policy.

The relation between the observed human motion and the executed robot motion was then evaluated by comparing the measured end-effector displacement with the mean leader-wrist displacement. The leader signal was computed from the right and left wrists as

$$\Delta \bar{\mathbf{x}}_k^L = \frac{1}{2} \left(\Delta \mathbf{x}_k^{L,r} + \Delta \mathbf{x}_k^{L,l} \right), \quad (4.10)$$

and then integrated over time to obtain the leader displacement used in the plots. This comparison is useful because the diffusion policy is expected to map the observed leader motion into a compatible follower response, rather than simply reproducing the same trajectory. The comparison is shown in Figure 4.9.

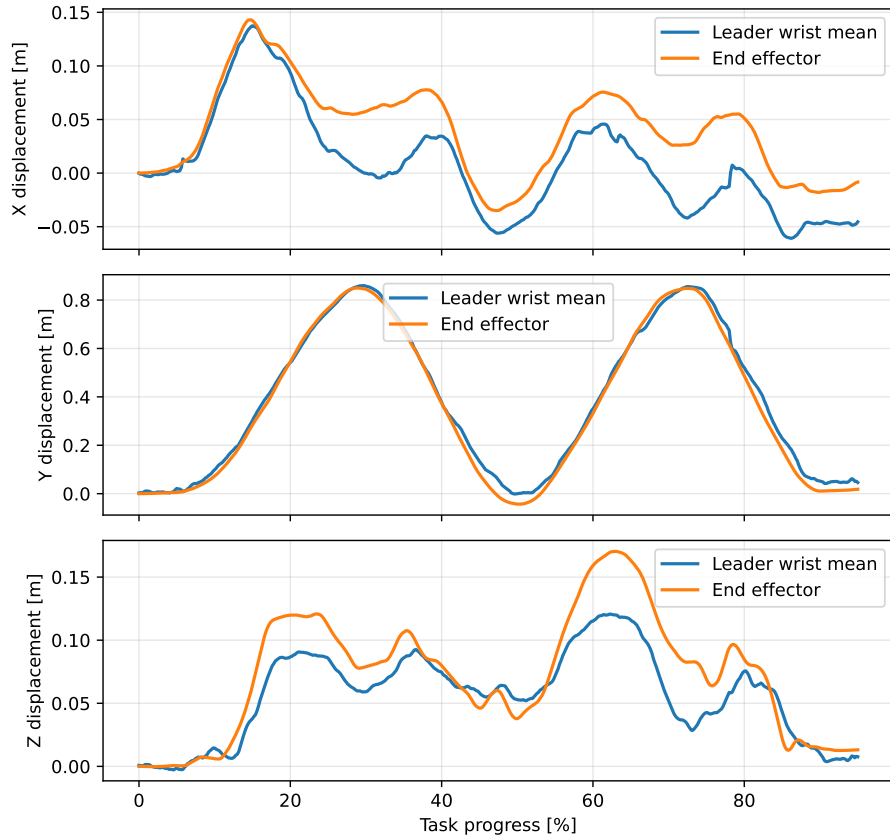


Figure 4.9: Comparison between the mean leader-wrist displacement and the measured end-effector displacement during the representative real-robot manipulation along the y direction.

Figure 4.9 illustrates the relation between the leader motion observed by the perception system and the end-effector motion executed by the robot. The two trajectories are not expected to overlap exactly, because the robot acts as the follower and the diffusion

policy maps the leader’s motion into a compatible cooperative response. Nevertheless, the coherent trend between the two signals indicates that the robot motion is consistent with the observed human intention.

Finally, the low-level tracking behavior was inspected by comparing the impedance reference pose with the measured end-effector pose. The tracking error is defined as

$$\mathbf{e}_k^{\text{track}} = \mathbf{x}_k^{\text{ref}} - \mathbf{x}_k^{\text{ee}}. \quad (4.11)$$

The corresponding plots report the Cartesian components of $\mathbf{x}_k^{\text{ref}}$ and $\mathbf{x}_k^{\text{ee}}$ over normalized task progress. A small tracking gap indicates that the reference generated by the diffusion policy and corrected by the admittance loop remains feasible for the impedance controller. The tracking behavior is reported in Figure 4.10.

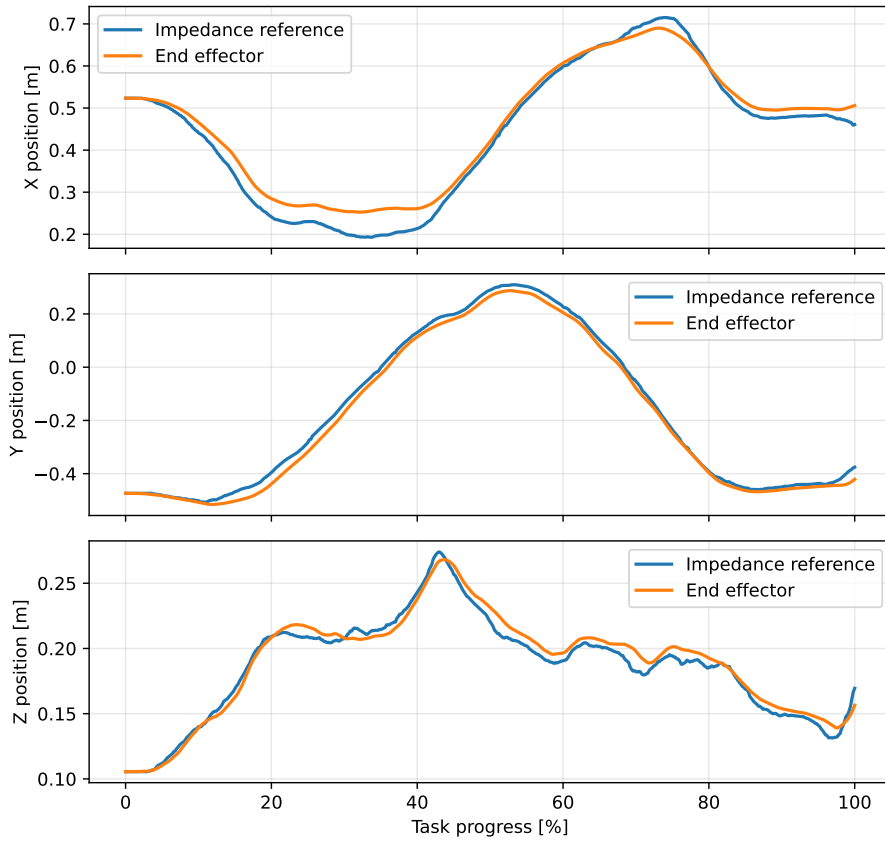


Figure 4.10: Tracking of the impedance reference by the measured end-effector pose during the representative real-robot manipulation.

Figure 4.10 shows that the measured end-effector trajectory follows the impedance reference generated by the complete pipeline. This confirms that the high-level commands

produced by the diffusion policy and corrected by the admittance loop are compatible with the low-level controller and can be executed during real physical collaboration.

The qualitative observations provided by the previous plots are summarized by three scalar indicators. First, the relative contribution of the diffusion policy with respect to the admittance correction was measured through the median contribution ratio:

$$R_{\text{diff}/\text{adm}} = \text{median}_k \frac{\|\mathbf{c}_k^{\text{diff}}\|_2}{\|\mathbf{c}_k^{\text{adm}}\|_2}. \quad (4.12)$$

This indicator quantifies whether the commanded motion is mainly generated by the predictive diffusion policy or by the reactive admittance correction. Values greater than one indicate that the diffusion contribution is dominant over the analysed interval.

The tracking performance of the low-level controller was quantified using the mean and peak Cartesian tracking errors:

$$e^{\text{avg}} = \frac{1}{N} \sum_{k=1}^N \|\mathbf{e}_k^{\text{track}}\|_2, \quad (4.13)$$

$$e^{\text{max}} = \max_k \|\mathbf{e}_k^{\text{track}}\|_2. \quad (4.14)$$

The resulting values are reported in Table 4.6.

Table 4.6: Summary of the representative real-robot manipulation experiment.

Indicator	Result
Median contribution ratio	2.82
Mean tracking error	27.576 mm
Peak tracking error	50.588 mm

Overall, the representative real-robot execution shows that the complete pipeline can operate online during physical human–robot collaboration. The diffusion policy provides the main predictive contribution to the commanded motion, while the admittance loop compensates for residual interaction and modelling errors. The measured end-effector trajectory remains close to the impedance reference, confirming that the generated commands are feasible for the low-level controller. This result motivates the broader task-level comparison presented in the following ablation study.

4.5. Comparative study

A comparative real-robot study was conducted to evaluate the contribution of the diffusion policy during physical human–robot collaboration. Two experimental conditions were considered:

1. **Diffusion-assisted impedance–admittance control (Diffusion+Low-level):** the complete pipeline was enabled. Human motion was observed by the vision system and used by the diffusion policy to generate incremental motion commands for the robotic follower. Physical interaction was simultaneously accommodated by the impedance–admittance controller.
2. **Impedance–admittance baseline (Low-level-only):** the diffusion commands were disabled. The robot motion was therefore generated only by the physical interaction forces processed by the impedance–admittance controller.

The same low-level controller gains, safety limits, objects and initial robot configurations were maintained in both conditions. Therefore, the principal experimental variable was the presence or absence of the predictive assistance provided by the diffusion policy.

4.5.1. Experimental Task Definition

Five tasks were designed to evaluate different aspects of collaborative manipulation, as summarized in Table 4.7.

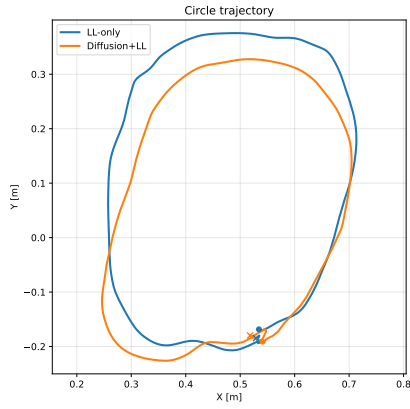
Table 4.7: Tasks performed during the real-robot comparative study.

ID	Task	Description
T1	Circular trajectory	The participant guided the shared object along an approximately circular trajectory in the x - y plane.
T2	Square trajectory	The participant attempted to reproduce a square trajectory in the y - z plane.
T3	Obstacle avoidance	The object was moved around an obstacle positioned on the table.
T4	Object placement	The participant precisely placed the manipulated object inside a target container.
T5	Linear trajectory	The shared object was moved along a straight trajectory parallel to the y axis and then back to the starting position.

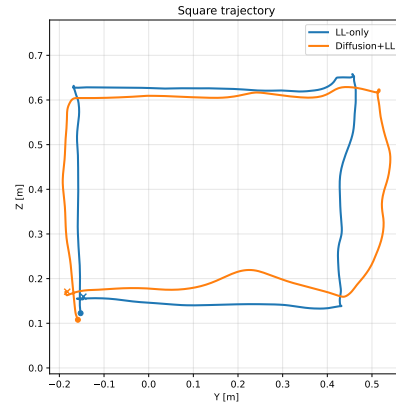
The circular and square trajectories evaluate the ability of the system to follow structured motions. The obstacle-avoidance task assesses adaptability and coordination in a constrained workspace. The placement task focuses on fine positioning accuracy, while the linear trajectory provides a simpler reference motion against which deviations and interaction effort can be measured.

Before analysing the interaction forces, the trajectories executed during the five tasks were inspected to provide a qualitative overview of the movements performed under the two control conditions. For each task, the participant was asked to reproduce the same macroscopic task objective in both conditions, namely with the diffusion-assisted pipeline and with the low-level-only baseline. Therefore, the goal was not to impose identical trajectories, but to ensure that both trials corresponded to the same task-level manipulation scenario.

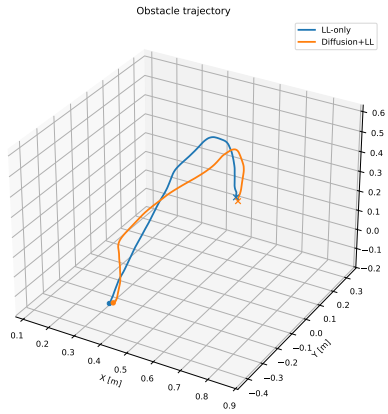
Figure 4.11 reports the measured end-effector trajectories for the five tasks under the two experimental conditions. The plots are used to qualitatively verify that the two executions belong to the same class of movement and that the task was successfully completed in both cases. However, the subsequent comparison is based on the interaction forces generated during the motion, rather than on trajectory tracking accuracy or execution time. For this reason, a precise pointwise alignment between the two trajectories is not required; what is required is that the same manipulation task is performed successfully at a macroscopic level in both control conditions.



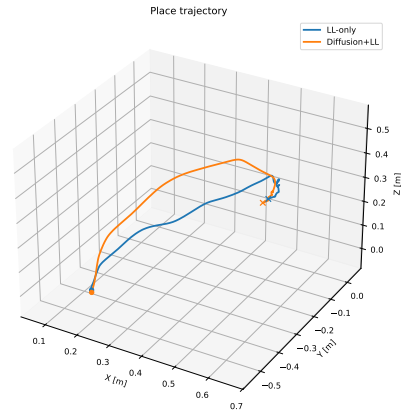
(a) T1: circular trajectory in the x - y plane.



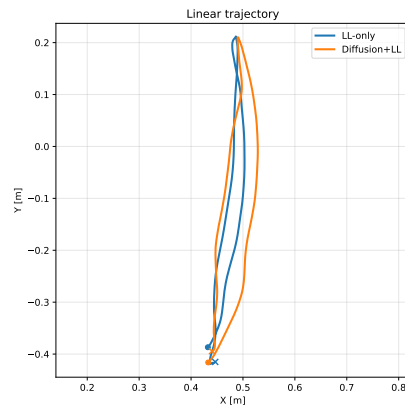
(b) T2: square trajectory in the y - z plane.



(c) T3: obstacle avoidance.



(d) T4: object placement.



(e) T5: linear trajectory along y axis.

Figure 4.11: Measured end-effector trajectories for the five experimental tasks under the admittance-only baseline and the diffusion-assisted pipeline.

4.5.2. Force-Based Comparative Metrics

After verifying the task-level trajectories, the five experiments were compared against the admittance-only baseline in terms of interaction force. In this comparison, the relevant question is whether the predictive assistance provided by the diffusion policy reduces the physical effort required from the human operator.

The force signal is the norm of the filtered contact wrench force,

$$F_k = \|\mathbf{w}_k\|_2. \quad (4.15)$$

For each task and control condition, the comparison is summarized using the RMS force and the peak force:

$$F_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{k=1}^N F_k^2}, \quad F_{\text{peak}} = \max_k F_k. \quad (4.16)$$

The relative reduction introduced by the diffusion-assisted pipeline is computed as

$$R_F = 100 \frac{F_{\text{LL}} - F_{\text{Diffusion+LL}}}{F_{\text{LL}}}, \quad (4.17)$$

where positive values indicate that the complete pipeline required lower interaction force than the admittance-only controller. The comparison is reported task by task using force profiles over normalized task progress and aggregate force summaries across the five experiments. An example of the normalized force profile comparison is shown in Figure 4.12.

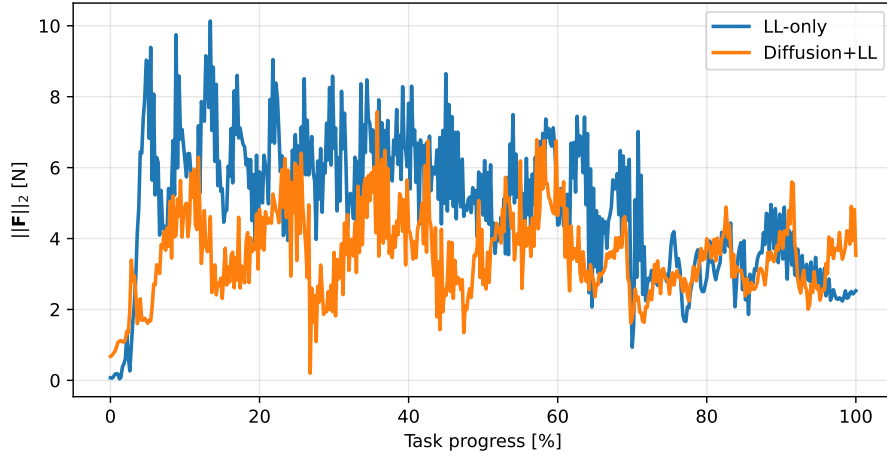


Figure 4.12: Example of the normalized interaction-force comparison between the admittance-only (LL-only) baseline and the diffusion-assisted pipeline (Diffusion+LL) in task T3.

Figure 4.12 compares the evolution of the interaction force during the same task under the two control conditions. A lower force profile for the diffusion-assisted pipeline indicates that part of the required motion is anticipated by the high-level policy, reducing the need for the human operator to generate motion through physical interaction alone.

The aggregate comparison across the five experiments is reported in Figure 4.13 and Figure 4.14.

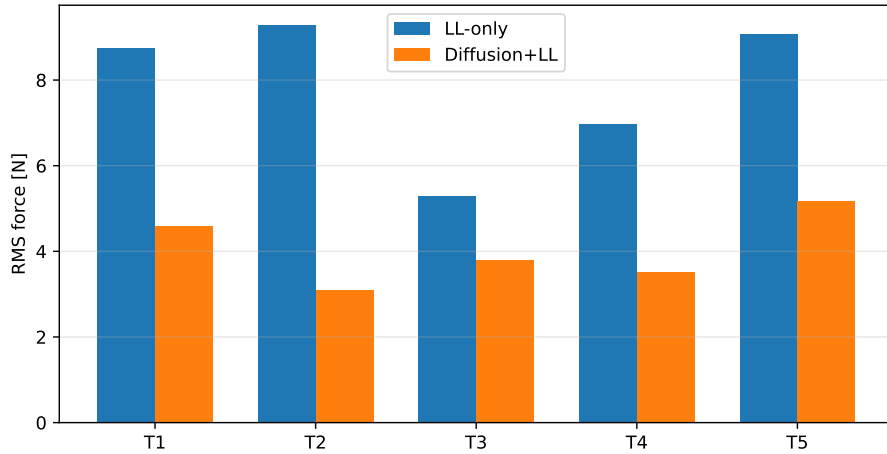


Figure 4.13: RMS interaction force for the five experiments under the admittance-only and diffusion-assisted conditions.

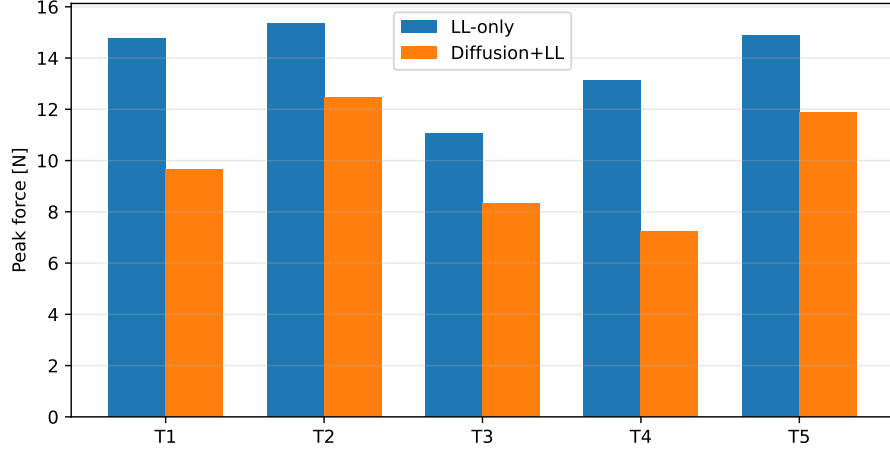


Figure 4.14: Peak interaction force for the five experiments under the admittance-only and diffusion-assisted conditions.

Figure 4.13 summarizes the average interaction effort required during each task, while Figure 4.14 highlights the maximum forces reached during execution. The RMS force is useful to assess the overall physical effort sustained throughout the task, whereas the peak force captures short but potentially demanding interaction events. Together, these two indicators provide a more complete comparison between the diffusion-assisted architecture and the low-level-only baseline. This force-centred comparison isolates the effect that is most relevant for heavy cooperative manipulation: when the diffusion policy anticipates the leader motion correctly, lower interaction forces indicate that the human needs less physical guidance to move the shared object.

To summarize the force-based comparison with a single scalar indicator, a global relative reduction was computed from the RMS force values across the five experiments. After replacing the square-task run with the corresponding new square experiment, the diffusion-assisted pipeline achieved a global RMS force reduction of $R_{F,\text{global}} = 37.6\%$ with respect to the admittance-only baseline. This value indicates that, over the full set of tasks, the proposed architecture substantially reduced the interaction-force level required to complete the cooperative manipulation experiments.

Overall, the ablation study provides the most direct evaluation of the contribution introduced by the diffusion policy. Since the low-level controller, safety limits, objects and initial configurations were kept unchanged, the differences in measured force can be attributed primarily to the presence of the predictive high-level assistance. When the diffusion-assisted pipeline produces lower RMS and peak forces than the low-level-only baseline, the robot contributes more actively to the cooperative motion and the human

operator is required to provide less physical guidance.

These results support the main objective of the proposed architecture: using vision-based motion prediction not only to generate feasible robot trajectories, but also to reduce the reactive interaction effort required during collaborative manipulation. At the same time, the comparison remains task dependent, since the benefit of the diffusion policy is expected to be larger when the observed leader motion is consistent with the motion patterns learned from the demonstrations.

5 | Conclusion

5.1. Conclusions

This thesis presented a vision-guided framework for human–robot co-manipulation based on the integration of RGB-D perception, diffusion-based motion prediction and compliant impedance–admittance control. The work focused on collaborative carrying tasks organized according to a leader–follower structure, where the motion coordination observed in human–human demonstrations was used as the starting point for generating adaptive follower behavior in a human–robot collaborative scenario. In this formulation, the human operator guides the motion and the robot acts as an adaptive follower.

The proposed approach addresses one of the central challenges of physical Human–Robot Collaboration: enabling the robot to generate motion references that are consistent with the observed human behavior while preserving compliant interaction during physical execution. Instead of relying only on reactive force-based behavior or predefined trajectories, the system uses human body motion as a predictive cue. Three-dimensional human key-points are extracted from RGB-D data and used, together with the recent follower motion, as conditioning information for a Transformer-based diffusion policy. The model predicts future follower Cartesian increments, which are then transformed into reference commands for the low-level controller.

A key aspect of the thesis is the transition from human–human collaboration to human–robot collaboration. During dataset construction, the leader and follower roles were extracted from human–human carrying demonstrations, allowing the model to learn motion relationships from natural collaborative behavior. During simulation and real-robot execution, however, the follower role was assigned to the robot. Therefore, the learned policy was not used simply to reproduce recorded human motion offline, but to provide predictive motion references that could be executed by a robotic manipulator interacting physically with a human operator.

A key element of the proposed formulation is the use of Cartesian deltas rather than absolute positions. This representation shifts the learning problem from the reproduction

of scene-dependent coordinates to the prediction of local motion evolution. As a result, the model focuses on the dynamic relation between the leader and the follower, which is more relevant for collaborative manipulation than the absolute location of the subjects in the workspace. In addition, the mask-aware formulation allows the system to handle missing or unreliable keypoints without discarding complete temporal windows. This is particularly important in RGB-D perception, where occlusions, invalid depth values and temporary keypoint losses are common.

The offline evaluation showed that the diffusion policy can predict follower wrist-motion increments with millimetre-scale accuracy on held-out human–human sequences. On the pooled test set, the model achieved an MAE of 1.416 mm, an RMSE of 2.253 mm, and a coefficient of determination equal to $R^2 = 0.8817$. These results indicate that the model was able to capture a substantial part of the variation in the demonstrated follower motion, despite being evaluated on sequences not used during training.

The simulation experiments demonstrated that the predicted increments can be integrated into a complete robotic architecture. When the diffusion policy was connected to the impedance–admittance controller in MuJoCo, the robotic follower reproduced the general trend of the demonstrated human–follower motion. The resulting trajectory error was larger than the offline prediction error, as expected, because the complete system includes control dynamics, frame transformations, saturation limits, timing effects and physical constraints. Nevertheless, the simulation confirmed that motion patterns learned from human–human demonstrations can be transferred to a robotic follower and used as high-level motion references when executed through a compliant control layer.

The real-robot evaluation further validated the feasibility of the complete online pipeline. During the representative manipulation, the diffusion policy generated the main predictive contribution to the commanded motion, while the admittance loop provided a smaller corrective contribution associated with residual interaction and modelling errors. The comparison between the impedance reference and the measured end-effector trajectory showed that the generated references were compatible with the low-level controller and could be tracked during physical human–robot collaboration.

Finally, the ablation study compared the complete diffusion-assisted architecture against a low-level-only impedance–admittance baseline. Since the same controller gains, safety limits, objects and initial configurations were maintained in both conditions, the comparison isolated the contribution of the predictive high-level policy. The force-based analysis showed that the diffusion-assisted pipeline can reduce the interaction effort required from the human operator when the observed leader motion is consistent with the mo-

tion patterns learned from demonstrations. This result supports the main hypothesis of the thesis: vision-based diffusion prediction can contribute to collaborative manipulation not only by generating feasible robot trajectories, but also by transferring coordination patterns learned from human–human interaction to human–robot execution, reducing the dependence on purely reactive physical guidance.

Overall, the thesis demonstrates that diffusion policies can be effectively connected with RGB-D human perception and compliant robot control for collaborative manipulation. The main value of the proposed architecture lies in its modularity: perception, prediction and physical execution are separated but integrated within a common ROS 2 framework. This modular structure makes the system interpretable, extendable and suitable for future improvements in both learning and control.

5.2. Main Contributions

The work developed in this thesis provides several contributions to the design of adaptive robotic systems for physical collaboration.

First, it proposes a complete perception–prediction–control pipeline for human–robot co-manipulation. The system starts from RGB-D recordings or online camera streams, extracts three-dimensional human keypoints, predicts future follower motion and executes the resulting references on a robotic manipulator through compliant control.

Second, the thesis introduces a dataset-construction strategy specifically designed to transfer collaborative carrying demonstrations from the human–human domain to the human–robot domain. Human–human interactions are converted into leader–follower temporal sequences, represented through Cartesian increments and organized into observation and action windows. This allows the model to learn from natural human coordination while preparing the learned follower behavior for execution by a robot. The use of validity masks allows the learning process to account for missing measurements explicitly, instead of treating them as valid numerical values or removing entire samples.

Third, the work adapts a Transformer-based diffusion policy to the prediction of follower Cartesian increments in a collaborative manipulation setting. The model is conditioned on the recent evolution of both the leader and the follower, allowing the prediction to depend on the interaction context rather than on the leader motion alone. The denoising formulation generates short action sequences that can be used in a receding-horizon manner during online execution.

Fourth, the thesis integrates the learned high-level policy with an impedance–admittance

control architecture. This integration is essential because collaborative manipulation is not only a prediction problem: the generated motion must be executed during physical contact with a human and a shared object. In this step, the follower role learned from human–human data is assigned to the robot, requiring the predicted motion to be compatible with robot dynamics, workspace limits and compliant interaction. The low-level controller provides compliance, reference tracking, nullspace regulation and motion bounding, while the diffusion policy provides anticipatory motion references.

Finally, the experimental evaluation considers the system at multiple levels. The diffusion model is evaluated offline on unseen human–human sequences, the complete architecture is validated in simulation with a robotic follower, and the online pipeline is tested on a real robot. The ablation study further compares predictive assistance with a low-level-only baseline, providing a direct assessment of the role played by the diffusion policy during physical human–robot collaboration.

5.3. Limitations

Despite the positive results, several limitations remain.

The first limitation concerns the size and diversity of the demonstration dataset. The diffusion policy was trained on a limited set of collaborative demonstrations. Although the use of Cartesian deltas and geometric augmentation improves robustness, the learned behavior remains dependent on the motion patterns present in the training data. Generalization to different users, object shapes, object masses, grasping configurations and task geometries may therefore be limited.

A second limitation is related to the perception pipeline. The system relies on a single RGB-D camera and on sparse human keypoints extracted by a pose-estimation model. This representation is compact and efficient, but it is sensitive to occlusions, partial body visibility, invalid depth measurements and camera placement. In addition, the depth component is intrinsically noisier than image-plane measurements, which can affect the reconstructed three-dimensional trajectories and the prediction accuracy, especially along the depth-related direction.

A third limitation concerns frame alignment and calibration. The online system uses camera-to-robot alignment to express the reconstructed human motion in the robot frame. Any residual calibration error directly affects the consistency between the perceived leader motion and the robot command frame. Although the adopted alignment strategy is sufficient for the experiments presented in this thesis, more accurate and robust calibration

would be required for deployment in less controlled environments.

Another limitation is that the diffusion policy predicts only Cartesian translational increments of the follower motion. Orientation commands, object pose, grasp configuration and task constraints are not explicitly predicted by the high-level model. As a result, the low-level controller must maintain or regulate these aspects independently. This is acceptable for the considered experimental setup, but it restricts the range of collaborative tasks that can be addressed.

Finally, the compliant control parameters are fixed during execution. The impedance and admittance layers provide safe and stable interaction behavior, but their parameters are not adapted online according to the predicted motion, interaction force or task phase. Consequently, the system cannot yet exploit the diffusion policy to modulate compliance proactively.

5.4. Future Developments

Several future developments can extend the work presented in this thesis.

A first direction is the expansion of the demonstration dataset. Collecting additional human–human demonstrations would improve the variability of the training data and support better generalization. Future datasets should include multiple participants, different object geometries, different masses, various motion directions and more complex collaborative tasks. This would allow the diffusion policy to learn a broader set of coordination strategies.

A second improvement concerns the perception system. Multi-camera RGB-D perception could reduce occlusions and improve the reliability of three-dimensional keypoint reconstruction. Alternatively, more advanced human representation methods, such as full-body 3D pose estimation or human mesh reconstruction, could provide richer information about posture and body motion. These approaches would need to be balanced against the real-time requirements of the online pipeline.

A third future direction is the inclusion of additional sensory modalities. The current policy is mainly conditioned on visual motion and recent follower displacement. Adding force/torque measurements, object pose, object velocity or contact-state information could make the prediction more aware of the physical interaction. This would be particularly relevant for heavy or asymmetric objects, where the human intention is expressed not only through motion but also through interaction forces.

Another important extension is the prediction of full 6D robot motion. Future versions of the policy could generate both translational and rotational motion references, allowing the robot to coordinate not only the position of the end effector but also the orientation of the shared object. This would make the framework applicable to a wider set of collaborative manipulation tasks, including object placement, rotation, insertion and constrained transportation.

The integration between diffusion prediction and compliant control could also be further developed. A promising direction is variable impedance or variable admittance control, where the high-level policy does not only predict motion increments but also provides information used to adapt stiffness, damping or admittance parameters. In this way, the robot could become more compliant during uncertain or contact-rich phases and more precise during goal-directed positioning phases.

From the learning perspective, future work could investigate online adaptation and personalization. Human collaboration styles vary significantly across users, and a policy trained offline may not be optimal for every partner. Lightweight fine-tuning, residual adaptation or user-conditioned policies could allow the robot to progressively adapt to a specific operator while preserving the safety guarantees of the low-level controller.

Further work should also address more rigorous experimental validation. A larger real-robot user study with repeated trials, multiple participants and statistical analysis would provide a stronger evaluation of the benefits introduced by the diffusion-assisted architecture. In addition to force metrics, future studies could include task completion time, smoothness, subjective workload, perceived robot helpfulness and user comfort.

In conclusion, this thesis provides a first complete implementation of a diffusion-assisted, vision-guided and compliant framework for human–robot co-manipulation. The results show that predictive motion generation can be successfully integrated with physical robot control and can contribute to reducing reactive interaction effort. At the same time, the identified limitations indicate that broader datasets, richer sensing, adaptive compliance and more extensive real-world validation are necessary steps toward more general, robust and natural collaborative robotic systems.

Bibliography

- [1] F. J. Abu-Dakka and M. Saveriano. Variable impedance control and learning—a review. *Frontiers in Robotics and AI*, Volume 7 - 2020, 2020. ISSN 2296-9144. doi: 10.3389/frobt.2020.590681. URL <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2020.590681>.
- [2] A. Ajoudani, A. M. Zanchettin, S. Ivaldi, A. Albu-Schäffer, K. Kosuge, and O. Khatib. Progress and prospects of the human-robot collaboration. *Autonomous Robots*, 42, 06 2018. doi: 10.1007/s10514-017-9677-2.
- [3] B. D. Argall, S. Chernova, M. Veloso, and B. Browning. A survey of robot learning from demonstration. *Robotics and Autonomous Systems*, 57(5):469–483, 2009. ISSN 0921-8890. doi: <https://doi.org/10.1016/j.robot.2008.10.024>.
- [4] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh. Openpose: Realtime multi-person 2d pose estimation using part affinity fields, 2019. URL <https://arxiv.org/abs/1812.08008>.
- [5] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion, 2024. URL <https://arxiv.org/abs/2303.04137>.
- [6] J. Craig and M. Raibert. A systematic method of hybrid position/force control of a manipulator. In *COMPSAC 79. Proceedings. Computer Software and The IEEE Computer Society's Third International Applications Conference, 1979.*, pages 446–451, 1979. doi: 10.1109/CMPSAC.1979.762539.
- [7] A. De Santis, B. Siciliano, A. De Luca, and A. Bicchi. An atlas of physical human-robot interaction. *Mechanism and Machine Theory*, 43:253–270, 03 2008. doi: 10.1016/j.mechmachtheory.2007.03.003.
- [8] European Commission, Directorate-General for Research and Innovation, M. Breque, L. De Nul, and A. Petridis. *Industry 5.0 – Towards a sustainable, human-centric and resilient European industry*. Publications Office of the European Union, 2021. doi: doi/10.2777/308407.

- [9] F. Ferraguti, C. Secchi, and C. Fantuzzi. A tank-based approach to impedance control with variable stiffness. In *2013 IEEE International Conference on Robotics and Automation*, pages 4948–4953, 2013. doi: 10.1109/ICRA.2013.6631284.
- [10] F. Ficuciello, L. Villani, and B. Siciliano. Variable impedance control of redundant manipulators for intuitive human–robot physical interaction. *IEEE Transactions on Robotics*, 31(4):850–863, 2015. doi: 10.1109/TRO.2015.2430053.
- [11] B. Hannaford and J.-H. Ryu. Time-domain passivity control of haptic interfaces. *IEEE Transactions on Robotics and Automation*, 18(1):1–10, 2002. doi: 10.1109/70.988969.
- [12] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models, 2020. URL <https://arxiv.org/abs/2006.11239>.
- [13] G. Hoffman, T. Bhattacharjee, and S. Nikolaidis. Inferring human intent and predicting human action in human–robot collaboration. *Annual Review of Control, Robotics, and Autonomous Systems*, 7, 11 2023. doi: 10.1146/annurev-control-071223-105834.
- [14] N. Hogan. Impedance control: An approach to manipulation. In *1984 American Control Conference*, pages 304–313, 1984. doi: 10.23919/ACC.1984.4788393.
- [15] Intel. Intel realsense depth camera d415 specifications. <https://www.intel.com/content/www/us/en/products/sku/128256/intel-realsense-depth-camera-d415/specifications.html>. Accessed: 2026-06-06.
- [16] Intel RealSense. Intel realsense sdk 2.0 documentation. <https://dev.realsenseai.com/docs/sdk-knowledge-base>. Accessed: 2026-06-06.
- [17] International Organization for Standardization. Robots and robotic devices – collaborative robots. Technical Specification ISO/TS 15066:2016, ISO, Geneva, CH, 2016.
- [18] International Organization for Standardization. Robots – safety requirements – part 1: Robots and part 2: Robot systems and integration. Standard ISO 10218:2025, ISO, Geneva, CH, 2025.
- [19] A. Kanazawa, M. J. Black, D. W. Jacobs, and J. Malik. End-to-end recovery of human shape and pose, 2018. URL <https://arxiv.org/abs/1712.06584>.
- [20] A. Keemink, H. Kooij, and A. Stienen. Admittance control for physical human–robot

- interaction. *The International Journal of Robotics Research*, 37:027836491876895, 04 2018. doi: 10.1177/0278364918768950.
- [21] M. Kekana, S. Du, N. Steyn, A. Benali, and H. Djerroud. A review of human intention recognition frameworks in industrial collaborative robotics. *Robotics*, 14 (12), 2025. ISSN 2218-6581. doi: 10.3390/robotics14120174. URL <https://www.mdpi.com/2218-6581/14/12/174>.
- [22] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3 (1):43–53, 1987. doi: 10.1109/JRA.1987.1087068.
- [23] R. Kropivšek Leskovar, J. Čamernik, and T. Petrič. Leader–follower role allocation for physical collaboration in human dyads. *Applied Sciences*, 11(19), 2021. ISSN 2076-3417. doi: 10.3390/app11198928. URL <https://www.mdpi.com/2076-3417/11/19/8928>.
- [24] Y. Lai, G. Paul, Y. Cui, and T. Matsubara. User intent estimation during robot learning using physical human robot interaction primitives. *Autonomous Robots*, 46, 02 2022. doi: 10.1007/s10514-021-10030-9.
- [25] C. T. Landi, F. Ferraguti, L. Sabattini, C. Secchi, and C. Fantuzzi. Admittance control parameter adaptation for physical human-robot interaction. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, page 2911–2916. IEEE, May 2017. doi: 10.1109/icra.2017.7989338. URL <http://dx.doi.org/10.1109/ICRA.2017.7989338>.
- [26] P. Lasota, T. Fong, and J. Shah. A survey of methods for safe human-robot interaction. *Foundations and Trends in Robotics*, 5:261–349, 05 2017. doi: 10.1561/23000000052.
- [27] M. Loper, N. Mahmood, J. Romero, G. Pons-Moll, and M. Black. Smpl: a skinned multi-person linear model. volume 34, 11 2015. doi: 10.1145/2816795.2818013.
- [28] M. Lorenzini, M. Lagomarsino, L. Fortini, S. Gholami, and A. Ajoudani. Ergonomic human-robot collaboration in industry: A review. *Frontiers in Robotics and AI*, Volume 9 - 2022, 2023. ISSN 2296-9144. doi: 10.3389/frobt.2022.813907. URL <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2022.813907>.
- [29] D. Maji, S. Nagori, M. Mathew, and D. Poddar. Yolo-pose: Enhancing yolo for multi

- person pose estimation using object keypoint similarity loss, 2022. URL <https://arxiv.org/abs/2204.06806>.
- [30] E. Matheson, R. Minto, E. Zampieri, M. Faccio, and G. Rosati. Human-robot collaboration in manufacturing applications: A review. *Robotics*, 8:100, 12 2019. doi: 10.3390/robotics8040100.
- [31] E. Mielke, E. Townsend, D. Wingate, J. L. Salmon, and M. D. Killpack. Human-robot planar co-manipulation of extended objects: data-driven models and control from human-human dyads. *Frontiers in Neurorobotics*, Volume 18 - 2024, 2024. ISSN 1662-5218. doi: 10.3389/fnbot.2024.1291694. URL <https://www.frontiersin.org/journals/neurorobotics/articles/10.3389/fnbot.2024.1291694>.
- [32] E. Ng, Z. Liu, and M. K. III. Diffusion co-policy for synergistic human-robot collaborative tasks, 2023. URL <https://arxiv.org/abs/2305.12171>.
- [33] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters. An algorithmic perspective on imitation learning. *Foundations and Trends® in Robotics*, 7(1-2):1–179, Mar. 2018. ISSN 1935-8261. doi: 10.1561/23000000053. URL <http://dx.doi.org/10.1561/23000000053>.
- [34] L. Roveda, J. Maskani, P. Franceschi, A. Abdi, F. Braghin, L. M. Tosatti, and N. Pedrocchi. Model-based reinforcement learning variable impedance control for human-robot collaboration. *Journal of Intelligent & Robotic Systems*, 100(2):417–433, 2020. doi: 10.1007/s10846-020-01183-3.
- [35] L. Roveda, A. Testa, A. A. Shahid, F. Braghin, and D. Piga. Q-learning-based model predictive variable impedance control for physical human-robot collaboration. *Artificial Intelligence*, 312:103771, 2022. doi: 10.1016/j.artint.2022.103771.
- [36] Z. Rysbek, K.-H. Oh, and M. Zefran. Recognizing intent in collaborative manipulation. In *INTERNATIONAL CONFERENCE ON MULTIMODAL INTERACTION*, ICMI '23, page 498–506. ACM, Oct. 2023. doi: 10.1145/3577190.3614174. URL <http://dx.doi.org/10.1145/3577190.3614174>.
- [37] A. Sampieri, G. D’Amely, A. Avogaro, F. Cunico, G. Skenderi, F. Setti, M. Cristani, and F. Galasso. Pose forecasting in industrial human-robot collaboration, 2022. URL <https://arxiv.org/abs/2208.07308>.
- [38] Y. S. Shao, T. Li, S. Keyvanian, P. Chaudhari, V. Kumar, and N. Figueroa. Constraint-aware intent estimation for dynamic human-robot object co-manipulation, 2024. URL <https://arxiv.org/abs/2409.00215>.

- [39] K. B. Shaw, D. L. Cordon, M. D. Killpack, and J. L. Salmon. A decomposition of interaction force for multi-agent co-manipulation, 2024. URL <https://arxiv.org/abs/2408.01543>.
- [40] B. Siciliano, L. Sciavicco, and V. Luigi. *Robotics: modelling, planning and control*. Springer London, 01 2009.
- [41] G. Solak, G. J. G. Lahr, I. Ozdamar, and A. Ajoudani. Context-aware collaborative pushing of heavy objects using skeleton-based intention prediction, 2025. URL <https://arxiv.org/abs/2505.10239>.
- [42] K. Sun, B. Xiao, D. Liu, and J. Wang. Deep high-resolution representation learning for human pose estimation, 2019. URL <https://arxiv.org/abs/1902.09212>.
- [43] E. C. Townsend, E. A. Mielke, D. Wingate, and M. D. Killpack. Estimating human intent for physical human-robot co-manipulation, 2017. URL <https://arxiv.org/abs/1705.10851>.
- [44] Ultralytics. Coco-pose dataset documentation. <https://docs.ultralytics.com/datasets/pose/coco/>, . Accessed: 2026-06-06.
- [45] Ultralytics. Ultralytics yolo pose estimation documentation. <https://docs.ultralytics.com/tasks/pose/>, . Accessed: 2026-06-06.
- [46] V. Villani, F. Pini, F. Leali, and C. Secchi. Survey on human–robot collaboration in industrial settings: Safety, intuitive interfaces and applications. *Mechatronics*, 55: 248–266, 2018. ISSN 0957-4158. doi: <https://doi.org/10.1016/j.mechatronics.2018.02.009>.
- [47] B. Wilhelm, B. Manfred, M. Braun, P. Rally, and O. Scholtz. Lightweight robots in manual assembly - best to start simply!, 2016. URL <https://publica.fraunhofer.de/handle/publica/298032>.
- [48] X. Yang, D. Kukreja, D. Pinkus, A. Sagar, T. Fan, J. Park, S. Shin, J. Cao, J. Liu, N. Ugrinovic, M. Feiszli, J. Malik, P. Dollar, and K. Kitani. Sam 3d body: Robust full-body human mesh recovery, 2026. URL <https://arxiv.org/abs/2602.15989>.
- [49] Z. Zhang, T. Li, and N. Figueroa. Constrained passive interaction control: Leveraging passivity and safety for robot manipulators. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13418–13424, 2024. doi: [10.1109/ICRA57147.2024.10610232](https://doi.org/10.1109/ICRA57147.2024.10610232).

A | Appendix A

A.1. Leader Keypoints Used in the Observations

The leader motion is represented using a selected subset of the COCO pose keypoints provided by the YOLO pose estimator. The selected leader keypoints are reported in Table A.1.

COCO index	Keypoint	Role in the representation
0	Nose	Upper-body orientation reference
5	Left shoulder	Torso and arm posture
6	Right shoulder	Torso and arm posture
7	Left elbow	Arm motion
8	Right elbow	Arm motion
9	Left wrist	Hand and carrying motion
10	Right wrist	Hand and carrying motion
11	Left hip	Pelvis and body direction
12	Right hip	Pelvis and body direction
13	Left knee	Lower-body motion
14	Right knee	Lower-body motion
15	Left ankle	Walking motion
16	Right ankle	Walking motion

Table A.1: Leader keypoints used in the observation vector. The left and right labels follow the COCO anatomical convention.

Since each keypoint is represented by its three Cartesian delta coordinates, the leader contributes $13 \times 3 = 39$ features to the observation vector. Together with the follower right-wrist delta, represented by 3 additional features, this gives the complete observation dimension:

$$39 + 3 = 42. \tag{A.1}$$

A.2. Training Configuration

Table A.2 reports the complete numerical configuration used for diffusion-policy training. The parameters are grouped according to their role in the pipeline.

Table A.2: Complete training configuration used for the conditional diffusion policy.

Parameter	Value	Description
Input and target representation		
Observation representation	Cartesian deltas	Recent leader–follower motion expressed as temporal increments.
Observation dimension	42	Follower right-wrist delta plus selected leader keypoint deltas.
Effective conditioning dimension	84	Observation values concatenated with the observation validity mask.
Action representation	Cartesian deltas	Future follower right-wrist motion expressed as increments.
Action dimension	3	Cartesian delta of the follower right wrist.
Temporal windowing		
Observation length	$T_{obs} = 30$	Number of past timesteps used for conditioning.
Prediction horizon	$H = 5$	Number of future timesteps predicted beyond the observation window.
Action–observation overlap	$O = 2$	Number of action tokens overlapping with the end of the observation window.
Action length	$T_a = 7$	Total decoder action length, given by $T_a = O + H$.
Total temporal span	35 timesteps	Unique timesteps covered by each sample, given by $T_{obs} + H$.
Window stride	$s_{win} = 10$	Temporal shift between consecutive extracted windows.
Window overlap	$O_{win} = 25$	Overlap between consecutive samples, computed as $35 - 10$.
Transformer architecture		
Architecture	Encoder–decoder Transformer	Encoder processes conditioning tokens; decoder denoises action tokens.

Parameter	Value	Description
Embedding dimension	$d_{emb} = 64$	Common latent dimension used for observation and action embeddings.
Encoder layers	4	Number of Transformer encoder blocks.
Decoder layers	4	Number of Transformer decoder blocks.
Attention heads	4	Number of attention heads in each multi-head attention layer.
Feed-forward dimension	$4d_{emb}$	Hidden dimension of the Transformer feed-forward blocks.
Embedding dropout	0	No dropout applied to the input embeddings.
Attention dropout	0.1	Dropout applied inside attention layers for regularization.
Activation function	GELU	Non-linear activation used in feed-forward blocks.
Normalization scheme	Pre-normalization	Layer normalization applied before Transformer sublayers.
Diffusion process		
Training diffusion timesteps	100	Number of forward diffusion steps used during training.
Inference denoising steps	5	Number of reverse denoising steps used during inference.
Noise schedule	Squared cosine	Schedule used to define the diffusion noise variance.
Prediction type	ϵ -prediction	The network predicts the Gaussian noise added to the clean action trajectory.
Primary loss	Masked MSE	Denoising loss computed only over valid action coordinates.
Auxiliary loss	Cumulative displacement loss	Regularizes the integrated displacement obtained from predicted deltas.
Cumulative loss weight	$\lambda_{cum} = 10^{-4}$	Weak regularization weight for trajectory-level consistency.
Dataset split and batching		

Parameter	Value	Description
Training split	90%	Portion of extracted windows used for training.
Validation split	10%	Portion of extracted windows used for validation.
Test split	Separate held-out episode	Final evaluation performed on an episode excluded from training and validation.
Random seed	Fixed	Used to make the training-validation split reproducible.
Batch construction	Same-episode sampler	Each mini-batch contains windows extracted from the same episode.
Training shuffling	Enabled	Windows and episode batches are shuffled during training.
Validation/test shuffling	Disabled	Deterministic ordering used during evaluation.
Normalization		
Normalization type	Gaussian scaling	Features are normalized using mean and standard deviation.
Statistics source	Training set only	Normalization parameters are computed only from training data.
Checkpoint storage	Enabled	Normalization parameters are saved with the trained model.
Optimization		
Optimizer	AdamW	Optimizer used for denoising-network training.
Batch size	128	Number of samples per mini-batch.
Training epochs	3000	Maximum number of training epochs.
Initial learning rate	10^{-4}	Initial optimizer learning rate.
Minimum learning rate	10^{-6}	Lower bound reached by the learning-rate schedule.
Weight decay	10^{-4}	Regularization applied to model parameters.
AdamW betas	(0.95, 0.999)	Momentum parameters of the AdamW optimizer.

Parameter	Value	Description
Gradient clipping	1.0	Maximum gradient norm allowed during optimization.
Validation and model selection		
EMA decay	0.995	Decay factor used for the exponential moving average.
Validation frequency	Every 5 epochs	Interval between validation runs.
Checkpoint frequency	Every 15 epochs	Interval between saved checkpoints.
Early stopping patience	50 validation checks	Number of validation checks without improvement before stopping.
Equivalent patience	250 epochs	Since validation is performed every 5 epochs.

List of Figures

2.1	Main levels of human–robot collaboration: coexistence, synchronization, cooperation and collaboration. Reproduced from Bauer <i>et al.</i> [47].	6
2.2	Taxonomy of variable impedance control and learning approaches. Adapted from Abu-Dakka and Saveriano [1].	11
2.3	Cues for human intention recognition in industrial collaborative robotics. Adapted from Kekana <i>et al.</i> [21].	14
3.1	High-level architecture of the proposed methodology. The offline pipeline (top) processes human demonstrations to train the diffusion model, while the online pipeline (bottom) uses the trained policy to predict Cartesian increments for the impedance controller.	23
3.2	3D keypoint extraction pipeline from RGB-D recordings. The flow is split into two branches: the RGB frame is used to estimate 2D keypoints via YOLO, while the aligned depth map provides the depth information needed to project the points into the camera’s reference frame. The resulting 3D trajectories are finally saved in CSV format.	25
3.3	Original camera reference frame	27
3.4	Preprocessing pipeline used to transform raw RGB-D keypoint recordings into training samples for the diffusion policy. Absolute trajectories are organized, augmented, converted into Cartesian deltas, represented through NaN-aware tensors and validity masks, split into episodic temporal windows and normalized with masked feature-wise statistics.	29
3.5	Training window with observation-action overlap. The conditioning window in dark blue contains T_{obs} past observations, whereas the target action sequence in light blue includes an overlap region O followed by a future prediction horizon H	33

3.6	Transformer denoiser architecture used inside the conditional diffusion policy. The conditioning branch embeds the normalized observation tokens and their validity masks, prepends the sinusoidal diffusion-step embedding, and adds learnable positional embeddings before the Transformer encoder. The action branch embeds the noisy action trajectory and adds action positional embeddings before the Transformer decoder. The decoder performs masked self-attention over the action tokens and cross-attention over the encoded conditioning memory. The output projection maps the decoder features to the predicted Gaussian noise $\hat{\epsilon}_\theta$	39
3.7	Training pipeline of the conditional diffusion policy. Episodic demonstrations are converted into temporal windows and organized into conditioning sequences and clean normalized action trajectories. At each iteration, a sampled diffusion timestep and Gaussian noise corrupt the clean action trajectory through the DDPM forward process. The Transformer denoiser predicts the added noise from the noisy trajectory, timestep, and conditioning sequence. The masked diffusion loss and auxiliary cumulative delta loss are combined and optimized with AdamW, while an exponential moving average of the model parameters is maintained.	44
3.8	Structure of the low-level impedance-admittance controller. The diffusion policy and the admittance dynamics generate incremental Cartesian corrections that update a shared persistent target. Cartesian impedance maps the reference-tracking errors into a task wrench, which is converted into joint torques and combined with nullspace and Coriolis terms before torque-safety filtering.	48
3.9	Online inference and execution pipeline. The RGB-D perception module extracts the leader keypoints and constructs the conditioning sequence. The diffusion policy predicts a sequence of Cartesian increments in the model frame, from which the first non-overlapping future increment is selected. The selected increment is mapped into the robot frame, bounded and published to the low-level controller. When it becomes available, the controller applies the diffusion increment once and combines it with the admittance contribution in the recursive update of the desired Cartesian position.	55

3.10	ROS2 communication graph of the online robotic system. The diffusion increment is transmitted from the high-level node to the low-level controller through <code>/delta_diffusion</code> , while <code>/actual_ee_pose</code> provides end-effector feedback in the opposite direction. Dashed connections represent direct hardware interfaces rather than ROS2 topics.	60
4.1	Offline human-human keypoint extraction used to build the training datasets.	63
4.2	Online human-robot perception output during the co-manipulation with the robot.	63
4.3	Ground-truth and predicted follower wrist increments along the x , y , and z directions for <code>Manipulation1</code>	66
4.4	Ground-truth and predicted follower wrist increments along the x , y , and z directions for <code>Manipulation2</code>	66
4.5	Ground-truth and predicted follower wrist increments along the x , y , and z directions for <code>Manipulation3</code>	67
4.6	MuJoCo collaborative manipulation scenario.	68
4.7	Comparison between the trajectory executed by the simulated robot and the human follower trajectory recorded in the dataset.	70
4.8	Cumulative contribution of the diffusion policy and admittance correction during the representative real-robot manipulation.	74
4.9	Comparison between the mean leader-wrist displacement and the measured end-effector displacement during the representative real-robot manipulation along the y direction.	75
4.10	Tracking of the impedance reference by the measured end-effector pose during the representative real-robot manipulation.	76
4.11	Measured end-effector trajectories for the five experimental tasks under the admittance-only baseline and the diffusion-assisted pipeline.	80
4.12	Example of the normalized interaction-force comparison between the admittance-only (LL-only) baseline and the diffusion-assisted pipeline (Diffusion+LL) in task T3.	82
4.13	RMS interaction force for the five experiments under the admittance-only and diffusion-assisted conditions.	82
4.14	Peak interaction force for the five experiments under the admittance-only and diffusion-assisted conditions.	83

List of Tables

3.1	Default parameters of the real-robot low-level controller. EWMA denotes an exponentially weighted moving average filter.	53
4.1	Published COCO-Keypoints validation performance of the YOLO26 pose models used in the perception pipeline [45].	62
4.2	Composition of the three held-out test episodes used to evaluate the model. A valid target value denotes one non-missing scalar coordinate included in the metric computation.	64
4.3	Offline prediction metrics. Errors are computed on Cartesian wrist-motion increments.	65
4.4	Coordinate-wise prediction error of the model, pooled over the three held-out test sequences.	67
4.5	Trajectory reproduction and intent-estimation results in simulation.	72
4.6	Summary of the representative real-robot manipulation experiment.	77
4.7	Tasks performed during the real-robot comparative study.	78
A.1	Leader keypoints used in the observation vector. The left and right labels follow the COCO anatomical convention.	97
A.2	Complete training configuration used for the conditional diffusion policy.	98

Acknowledgements

Al termine di questo percorso desidero dedicare qualche parola alle persone che, in modi diversi, hanno contribuito alla realizzazione di questa tesi e al raggiungimento di questo importante traguardo.

Desidero innanzitutto ringraziare il Professor Loris Roveda per avermi dato l'opportunità di svolgere questa tesi e di lavorare su un progetto stimolante e innovativo, che mi ha permesso di crescere sia dal punto di vista accademico che personale.

Un ringraziamento va a Charbel, che mi ha seguito durante tutto il lavoro di tesi. Grazie per la pazienza, la disponibilità e il tempo che mi hai dedicato. Le discussioni, i consigli e il confronto continuo sono stati fondamentali per la riuscita di questo lavoro.

Il ringraziamento più grande è rivolto ai miei genitori, ai quali devo molto più di quanto queste righe possano esprimere. Grazie per avermi sostenuto in ogni scelta, per aver sempre creduto nelle mie capacità e per avermi insegnato il valore dell'impegno e della costanza. In questi anni siete stati il mio punto di riferimento, incoraggiandomi a dare sempre il massimo senza mai farmi mancare il vostro affetto. Ogni traguardo raggiunto porta con sé anche una parte del vostro sacrificio, del vostro sostegno e della vostra fiducia. Questa laurea è anche vostra.

Ringrazio di cuore la mia sorellina Elisa, che con il suo modo di essere è sempre riuscita a strapparmi un sorriso nei momenti più stressanti. Tra uno scherzo e una battuta mi hai aiutato ad affrontare le difficoltà con maggiore serenità, rendendo questo percorso più leggero.

Un pensiero speciale va ai miei nonni, Antonella, Carlo, Maria e Roberto, che con il loro affetto e la loro presenza hanno sempre saputo farmi sentire il calore della famiglia. Grazie per l'incoraggiamento, per l'affetto con cui avete seguito questo percorso e per i valori preziosi che mi avete trasmesso nel tempo.

Un grazie speciale va a Ilaria. Grazie per essermi stata accanto ogni giorno, per aver condiviso con me sia i momenti di entusiasmo che quelli più difficili, per aver sopportato lo stress e persino le notti in cui, mentre dormivo, ti parlavo di idee su come migliorare i

progetti. Sei sempre riuscita a incoraggiarmi e a spronarmi a dare il meglio di me, e hai gioito per ogni mio piccolo traguardo. La tua pazienza, il tuo affetto e la tua fiducia sono stati un sostegno fondamentale.

Desidero ringraziare anche i miei amici Alessandro, Antonio, Lorenzo, Lorenzo e Riccardo. La vostra amicizia rappresenta una parte importante della persona che sono oggi. Insieme abbiamo condiviso esperienze, risate e tantissimi ricordi che porterò sempre con me. Grazie per esserci stati nei momenti importanti e per avermi ricordato, anche nei periodi più impegnativi, che c'è sempre tempo per un sorriso, una chiacchierata o una serata insieme.

Infine, un pensiero speciale va ai miei amici, i cosiddetti "Cavalli del Poli": Andrea, Damiano, Daniele, Dario, Davide, Denis, Elia, Gioele, Luca, Matteo e Stefano. Grazie per tutte le innumerevoli ore trascorse insieme tra studio, progetti, esami e serate a Bovisa. Insieme abbiamo affrontato difficoltà, condiviso risate e trasformato il Politecnico in una seconda casa. Il vostro supporto e la vostra amicizia hanno reso questi anni un'esperienza unica che porterò sempre con me.

Concludo ringraziando tutte le persone che, anche senza essere citate direttamente, hanno contribuito a questo percorso con il loro sostegno e vicinanza.

A tutti voi va il mio più sincero grazie.